

ctBurton: A Constraint-based Generative Planner with Flexible Time

Brian Williams

WILLIAMS@MIT.EDU

David Wang

DAVIDCW@MIT.EDU

*Computer Science and Artificial Intelligence Laboratory
Massachusetts Institute of Technology
Cambridge, Massachusetts 02139*

Abstract

The control of networks of interacting devices requires planning algorithms that account for devices that perform automatic timed transitions, that generate plans for meeting goals over time and that schedules events to meet deadlines. Consider a planner for an imaging satellite with a camera that cannot tolerate engine exhaust. The planner would need to determine that opening a valve causes a chain reaction that ignites the engine, and thus needs to shield the camera. While planners have been developed that support deadlines and time-evolved goals, no current planner can handle automatic timed transitions. In addition, many networked systems are controlled robustly by adjusting the duration of their control actions. However, most state-of-the-art PDDL temporal planners assume that execution times are fixed, and do not model uncertainty in the duration of actions. We present ctBurton, a constraint-based temporal planner that performs combined activity planning and scheduling, while supporting each of these features.

We formalize the problem of combined planning and scheduling of concurrent processes by introducing the concept of parameterized timed automata and then solve this problem with a constraint encoding based on mixed logic temporal problems. First, we generalize the traditional timed automata model to a new action model, called *parameterized concurrent timed automata (PCTA)*, which allows the duration between transitions to be both controlled and uncertain, and extends to automated transitions. Second, we define the execution semantics of PCTA in terms of traces and runs. Third we introduce *state plans* as a commonsense plan-like specification of goals over time, whose meaning is grounded in allowable runs. We then specify plan correctness in terms of the relationship between trace, run and state plan. Finally, we present the constraint-based planner ctBurton, which solves this PCTA planning problem. ctBurton encodes this problem using a continuous-time analogue to a planning graph, called a constraint-based trellis, which represents all runs of a PCTA over a finite horizon. This graph is translated to a mixed logic temporal problem, expressed in MiniZinc, which is solved using OpSat or another suitable MiniZinc solver. We evaluate ctBurton's performance on real-world problems and compare it to existing temporal planners, using benchmarks from the International Planning Competition. Finally, we show how this problem formulation and planner can be extended to support both interval-based and probabilistic models of temporal uncertainty.

1. Introduction

Embedded machines are being composed into ever more complex networked systems, including Earth satellite and insitu ocean observing systems and transportation networks. The complexity of these systems requires automated coordination. The rich behaviors of these systems pose unique challenges with respect to modeling, planning and control. We consider five challenges: first, is to model *timed transitions* – after turning a projector off, a cool-down period must be obeyed before

the projector turns on again; second, *periodic transitions* – a satellite’s orbit provides predictable time-windows over which it is able to observe a phenomenon, and, third, *required concurrency* [12] – a communication channel must be open for the duration of a transmission. Fourth, an operator may wish to specify the desired behavior of these systems in terms of when different states need to be achieved over time, called *time-evolved goals*. Finally, for many real-world planning problems, the duration of activities can be varied. An action duration can often be controlled, for example by using speed to adjust arrival time. Adjustments to this timing can improve feasibility, robustness and optimality. In addition, uncontrolled effects can lead to uncertain duration, such as the influence of traffic and weather on travel time. In this paper we frame a hybrid activity planning and scheduling problem, and describe a planner, that supports the temporal features listed above and ensures correctness with respect to a model of temporal disturbance.

While there is a long history of temporal activity planners that address a subset of these features, no single planner supports this complete set of features within their action model, their goal-specification and their solution.

For example, timeline planners, such as Europa [18] and Aspen [7], express a rich set of constraints on metric time and resources, and support combined planning and scheduling, but traditionally depend on domain-specific heuristics in order to efficiently guide backtrack or repair-based search. Some model-based planners, such as Burton [39, 8], the namesake of our planner, uses concurrent automata action models and exploits the causal structure of the action model to perform fast generative planning. While this type of planner does not need domain specific heuristics, it lacks the ability to reason over metric specifications of time. Finally, metric temporal PDDL planners based on heuristic forward search [11, 5, 32] are domain-independent, fast and scalable, but often do not allow for flexibility in the duration of activities, do not account for uncertainty in actions and lack support for several important features listed above, such as timed and periodic transitions.

In contrast, ctBurton is a fast temporal activity planner and scheduler that is designed to control concurrent processes, such as networked devices and multi-agent teams. Our contribution is four fold: first, we introduce the problem of planning as the control of concurrent processes, while supporting a set of goal, model and solution features that are not found previously in a single formulation. Second, we introduce a new formal model called parameterized time automata, which offers a clean action model for combining scheduling with planning. Third, we introduce ctBurton, which achieves efficiency for this problem by introducing a new constraint-based encoding using a continuous-time constraint-based trellis and a mixed logic temporal constraint programming. Fourth, we demonstrate the effectiveness of this approach on planning benchmarks against the state of the art, most notably those benchmarks with required concurrency. Finally, we extend this problem formulation and planner to support both interval-based and probabilistic models of temporal uncertainty, including simple temporal networks under uncertainty (STNUs)[36] and chance-constrained probabilistic simple temporal networks (cc-PSTNs)[16, 37, 15].

The remainder of this paper is organized as follows. First we define ctBurton’s deterministic problem and uncertain invariants in terms of parameterized timed automata, traces, runs and state plans, and solution correctness in terms of these concepts. Parameterization is the key concept that allow us to introduce temporal flexibility and uncertainty into our action model. Next, we present ctBurton’s encoding as a mixed logic temporal constraint programming problem. Finally, we close with an empirical validation of ctBurton, compared to state of the art temporal planners, on real-world and International Planning Competition (IPC) benchmarks.

2. Problem Formulation

In this section we present a formal statement of the concurrent temporal planning and scheduling problem that ctBurton solves.

2.1 PCTA Planning Problem

ctBurton takes as input an action model, expressed as a parameterized variant of a concurrent, interacting timed automaton (*PCTA*), an initial state and a goal specification, which imposes constraints on the desired state evolution of the automaton. ctBurton outputs a trace of *PCTA*, called a *control plan*, which describes a control input to the *PCTA* that satisfies the goal specification when executed from the initial state. In terms of planning, the trace encodes a grounded, concurrent activity plan and its schedule.

Definition 1. *PCTA State Plan Planning Problem*

A PCTA state plan planning problem $Plan_{PCTA} = \langle A, e_0, s_0, SP_{goal}, h \rangle$ is comprised of a parameterized concurrent timed automaton A , start event e_0 , initial state s_0 , goal specification SP_{goal} and planning horizon h . A is determinate and its transitions are specified in canonical form. e_0 is a real-valued event. s_0 is a (full) state assignment of A at e_0^+ . SP_{goal} is a state plan that includes start event e_0 and constrains the location and control variables of A . h is the maximum number of state changing events that may occur in the solution run.

An end-state planning problem is a PCTA state plan planning problem whose state plan is specified by $SP_{goal} = \langle c_g, c_s[l, u] \rangle$ where c_g and c_s are state constraints, c_g holds at the end of the plan e_e , c_s holds up to e_e , and the end goal is achieved within $e_e - e_0 \in [l, u]$.

The solution to $Plan_{PCTA}$ is a valid control plan in the form of a trace of A together with its run, or a conflict if no such plan exists. Both the trace and run are described by time-stamped sequences and the number of time stamps that appear in each is bound by horizon h . A control plan is valid with respect to $Plan_{PCTA}$ if its run, starting in s_0 at e_0 , satisfies SP_{goal} . A control plan is optimal with respect to $Plan_{PCTA}$ if no valid control plan exists whose run has a higher reward. A conflict of $Plan_{PCTA}$ is a sub-plan SP'_{goal} of SP_{goal} such that $\langle A, e_0, s_0, SP'_{goal} \rangle$ has no valid control plan. $\square$

Three concepts in this problem statement are key: first is the concept of parameterized concurrent timed automaton (PCTA), which provides a concurrent action model, and permits a planner to use clock parameters to adjust the duration of these actions. Second is the concept of a trace, together with a run, which specify an output control plan and its execution. Third is the concept of state plan, which is used to represent goals over time. We provide an overview of these concepts in this section and then define them in subsequent sections.

First, ctBurton's PCTA action representation builds upon concurrent automata, similar to the original Burton planner [39]. The key difference is that for PCTA the automata are timed and parameterized. The planning community employs several factored automata-like representations, including SAS+ [2] and domain transition graphs [20]. In addition, concurrent automata are often used by the embedded systems community to factor systems that exhibit strong, required concurrency. This is the target domain of both Burton and ctBurton. A timed automaton, in particular is an elegant, pervasively used model for timed systems, and a building block for our work. However timed automata fix the duration of activities in their clock conditions and do not offer hooks that decision making procedures can use to adjust these timing. The key distinguishing feature of a PCTA

is its addition of clock parameters to timed automata, which we use to support combined activity planning and scheduling.

A PCTA consists of input control variables and concurrently operating timed automata. Each component automaton A consists of sets of locations and transitions between these locations. A 's "state" is captured by the combination of a location variable, which points to A 's current location, and a clock variable, which counts the time since A last entered a new location. The state of a PCTA is then the combined state of all its components.

To impose metric constraints on time, each of A 's transitions has a guard that may be conditioned on its clock variable. This is in addition to A 's inputs, which are state variables of other component automata. An automaton transitions when the guard of one of its applicable transitions is satisfied. A transition may restart a clock variable whenever a new location is entered.

Traditionally, timed automata fix the time of transition to be a constant, and as a result the amount of time that an automaton resides in a location. In order to support scheduling of events and the amount of time that is spent idling in a location, A PCTA allows the time of A 's transitions to be adjusted, by specifying clock conditions that use adjustable clock parameters, rather than constants, and by setting the values of clock parameters as part of the hybrid planning and scheduling problem.

A clock parameter may be reset, similar to a clock variable, when a transition is performed, with the result of establishing a new cutoff for a clock condition, hence a new duration for a future activity. A clock parameter reset is non-deterministic in that it specifies a range of possible values. The value for a clock parameter is then selected from this range after each reset, by a scheduling policy.

Second, a trace describes an input trajectory of a PCTA as a sequence of changes to input values and their time of change. The trace includes a sequence of clock parameter values, which specify the scheduled duration that the automaton resides in particular location. A run describes the behavior of a PCTA in response to an initial state and a trace. The run is a sequence of location changes plus a sequence of clock resets.

To describe traces and runs syntactically we use *timed sequences*, which are comprised of timed states or timed transitions. Traces and runs over multiple input and state variables can be partitioned into concurrent traces and runs, specified over partitions of the input and state variables. The solution to a planning problem is a trace, called a control plan $SP_{control}$, whose run satisfies SP_{goal} when executed at initial state s_0 .

Finally, ctBurton replaces a goal state, used in classic planning, with a *state plan*. The planning and logic-based control communities often turn to temporal logic as a rich formalism for describing goals over time. ctBurton could use a variant of metric temporal logic, adjusted to support flexible metric temporal constraints and uncontrollable duration. However, using temporal logic is not easy for many laypersons, was defined in particular for expressing liveness and safety properties, and is more expressive than is often needed for specifying goals over time.

ctBurton supports state plans as a more restricted but intuitive formalism for expressing goals. A state plan is similar to a least commitment temporal plan, but constrains state evolution rather than action. The motivation for a state plan is that a user will often have a sketch in mind for the behavior they want, but have not fleshed out the details. These sketches tend to look plan-like, except that actions are replaced with goals. State plans have been used extensively by continuous, hybrid and discrete state planners, such as Sulu[26], Chekov[21, 22], tBurton[38], Kongming[27], and cKongming[6]. A state plan might be generated by a user, a planner that operates at a higher level, or a program.

The result of invoking ctBurton on a state plan is to instantiate the state plan in the form of a trace that is executable on the problem's PCTA. Traditionally a temporal plan consists of a set of activities, where each activity includes an executable command, such as "put(A,B)", along with two events, corresponding to the time that the command starts and ends. Start and end events of different activities are related through temporal constraints. A state plan replaces an activity with an *episode*, which retains start and end events, but replaces the executable command with constraints on an automaton's control and location variables. An example constraint might be "above(A,B)". Similar to activity plans, temporal constraints are used to coordinate between state plan episodes. Together, the episode state constraints and temporal constraints restrict the evolution of automaton A's control and location variables.

In subsequent sections we define the major elements of this problem, including parameterized timed automata, trace and run, control plan and state plan. We follow this with the development of our planner. We finish this section with an example followed by an overview of our approach to modeling uncertainty.

2.2 Projector Example

To illustrate our definitions we introduce the problem of planning to control a presentation system, which is comprised of a computer, a projector, and a connection between them. The computer takes time to boot-up and shutdown. The projector, likewise, takes time to warm up and cool down. In addition, the projector shuts down automatically when disconnected from the computer. Figure 1 models this system as a timed concurrent automaton.

Intuitively, a timed concurrent automaton is simply a set of state machines that run at the same time, one for each component: Computer, Connection and Projector. The automata are timed in that the automata transitions are allowed to depend on the current time relative to entering a particular state. Returning to Figure 1, the automaton for the computer has four "states", which include "On" and "Off" and two transitional states, "Booting" and "Shutdown". The transitional states represent the time taken while moving from Off to On and On to Off, respectively. The arc from On to Shutdown is labeled with "cmd = turn-off", indicating that a turn-off command initiates the shutdown process. The constraint $c < 30$, which labels an idling transition on Shutdown, indicates that, when the computer enters Shutdown, it will be in that state for 30 seconds. Variable c denotes the value of a clock associated with the computer, which represents the passage of time from when the state Shutdown is entered. The arc from Shutdown to Off, labeled " $30 \leq c$ ", indicates that the computer transitions to the off state at the end of 30 seconds. A computer also consumes power. Today an average desktop computer consumes 100 Watts when On, around 10 Watts when Off, but sleeping. Perhaps the computer consumes more energy, say 150 Watts, when booting up and shutting down.

The automaton for Connection is similar, but simpler in that it switches between the connected and disconnected states without delay. The Projector automaton has the added complexity that the turn-off command needs to be held for at least two seconds, to initiate cool down. This wait for two seconds is implemented with the Confirm state. In addition, the projector shuts down whenever the computer is off or disconnected for at least a minute, and is implemented with the Wait state.

A planning problem includes an initial state and goals to achieve over time. Suppose we walk into a conference room, first thing in the morning, and we want to give a short presentation. Then our initial state might be that the computer and projectors are Off and disconnected from each other.

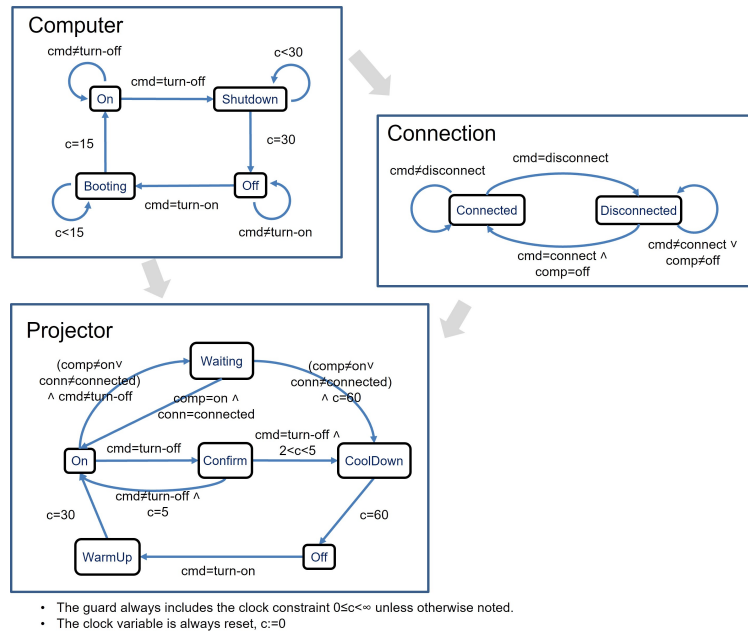


Figure 1: A simple computer-projector system represented as a Timed Concurrent Automaton, with input *cmd* and three primitive automata - computer, connection and projector. Idling transitions are left implicit.

Update transitions by

- Draw figures using draw.io.
- adding clock resets,
- removing conditions on locations,
- adding clock parameters,
- adding reward, and
- adding idling transitions.

A typical state plan might involve having the projector on within a minute, keeping it on for at least 30 minutes, then having the computer and projector disconnected afterwards.

Achieving this state plan is simple. The computer and projector can be immediately commanded on and commanded to be connected. The connection is immediate. The computer boots for 15 seconds, after which it is on, while the projector warms up for about 30 seconds and then comes on. No further commands are given for 30 minutes, until the presentation is over, at which time they are commanded to disconnect.

In many domains, the duration of some activities can be controlled. For example, we can control how fast a car arrives at its destination by adjusting its speed. In our example, suppose we can adjust the time that the computer boots up, to be within the range of 10 to 15 seconds. To control this duration we introduce a clock parameter c_p to our PCTA. We use control parameter c_p to end the Booting process by replacing condition $15 \leq c$, on the transition from Booting to On, with $c_p \leq c$. Finally, to restrict the range of c_p to be between 10 and 15 seconds, we add a clock parameter reset $c_p \in [10, 15]$ to the computer transition from Off to Booting. This reset is similar to a clock reset, except that it represents a non-deterministic choice for c_p from the interval $[10, 15]$, which can be made by a scheduler. A PCTA includes a trace for each clock parameter, as well as its input traces. A clock parameter trace changes its value each time the parameter is reset and maintains its value between resets.

2.3 Temporal Planning with Uncertain Durations

Most temporal planning problems in the real world contend with activities whose duration is uncertain. For example, it is likely that Booting the computer takes 10 to 15 seconds, but is not under control of the user. For a temporal plan to be robust to this uncertainty, plan correctness needs to be evaluated with respect to the possible outcomes of these uncontrolled durations.

Uncertain durations have been formalized extensively for scheduling problems, but not for problems that involve concurrent planning and scheduling. In this section we introduce uncertain duration into our temporal planning problem. We elaborate upon this definition and our probabilistic planner encoding during the remainder of this paper.

Within the fields of decision-making under uncertainty, such as scheduling, optimization and control, uncertainty is typically modeled either as a set of possible values, called a *set-bounded* approach, or as distributions over uncertain values, called a *probabilistic* approach. For scheduling under uncertainty, durations are split into *controllable durations*, which are set by the user or a decision procedure, and *uncontrollable durations*, which are set by nature. A commonly used set-bounded extension to simple temporal networks is called a *Simple Temporal Network under Uncertainty (STNU)* [36] and a commonly used probabilistic extension is called a *Chance-Constrained Probabilistic Simple Temporal Network (cc-PSTN)* [15].

In a STNU, an uncontrolled duration is specified by an interval range, such as 10 to 15 seconds. An STNU can be scheduled, called *strongly controllable*, if there exists a schedule to the controlled durations that is consistent with the temporal constraints of the STNU for every assignment to controlled durations that fall within their specified intervals.

In a cc-PSTN, an uncontrolled duration is specified by a probability distribution, such as a normal distribution centered at twelve and a half seconds and a two sigma deviation of two and a half seconds. A chance-constraint specifies a maximum acceptable risk of violating the simple temporal constraints, given possible assignments to uncontrolled durations. This risk bound acknowledges

the fact it is often not possible to satisfy a set of temporal constraints with probability one when the distributions are unbounded, as is the case for normal distributions.

Mirroring these two approaches, we provide two definitions for temporal planning under uncertainty, called the *PCTA Planning Under Uncertainty Problem* and the *Chance-Constrained PCTA Planning Problem*. In both formulations we split clock parameters into controllable and uncontrollable clock parameters. Controllable clock parameters behave the same as for deterministic *PTA*. When a controllable parameter is reset, the planner assigns it a value from the range specified by the reset. Uncontrollable clock parameters are also reset to range of possible values, but the value selected by the reset is assigned by nature, not the planner. While controlled clock parameters have a single trace, uncontrollable clock parameters have a set of traces, corresponding to the different outcomes of nature.

A single trace for a each clock parameter is then replaced by a set of possible traces that the uncontrollable clock parameter might manifest. Planning under uncertainty evaluates correctness with respect to this set of outcomes, and their resulting traces.

2.3.1 SET-BOUNDED FORMULATION

The PCTA planning under uncertainty problem uses a set bounded model of uncertainty, and replaces the PCTA within a deterministic planning problem with a PCTA under uncertainty (PCTAU). A PCTAU adds uncontrolled clock parameters to the controlled clock parameters. An uncontrolled clock parameter is reset to a value within an interval, similar to a controlled clock parameter. These controlled and uncontrolled parameters are analogous controlled and uncontrolled durations in an STNU. Returning to our computer example, we might treat c_p as an uncontrolled parameter. The reset $c_p \in [10, 15]$ then specifies that booting could take anywhere between 10 and 15 seconds.

The output of a PCTAU planning problem is called an *emphcontrollable trace* and specifies trajectories to the input and controllable clock parameters. An outcome extends a controllable trace with a trace for each uncontrolled clock parameter. An outcome must produce a consistent run. The set of all outcomes corresponds to the set of possible choices that nature can make. A controllable trace is then a solution if every one of its outcomes satisfies the problem's state plan. This is analogous to an STNU, where a schedule to controllable durations is valid (called *strongly controllable*) if the schedule is consistent with the STNU for all feasible assignments to uncontrolled durations.

For the projector problem, the solution that we introduced for our deterministic formulation is also a solution when we introduced an uncertain interval for booting time. The state plan specifies that the computer and projector need to be ready within a minute. The proposed solution initiates the connection and powers the projector and computer immediately. The computer takes 10-15 seconds to move to On, which is less than the required minute. The only difference between the two solutions is that the PCTA solution has a single trace for the computer clock parameter, which is controllable. The PCTAU solution treats the computer clock parameter as uncontrollable, and contains all possible traces to this parameter that are consistent with the range $[10, 15]$.

We define the PCTAU planning problem below, while we define PCTAU and its trace in subsequent sections.

Definition 2. PCTA Planning Under Uncertainty Problem

A PCTA planning under uncertainty problem $Plan_{PCTAU} = \langle A, e_0, s_0, SP_{goal} \rangle$ is a PCTA planning problem in which A is a PCTA under uncertainty (PCTAU). The solution to $Plan_{PCTAU}$ is a valid control plan in the form of a controllable trace T_C of A together with its stochastic run, start-

ing in s_0 at e_0 , or a conflict if no such plan exists. A controllable trace T_C is valid with respect to $Plan_{PCTAU}$ if its stochastic run, starting in s_0 at e_0 , satisfies SP_{Goal} . T_C is optimal with respect to $Plan_{PCTAU}$ if no valid control plan exists whose stochastic run has higher minimum reward R_{min} . $\square$

2.3.2 PROBABILISTIC FORMULATION

A chance-constrained PCTA planning problem is a probabilistic formulation that replaces the PCTA with a *probabilistic PCTA* (pPCTA). A pPCTA describes possible values of an uncontrolled clock parameter using a probability distribution, rather than an interval. Common distributions are normal distributions and exponential distributions. These distributions are specified for each reset of an uncontrolled clock parameter. Upon reset, the value of an uncontrolled clock parameter is drawn from the reset's distribution and the distributions of distinct resets are independent. Possible values of controlled clock parameters are still specified by intervals as is the case for PTA and PTAU.

Returning to the projector example, we include clock parameter c_p as a stochastic variable and we replace parameter reset $c_p \in [10, 15]$ with a normal distribution, say $c_p \in N[12.5, 1.25]$. The tails of this distribution drop off quickly below 10 and above 15.

A pPCTA has the concept of controllable and uncontrollable traces, analogous to a PCTAU. The difference is that a probability distribution is defined over the uncontrollable clock parameter traces of the pPCTA. A controllable input trace produces a distribution over its outcomes. This distribution is over uncontrolled clock parameter traces and is the result of the resets of the uncontrolled clock parameters. Given that reset distributions are independent, the probability of an uncontrolled trace is simply the product of the probability of the assignment the trace makes at each reset. The projector example invokes just one parameter reset for the computer, hence the distribution over the computer parameter trace is the same normal distribution as specified for the reset.

Often we can not generate a plan for a pPCTA problem that always achieves the problem's state plan. The reason is that normal and exponential distributions have infinite tails and all delays are possible, albeit unlikely. A pPCTA planning problem specifies a maximum acceptable risk Δ that the run of a solution violates the problem's state plan. This is called a chance constraint. Returning to our projector example, the state plan specifies that the computer and projector are on within a minute. Suppose our acceptable risk of failure is at most 5 percent ($\Delta = .05$). Our proposed solution of turning on the computer and projector immediately easily satisfies this chance constraint.

We define the pPCTA planning problem below, while we define pPCTA and its trace in subsequent sections.

Definition 3. Chance-constrained PCTA Planning Problem

A chance-constrained PCTA planning problem $Plan_{pPCTA} = \langle A, e_0, s_0, SP_{goal}, \Delta \rangle$ restricts automaton A to a probabilistic PCTA (pPCTA) and extends the PCTA planning problem to include a risk bound $\Delta \in [0, 1]$.

The solution to $Plan_{pPCTA}$ is a valid control plan in the form of a controllable trace T_C of A together with its stochastic run, starting in s_0 at e_0 , or a conflict if no such plan exists. A controllable trace T_C is valid with respect to $Plan_{pPCTA}$ if its stochastic run, starting in s_0 at e_0 , satisfies SP_{Goal} within risk Δ . T_C is optimal with respect to $Plan_{pPCTA}$ if no valid control plan exists whose stochastic run has higher expected reward $R(T_C)$. $\square$

3. Parameterized Timed Models

This section we define concurrent parameterized timed automaton, the first element of our Planning Problem (Definition 1). Planning is about controlling the time-evolved behavior of a system. In this paper we describe time-evolved behavior of *state variables* and their associated trajectories over time.¹ Our projector example includes state variables that describe the state of the computer, projector and connection over time, and the value of *cmd*, which is the control input.

3.1 Parameterized Timed Automata

We employ a discrete-state, continuous-time model for these state trajectories, meaning that each state variable takes on one of a finite set of values, and each value of a state variable holds over a continuous interval of time, where the interval length can be different between successive pairs of values. For example, we describe the computer as being in the Off state for 3 minutes, then Booting for 15 seconds, and On for 30 minutes.

Definition 4. State Variable

A state variable v is a variable whose assignment is a continuous timed trajectory $v(t) : \mathbb{R} \rightarrow R(v)$, where $R(v)$ denotes the range of values that trajectory $v(t)$ can take on at Amazon point in time. $\square$

Our action model, a *Parameterized Concurrent Timed Automaton (PCTA)*, describes the evolution of these state variables using a variant of state machines. A *primitive timed automaton* describes the behavior of a single component or process, such as a projector, as described above. A system’s behavior, composed of multiple components, is described by a set of concurrently executing automata, each of which can be conditioned on the states of other automata. In our projector example, we describe three concurrently operating automata, whose transitions are conditioned on “cmd” and clock “c”, called a guard condition. In this example the automata are not conditioned on the states of other automata, but this is allowed.

Both primitive and concurrent timed automata are instances of the more general concept of *timed automata*. Timed automata use guard conditions on clock variables to specify when a transition occurs. For example, the computer automaton includes a transition from Shutdown to Off, which includes the condition “ $c = 30$ ”. This transition specifies that shutdown ends after 30 seconds.

A traditional timed automaton formalism does not allow the deadline of 30 seconds to be externally set by a decision procedure, it is fixed by the model. This is a significant limitation for real-world planning problems, which involves selecting actions and scheduling their duration.

A contribution of our model is that we enable the duration of activities to be controlled by generalizing timed automata to the new concept of *parameterized timed automata*, in which a parameter is a decision variable that is used to adjust each duration. For example, we replace the number 30 in the condition “ $c = 30$ ” with a parameter “sdn”, resulting in the condition “ $c = sdn$ ”, where “sdn” denotes shut down time. A decision procedure can then assign “sdn”, in order to adjust the duration of Shutdown.

More specifically, our work on PCTA planning builds upon prior work on planning using concurrent automata models, both untimed [40] and timed [24], in which component automata communicate through shared variables. As with this previous work, PCTA transition guards are expressions over control and automata variables that are stated in propositional state logic. A propositional state

1. An alternative term is ‘Fluent’, used within the AI knowledge representation community. State variable has more broad usage.

logic formula is a logical formula in which propositions are variable assignments and are combined using standard propositional connectives. These formulae are used extensively to specify guard conditions in the Projector model of Figure 1. This formulation of guard conditions is in contrast to the use of symbols from an alphabet, which is commonly used by timed automata models. Similar to timed automata, we use the term “location”, instead of the traditional term “state”, which is used for state machines and un-timed automata, or the concept of “mode”, which is used by the model-based reasoning community [19].

Like timed automata, we augment system state to include values of clock variables, which measure the progress of time. A clock variable measures the amount of time that an automaton resides in each of its locations. A timed automaton enforces ‘deadlines’ by resetting a clock variable whenever a new location is entered and by augmenting transition guards with conditions on this clock variable, thus specifying when the automaton can depart the transition’s source location. For example, a clock c might be reset ($c = 0$) when entering a location called “bake muffins” and a transition out of “bake muffins” might have the condition $c = 30$, specifying that the muffins are finished baking at 30 minutes. A transition can include a reset, such as “ $c = 0$,” which sets the clock to begin counting the time that is spent in the transition’s target location. This formalism is key to allowing us to model durative actions and to model behaviors with time delay.

It is fundamental to our work that we support the paradigm of temporally flexible planning and execution, in which robustness is improved by dynamically controlling the deadline that each behavior meets. We highlighted above that this is a challenge for timed automata. Timed automata are determinate, meaning that they specify a unique next state, and operate on a fixed schedule. In our baking example, the model fixed the deadline of baking to exactly 30 minutes. Timed automata do not expose a “control knob” that a decision algorithm can use to set these deadlines. *Parameterized timed automata* allow transition deadlines to be set. We are familiar with control variables, which offer knobs for adjusting state variables. By analogy, we introduce *clock parameters* as control knobs for setting the deadlines of clock variables. For example, we might introduce clock parameter p to represent baking time.

Similar to clock variables, a clock parameter can be reset upon transition into a location. A clock parameter differs from a clock variable in that the reset value of a clock parameter can take on a range of possible values, while the reset value of a clock variable is a constant. The range of values for a clock parameter reset represents a non-deterministic choice, whose value is chosen by a scheduling policy, some time during planning or plan execution. For example, a transition into location “bake muffins” might include the *clock parameter reset* $p \in [25, 35]$. This indicates that muffins can be baked for any range of times between 25 and 30 minutes, at the discretion of the decision maker.² Clock parameters can appear in clock conditions of transition guards in the place of constants. For example, consider the transition out of location “bake muffins”. We replace the constant 30 in $c = 30$ with clock parameter p , resulting in $c = p$.

2. For temporal planning or execution problems with temporal uncertainty, clock parameters can represent deadlines that are controlled by nature as well as by a decision maker. These “uncontrolled” values can be represented by an interval, similar to an STNU[36], or a distribution, similar to a pSTN[16].

3.1.1 DEFINITION OF PARAMETERIZED TIMED AUTOMATON

Next we turn to definitions. We start with the general concept of parameterized timed automaton, which describes the evolution of a continuous-time state variable. We then specialize this concept to primitive and concurrent automata.

Definition 5. Parameterized Timed Automaton (PTA)

A parameterized timed automaton A is described by a quintuple $\langle \mathcal{L}, \mathcal{C}, \mathcal{U}_I, \mathcal{U}_{I_C}, \mathcal{U}_P, \mathcal{T} \rangle$, where $\mathcal{L}$, $\mathcal{C}$, $\mathcal{U}_I$, $\mathcal{U}_{I_C}$, and $\mathcal{U}_P$ are functions of time:

- $\mathcal{L} : \mathbb{R} \rightarrow R(\mathcal{L})$ is the set of location variables of A . Each $l \in \mathcal{L}$ is a function of real-valued time with finite range $R(\mathcal{L}) = \{l_i\}$, representing locations over which this automaton transitions.
- $\mathcal{C} : \mathbb{R} \rightarrow R(\mathcal{C})$ is the set of clock variables of A . Each $c \in \mathcal{C}$ is a function of real-valued time whose range is the non-negative reals ($R(c) = \mathbb{R}_+$) and increases with time ($dc/dt = 1$).
- $\mathcal{U}_I : \mathbb{R} \rightarrow R(\mathcal{U}_I)$ is the finite set of (exogenous) input variables of A . Each $u \in \mathcal{U}_I$ is a function of real-valued time with associated range $R(u) = \{u_i\}$. The elements of $R(u)$ are referred to as commands. When $\perp$ is included in this range, it is called the nullary command, and indicates that no command is given.
- $\mathcal{U}_{I_C} : \mathbb{R} \rightarrow R(\mathcal{U}_{I_C})$ is the set of input clock variables of A . Each $c \in \mathcal{U}_{I_C}$ is a function of real-valued time whose range is the non-negative reals ($R(c) = \mathbb{R}_+$) and increases with time ($dc/dt = 1$).
- $\mathcal{U}_P : \mathbb{R} \rightarrow R(\mathcal{U}_P)$ is the set of clock parameters of A . Each $u_p \in \mathcal{U}_P$ is a function of real-valued time and has an interval range $R(u_p) = [l, u] \subseteq \mathbb{R}_+$.
- $\mathcal{U}_{I_P} : \mathbb{R} \rightarrow R(\mathcal{U}_{I_P})$ is the set of input clock parameters of A . Each $u_p \in \mathcal{U}_{I_P}$ is a function of real-valued time and has an interval range $R(u_p) = [l, u] \subseteq \mathbb{R}_+$.

$\mathcal{L}$, $\mathcal{C}$, $\mathcal{U}_I$, $\mathcal{U}_{I_C}$, $\mathcal{U}_P$, and $\mathcal{U}_{I_P}$ are aggregated to:

- $\mathcal{V} = \mathcal{L} \cup \mathcal{C}$ is the set of state variables of A . An assignment $\mathcal{V}(t)$ is called the state of automaton A at time t .
- $\mathcal{U} = \mathcal{U}_I \cup \mathcal{U}_{I_C} \cup \mathcal{U}_P \cup \mathcal{U}_{I_P}$ is the set of control variables of A , comprised of input variables, input clock variables, clock parameters, and input clock parameters. An assignment $\mathcal{U}(t)$ is called the control assignment of automaton A at time t .
- $\mathcal{W} = \mathcal{V} \cup \mathcal{U}$ is the set of variables of A , comprised of state and control variables of A .

In addition,

- The transition relation $\mathcal{T} = \{\tau_i\}$ is described by a set of transition relations $\tau_i(\mathcal{V}(t-\epsilon), \mathcal{U}(t-\epsilon), \mathcal{V}(t+\epsilon), \mathcal{U}_P(t+\epsilon))$ as $\epsilon \rightarrow 0^+$. $\mathcal{L}(t-\epsilon)$ of $\mathcal{V}(t-\epsilon)$ and $\mathcal{L}(t+\epsilon)$ of $\mathcal{V}(t+\epsilon)$ denote source and target locations. $\mathcal{C}(t-\epsilon)$ of $\mathcal{V}(t-\epsilon)$ and $\mathcal{U}_P(t-\epsilon)$ of $\mathcal{U}(t-\epsilon)$ denote values at the time of transition of clock variables and clock parameters associated with the automaton. $\mathcal{C}(t+\epsilon)$ of $\mathcal{V}(t+\epsilon)$ and $\mathcal{U}_P(t+\epsilon)$ of $\mathcal{U}(t+\epsilon)$ denote reset values to clock variables and parameters.

- The reward function $\mathcal{R} = \{\rho_i\}$ is described by a set of rewards $\rho_i(\mathcal{V}(t - \epsilon), \mathcal{U}(t - \epsilon), \mathcal{L}(t + \epsilon)) \rightarrow \mathbb{R}_{\geq 0}$ as $\epsilon \rightarrow 0^+$, which map source and target locations, clock variable and parameter assignments to non-negative reward values, denoting reward per unit time. The value returned by ρ denotes reward per unit time. When its source and target are distinct, ρ_i returns the coefficient of a unit impulse $\delta(t)$.

□

Locations are the states of the automaton that it transitions. Clocks measure the progression of time while clock parameters are used to identify when a clock has progressed sufficiently to transition. The automaton has associated clock variables $\mathcal{C}$ and clock parameters $\mathcal{U}_P$ that it controls using transition resets. Transitions are controlled by inputs $\mathcal{U}$, as well as the automaton's clock variables and parameters. Inputs include input variables $\mathcal{U}_I$, which may be exogenous inputs, controlled externally, or location variables of other automata. An automaton may also take as input the clock variables $\mathcal{U}_{I_C}$ and parameters $\mathcal{U}_{I_P}$ of other automata.

Each transition relation τ_i corresponds to an arc in a state diagram, from a source to a target location. Each reward ρ_j corresponds to a value of reward that labels one of these arcs. Each τ_i has a corresponding reward specified by ρ_i .

A transition at time t is initiated just prior to t ($t - \epsilon$ as $\epsilon \rightarrow 0^+$) and reaches its target just after t , at $t + \epsilon$. We use t^- as short for $t - \epsilon$ and t^+ as short for $t + \epsilon$. We discuss this further in Section 3.2. Transitions with different start and end locations are called *state changing transitions*, while transitions that start and end at the same location are called *idling transitions*. We permit multiple transitions between the same start and end.

ctBurton requires automata to be determinate, meaning that that an automaton specifies a unique next state (location and clock) at every time step. As a result, transitions with different targets or resets can not be enabled at the same time, and explicit idling transitions need to be included that specify when locations do not change.

A reward ρ_i returns the reward that is received when ρ_i is enabled. ρ_i includes a source, target and guard condition, same as its corresponding transition τ_i , plus its reward value r . $r(\rho_i)$ denotes reward per unit time. To determine the reward of a run, $(R) = \{\rho_i\}$ is integrated over the continuous interval of the run. Note that, if a state changing transition had a finite rate of reward, its contribution to the reward of the run would be negligible, since the transition is instantaneous. Instead, we describe the reward of a state changing transition by an impulse, whose integral over $t + / - \epsilon$ is finite. The value specified for $r(\rho_i)$ of a state changing transition, is the coefficient of this impulse function; its integral is this coefficient.

3.1.2 COMPUTER AUTOMATON WITH FIXED DURATIONS

We illustrate a PTCA model using the computer state machine depicted in Figure 1. In this example the duration of the Booting and Shutdown activities (locations) is fixed. Hence we do not need to include clock parameters, and the parameterized timed automaton reduces to a traditional timed automaton. After we complete this example of a timed automaton. We introduced clock parameters, so that we can set the duration of the these activities as part of the decision problem.

A timed automaton specification for the computer (Figure 1) is given in Figure 2. We describe transitions through English phrases that map directly to the transition specification that we define in the next subsection. The Computer specification consists of a single location variable l_{cmp} and its associated domain {On, Shutdown, Off, Booting}, a single real-valued clock c_{cmp} , and a single

Computer Automaton with Fixed Duration

- Variables:
 - Location $l_{cmp} \in \{\text{On}, \text{Shutdown}, \text{Off}, \text{Bootng}\}$,
 - Clock $c_{cmp} \in \mathbb{R}_0^+$,
 - Input $cmd \in \{\text{turn-off}, \text{turn-on}\}$,
- State changing transitions:
 - On to Shutdown when $cmd = \text{turn-off}$ with reset $c_{cmp} = 0$,
 - Shutdown to Off when $c_{cmp} \geq 30$,
 - Off to Bootng when $cmd = \text{turn-on}$, with reset $c_{cmp} = 0$ with cost = 5Wh,
 - Bootng to On when $c_{cmp} \geq 15$.
- Idling transitions:
 - Idle on On when $cmd \neq \text{turn-off}$ with cost = 100W,
 - Idle on Shutdown when $c_{cmp} < 30$ with cost = 150W,
 - Idle on Off when $cmd \neq \text{turn-on}$ with cost 10W,
 - Idle on Bootng when $c_{cmp} < 15$ with cost = 150W.

Figure 2: Computer with fixed duration, specified by a parameterized timed automaton ($cmp = \langle \{l_{cmp}\}, \{c_{cmp}\}, \{cmd\}, \{\}, \mathcal{T} \rangle$). Clock parameters are not included since the duration of the Bootng and Shutdown activities are fixed at 15 and 30 seconds, respectively. Without clock parameters, the automaton reverts to a traditional timed automaton.

exogenous input cmd with domain $\{\text{turn-off}, \text{turn-on}\}$. The duration of Bootng and Shutdown are fixed and the clock conditions only involve constants (15 and 30 seconds), hence as just mentioned, clock parameters $\mathcal{U}_P$ are not needed.

The transition relation $\mathcal{T}$ for the Computer automaton includes four state changing transitions between locations and are described in the "state changing transitions" field of Figure 2.³ The first transition, for example, moves the computer from location On to Shutdown upon command turn-off. In addition, the transition resets the clock c_{cmp} to 0, in order to begin counting the duration that the computer resides in location Shutdown. Once the first transition starts the clock, the second transition ensures that the computer is in Shutdown for 30 seconds, before moving to Off. Note that the guard of the second transition says that the clock must be greater than or equal to 30 seconds, not equal to 30. In this case these are equivalent, as the transition will be taken at the first time that the guard condition is satisfied, which is when the clock equals 30 seconds. Likewise, the third and fourth transitions result in the computer being in the Bootng location for 15 seconds.

3. Note that we could reset the clock c_{cmp} upon entering *every* location. This is not needed for locations Off and On, since the time spent in these locations is not checked by the guards of their outgoing transitions.

$\mathcal{T}$ also includes four idling transitions, which are described by the "idling transitions" field of Figure 2. Each idling transition is similar to a state changing transitions with the same source location. There are three differences. First, the source and target of the idling transition are the same. Second, an idling transition is enabled when none of the state changing transitions from the same source location are enabled. Finally, an idling transition never performs a clock or parameter reset.

Each idling transition of the computer automaton specifies a cost in watts, 10W – 150W, which is a rate of energy consumption. We introduce an additional cost for initiating the Booting process of 5 Watt-hours, which is a unit of energy consumed. When costs are not specified, they are assumed to be 0.

3.1.3 EXECUTING A COMPUTER AUTOMATON WITH FIXED DURATIONS

Next consider an "execution" of the computer automaton, consisting of an input *trace* and its resulting *run*. A trace is a trajectory over the automaton's control variables, comprised of the automaton's input variables *and* clock parameters. A run is a trajectory over the automaton's state variables, comprised of its location and clock variables. A run is generated by the automaton's transitions, given a trace and initial state.

For a timed automaton, a trace determines its run, but the trace is not constrained by the resulting run. For a parameterized timed automaton, assignments to parameters within a trace are constrained by its run, through the clock parameter resets that result from transitions. We illustrate this after a simple example of trace and run.

We first illustrate trace and run on our computer example, which doesn't include clock parameters. Suppose at $t = 0$ the computer is in the Off location and the clock c_{cmp} is 0. Our goal is to power cycle the computer twice. As a result, the computer's location variable follows the trajectory

$$\text{Off} \rightarrow \text{Booting} \rightarrow \text{On} \rightarrow \text{Shutdown} \rightarrow \text{Off}.$$

We produce this trajectory with a trace that starts with input $cmd = \perp$ at $t = 0$. The trace changes cmd to "turn-on" at $t = 5$ seconds, and changes cmd to "turn-off" after at least 15 seconds, say $t = 25$. This corresponds to the input sequence

$$cmd = \langle \langle \perp, 0 \rangle, \langle \text{turn-on}, 5 \rangle, \langle \text{turn-off}, 25 \rangle \rangle.$$

Each pair, called a time-stamped value, lists a new value for cmd , followed by the time that cmd takes on this new value. cmd remains constant between these specified times.

This trace produces a run whose sequence of locations is

$$l_{cmp} = \langle \langle \text{Off}, 0 \rangle, \langle \text{Booting}, 5 \rangle, \langle \text{On}, 20 \rangle, \langle \text{Shutdown}, 25 \rangle, \langle \text{Off}, 55 \rangle \rangle.$$

The transitions from Off to Booting and On to Shutdown occur when commanded, while the transitions from Booting to On and Shutdown to Off occur autonomously, after the time designated by the clock conditions of their respective transitions.

Due to clock resets on the transitions, the run also generates the clock variable sequence

$$c_{cmp} = \langle \langle 0, 0 \rangle, \langle 0, 5 \rangle, \langle 0, 25 \rangle \rangle.$$

The first pair is generated by initialization at $t = 0$. The second pair results from the reset performed by the transition from Off to Booting, and the third pair results from the reset performed by the

transition from On to Shutdown. Between these pairs, the clock value rises with a slope of 1. This rise is in contrast to the input and location variable sequences, Where the value of a time-stamped pair is maintained constant up to the next time stamped pair.

3.1.4 COMPUTER AUTOMATON WITH PARAMETERIZED DURATIONS

The example above serves to review the traditional concept of timed automaton. While timed automaton serve as a powerful formalism for representing autonomous transitions, a key limit for planning is that they do not allow the durations of activities to be scheduled. Our key innovation is to introduced clock parameters in order to enable scheduling. Next we illustrate our use of clock parameters to schedule the duration of Booting and Shutdown for our computer example. We would like to be able to adjust the duration of Booting and Shutdown by plus or minus 5 seconds, relative to their previous duration, 30 and 15, respectively. To accomplish this we introduce the clock parameters $\mathcal{U}_P = \{p1_{cmp}, p2_{cmp}\}$ with ranges $[25, 35]$ and $[10, 20]$, respectively. We then introduce parameter resets that are performed when the locations Booting and Shutdown are entered and accompany the resets to the computer's clock variable,

On to Shutdown when $cmd = \text{turn-off}$ with reset $c_{cmp} = 0$ **and** $p1_{cmp}(t^-) \in [25, 35]$,

Off to Booting when $cmd = \text{turn-on}$ with reset $c_{cmp} = 0$ **and** $p2_{cmp}(t^-) \in [10, 20]$,

Next, we use these two clock parameters to adjust the duration of Shutdown and Booting. We accomplish this by using the clock parameters within the clock conditions of the outgoing transitions of Shutdown and Booting. First we modify the computer's state changing transition from Shutdown to Off, by replacing condition $c_{cmp}(t^-) \geq 30$ with $c_{cmp}(t^-) \geq p1_{cmp}$. The effect is to transition out of Shutdown when the clock reaches the scheduled duration $p1_{cmp}$, rather than 30 seconds. Recall that this scheduled duration is constrained by its reset to between 25 and 35 seconds. Likewise, we modify the transition from Booting to On, by replacing condition $c_{cmp}(t^-) = 15$ with $c_{cmp}(t^-) = p2_{cmp}$. The modified transitions are

Shutdown to Off when $c_{cmp} \geq p1_{cmp}(t^-)$,

Booting to On when $c_{cmp} \geq p2_{cmp}(t^-)$.

Since the automaton is determinate, we also need to update the idling transitions for Shutdown and Booting to be consistent with the changes to the outgoing transitions, resulting in

Idle on Shutdown when $c_{cmp} < p1_{cmp}(t^-)$,

Idle on Booting when $c_{cmp} < p2_{cmp}(t^-)$.

The updated model for the computer, with schedulable durations is shown in Figure 3, with the changes highlighted in bold.

3.1.5 SCHEDULING A COMPUTER AUTOMATON WITH PARAMETERIZED DURATIONS

Next we illustrate the process of scheduling using our parameterized computer model. By assigning a duration to a clock parameter $p1_{cmp}$, we schedule how long the computer stays in Shutdown, before moving to Off, and by assigning a duration to $p2_{cmp}$, we schedule how long the computer stays in Booting. The values we assign to clock parameters are not fixed for all time; this would,

Computer Automaton with Schedulable Duration

- Variables:
 - Location $l_{cmp} \in \{\text{On}, \text{Shutdown}, \text{Off}, \text{Booting}\}$,
 - Clock $c_{cmp} \in \mathbb{R}_0^+$,
 - Input $cmd \in \{\text{turn-off}, \text{turn-on}\}$,
 - **Clock parameters $p1_{cmp}$ and $p2_{cmp}$,**
- State changing transitions:
 - On to Shutdown when $cmd = \text{turn-off}$ with reset $c_{cmp} = 0$ **and** $p1_{cmp}(t^-) \in [25, 35]$,
 - Shutdown to Off when $c_{cmp} \geq p1_{cmp}(t^-)$,
 - Off to Booting when $cmd = \text{turn-on}$ with resets $c_{cmp} = 0$, $p2_{cmp}(t^-) \in [10, 20]$ **and** cost = 5Wh,
 - Booting to On when $c_{cmp} \geq p2_{cmp}(t^-)$.
- Idling transitions:
 - Idle on On when $cmd \neq \text{turn-off}$ with cost = 100W,
 - Idle on Shutdown when $c_{cmp} < p1_{cmp}(t^-)$ with cost = 150W,
 - Idle on Off when $cmd \neq \text{turn-on}$ with cost = 10W,
 - Idle on Booting when $c_{cmp} < p2_{cmp}(t^-)$ with cost = 150W.

Figure 3: Computer with schedulable duration, specified by a parameterized timed automaton ($cmp = \langle \{l_{cmp}\}, \{c_{cmp}\}, \{cmd\}, \{p1_{cmp}, p2_{cmp}\}, \mathcal{T} \rangle$). Clock parameters $p1_{cmp}$ and $p2_{cmp}$ are used to set the duration of the Booting and Shutdown activities to $+/-$ 5 seconds of 15 and 30 seconds, respectively.

for example, fix all instances of an activity, such as Booting, to the same duration. Instead, clock parameters, $\mathcal{U}_P : \mathbb{R} \rightarrow R(\mathcal{U}_P)$, are state trajectories (Definition 5) and can be adjusted over time using clock parameter resets.

This is important, as it allows us to schedule a different duration for every time that an automaton visits a location. For example, suppose we decide to power cycle the computer twice. The first time we decide to Boot and Shutdown the computer for 10 and 25 seconds, respectively, while the second time we choose a longer time, say 20 and 35 seconds, respectively. To schedule a different duration for each time a location is visited, a clock parameter reset is invoked every time the location is entered.

An automaton specifies a clock parameter reset as a range. A scheduling policy is then used to select a value for the clock parameter within this range, for every reset. ctBurton incorporates scheduling within the process of planning a trace. Parameterized timed automata scheduling is similar to activity scheduling in PDDL temporal planning, where PDDL actions are analogous to automaton locations. To invoke a PDDL action we put the automaton in its corresponding location, the activity duration is the amount of time the automaton dwells in this location, and the scheduler sets this duration by associating a clock parameter with the location and by resetting the clock parameter when the location is entered.

Returning to our example, during the computer's first cycle recall that we want the computer to Boot in 10 seconds and Shutdown in 25 seconds. During the computer's second cycle, recall that we extend the timing to 20 seconds for Booting and 35 seconds for Shutdown. In addition, suppose we want the computer to be On and Off for 5 seconds, each time. This is satisfied by the following location variable run

$$l_{cmp} = \langle \langle \text{Off}, 0 \rangle, \langle \text{Booting}, 5 \rangle, \langle \text{On}, 15 \rangle, \langle \text{Shutdown}, 20 \rangle, \langle \text{Off}, 45 \rangle, \\ \langle \text{Booting}, 50 \rangle, \langle \text{On}, 70 \rangle, \langle \text{Shutdown}, 75 \rangle, \langle \text{Off}, 110 \rangle \rangle.$$

The corresponding clock variable run is:

$$c_{cmp} = \langle \langle 0, 0 \rangle, \langle 0, 5 \rangle, \langle 0, 20 \rangle, \langle 0, 50 \rangle, \langle 0, 75 \rangle \rangle.$$

This run for the clock variable results from the clock being reset whenever the computer enters Booting or Shutdown.

The runs of the location variable and clock variable are produced by traces of the input variable cmd and the two clock parameters, $p1_{cmp}$ and $p2_{cmp}$. The input trace,

$$cmd = \langle \langle \perp, 0 \rangle, \langle \text{turn-on}, 5 \rangle, \langle \text{turn-off}, 20 \rangle, \langle \text{turn-on}, 50 \rangle, \langle \text{turn-off}, 75 \rangle \rangle$$

specifies when the computer turns on and turns off, hence moving from Off to Booting and On to Shutdown. The clock parameter trace

$$p1_{cmp} = \langle \langle 25, 20 \rangle, \langle 35, 75 \rangle \rangle$$

specifies the duration of Shutdown for the first and second cycle. 25 seconds and 35 seconds, respectively. The time stamps, 20 and 75 seconds, specify the time when these two durations are first enforced, which is the time when Shutdown is entered, during the first and second cycles. The clock parameter trace

$$p2_{cmp} = \langle \langle 10, 5 \rangle, \langle 20, 50 \rangle \rangle,$$

likewise specifies the duration of Booting for each of the two cycles, 10 and 20 seconds, respectively.

Each parameter trace resets its value to the reset’s range, every time the automaton enters the parameter’s location, Shutdown or Booting,

The following is an intermediate state of $p1_{cmp}$ ’s trace, where the range of $p1_{cmp}$ is constrained at each reset,

$$p1_{cmp} = \langle \langle [25, 35], 20 \rangle, \langle [25, 35], 75 \rangle \rangle.$$

A similar intermediate trace for $p2_{cmp}$ is

$$p2_{cmp} = \langle \langle [10, 20], 5 \rangle, \langle [10, 20], 50 \rangle \rangle.$$

A scheduler selects a desired duration from each range, resulting in the parameter traces given earlier. The set of wave forms that correspond to this example are shown in Figure ??.

David, please add figure with wave forms for this example as described.

In this paper our framing of the temporal planning problem performs scheduling *a priori*, at the time that the planner generates a trace. There is an extensive body of work in which scheduling is delayed to online execution. For example, a just-in-time dynamic scheduling policy, such as [28, 36], decides when to execute an activity at the time that it is executed. In the future section, at the end of this paper, we discuss extending PCTA planning to online planning and just-in-time scheduling.

3.2 Transitions and Dense Time

The execution of an automaton is a sequence of transition τ , their corresponding reward ρ and their time of transition. Next consider the structure and behavior of τ and ρ . Transitions of timed automata make discrete changes to an automaton’s location variable at real-valued points in time. We restrict each location to be maintained over an interval of time with non-zero duration, not just an instant. Automaton reward is then specified per unit time and is accumulated both during changes in location and over intervals where locations are maintained.

Our treatment of intervals where locations are maintained, as open, closed or semi-closed, can be subtle and warrants discussion. To maintain a model of dense time, an execution specifies the state of an automaton at every moment. To do so we partition the timeline using intervals that meet.

One approach to a dense model of time is to assume each location is maintained over a semi-closed interval; for example, “Door = Open” over “[5,9)”. In this model a transition at time t reaches its target at t , and is initiated just prior to t (denoted t^-).

An issue with this approach is that a PTA transition is an idealization of a real system in which the transition takes an interval of time ϵ to occur, not an instant. While this transition occurs, the location of the real system varies from source to target. As a consequence, choosing a single location at the point of transition, either the source or target location, is unrealistic, as the value is ambiguous. Modeling this ambiguity correctly is important, for example, to correctly perform plan execution monitoring for real world systems. To capture this ambiguity, we specify the automaton’s location at t as the disjunction of the transition’s source and target locations.

Given this ambiguity, we avoid relying upon the state of the system at the moment of transition. Instead we define our transition relation so that the transition’s source location and guard are evaluated just prior to the time of transition, at t^- , while the target location, clock variable reset and clock parameter reset go into effect just afterward, at t^+ . Each transition is then a relation $\tau_i(\mathcal{V}(t^-), \mathcal{U}(t^-), \mathcal{V}(t^+), \mathcal{U}_P(t^+))$.

David, please add a figure that shows open intervals before and after the transition point, with source and target values, and both values at the transition point. The simpler alternative is to show two closed intervals that overlap exactly at the transition point.

For this relation, a transition is *applicable* at t if the system state at t^- , just before t , includes the transition's source location. Furthermore, a transition is *enabled* when it is applicable and the transition's guard is satisfied at t^- , given the automaton's inputs and clock parameters at t^- . Finally, a transition is *satisfied* at t if enabled and the transition's target, clock variable reset and clock parameter reset hold at t^+ , immediately after t . When no time is specified, we assume the current time.

Definition 6. PTA Transition and Reward

Let A be a timed automaton, where $A = \langle \mathcal{L}, \mathcal{C}, \mathcal{U}_I, \mathcal{U}_{IC}, \mathcal{U}_P, \mathcal{T}, \mathcal{R} \rangle$. Each transition $\tau \in \mathcal{T}$ of A is a quadruple $\tau = \langle l_s, l_t, g, c_r \rangle$ and each reward $\rho \in \mathcal{R}$ of A is a quadruple $\rho = \langle l_s, l_t, g, r \rangle$, where

- $l_s, l_t \in R(\mathcal{L})$ are source and target locations,
- $g : R(\mathcal{V}) \times R(\mathcal{U}) \rightarrow \{\top, \perp\}$ is the guard relation.
- Clock reset $c_r(\mathcal{C}', \mathcal{U}'_C)$ is a constraint over subsets of the clock variables and parameters, $\mathcal{C}' \subseteq \mathcal{C}$ and $\mathcal{U}'_C \subseteq \mathcal{U}_P$, respectively. $c_r(\mathcal{C}', \mathcal{U}'_C) = c_{rc}(\mathcal{C}') \bowtie c_{ru}(\mathcal{U}'_C)$ is a composition of clock variable reset c_{rc} and clock parameter reset c_{ru} , where c_{rc} is an assignment to clock variables $\mathcal{C}'$ and c_{ru} is an interval assignment to clock parameters $\mathcal{U}'_C$.
- Reward value $r \in \mathbb{R}_{\geq 0}$ is a rate of reward per unit time.

We define several properties of transition τ and corresponding reward ρ :

- τ is an idling transition if $l_s(\tau) = l_t(\tau)$; otherwise, τ is a state changing transition.
- τ , respectively ρ , is applicable at t if $\mathcal{L}(t^-) = l_s$.
- τ , respectively ρ , is enabled at t if τ is applicable at t and if $g(\mathcal{C}(t^-), \mathcal{U}(t^-))$ is satisfied.
- When τ is enabled at t , automaton A instantaneously transitions at t from l_s to l_t and applies its clock reset c_r to its $\mathcal{C}'$ and $\mathcal{U}'_C$ at t^+ .
- τ is satisfied at t if three conditions hold, a) τ is enabled at t , b) $\mathcal{L}(t^+) = l_t$, and c) $\mathcal{C}(t^+) \wedge \mathcal{U}_P(t^+) \models c_r$.
- Let ρ be applicable if and only if τ is applicable, then ρ is the corresponding reward to τ .

□

ctBurton requires parameterized timed automata to be deterministic. A parameterized timed automaton is *deterministic* if there exists a unique transition, and hence a unique next state, for all consistent assignments to location variables, control variables, clock variables, and clock parameters. To ensure this condition, for any assignment to l_s , $\mathcal{U}$ and c we require that the guard g of exactly one transition is satisfied.

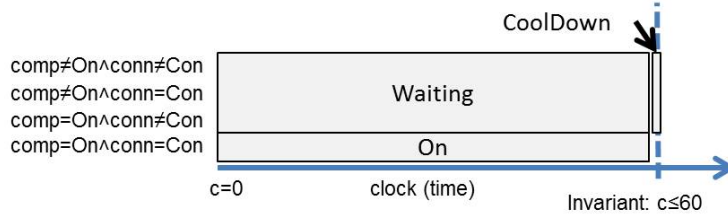


Figure 4: A graphical representation of the possible set of next-locations from the `Waiting` location of the well-formed and deterministic Projector automaton.

In Section 5 we define the execution semantics of our timed automata, including autonomous transition, in terms of traces and runs.

Figure 4 shows the set of next locations that the deterministic Projector automaton can reach, from its `Waiting` location. Note that there is exactly one possible next location for each input assignment. In order to produce plans with predictable behavior, ctBurton plans over *PCTA* models comprised only of deterministic automata.

3.3 Specifying Transitions and their Reward

To declaratively specify a transition relation τ_i and its reward ρ_i , ctBurton specifies sources and targets as location assignments and guards as propositional formulae. The propositions of the guard are either assignments to the automaton’s input variables or comparisons between its clock variables and either its clock parameters or real-valued constants. The resets of τ_i ’s are specified as clock variable and clock parameter assignments.

Finally, the value of reward ρ_i for transition τ_i is a non-negative real, denoting reward per unit time at the time of transition. For a state-changing transition, recall that ρ_i is the integral of an impulse δ , where the impulse is needed for the instantaneous transition to yield a non-zero reward.

Definition 7. PTA Transition and Reward Specification

Let $\tau \in \mathcal{T}$ be a transition of parameterized timed automaton A , where $\tau = \langle l_s, l_t, g, c_r \rangle$, and let $\rho \in \mathcal{R}$ be its corresponding reward, where $\rho = \langle l_s, l_t, g, r \rangle$. A transition and reward specification for τ and ρ is a quintuple $\langle ss, ts, \varphi, cs, rx \rangle$, whose elements specify the corresponding elements of τ and ρ :

- $ss ::= (\mathcal{L} = l_s)$ and $ts ::= (\mathcal{L} = l_t)$ are source and target specifications, which assign source and target locations $l_s, l_t \in R(\mathcal{L})$, respectively.
- φ is the guard specification, which is a disjunction in which each disjunct φ_k consists of a state condition conjoined with a clock condition. Each state condition $\varphi_{s,k}$ specifies a relation over input variables $\mathcal{U}_{\mathcal{I}}$ of A . It takes the form $\varphi_{s,k} ::= \text{true} \mid \text{false} \mid (u = v) \mid \neg\varphi_1 \mid \varphi_1 \wedge \varphi_2 \mid \varphi_1 \vee \varphi_2$, where $u \in \mathcal{U}_{\mathcal{I}}$ and $(u = v)$ is the assignment of u to a value from its domain.

Each clock condition $\varphi_{c,k}$ specifies a relation over clock variables $\mathcal{C} \cup \mathcal{U}_{\mathcal{I}_C}$ and clock parameters $\mathcal{U}_{\mathcal{P}} \cup \mathcal{U}_{\mathcal{I}_P}$. It takes the form $\varphi_{c,k} ::= \text{true} \mid c_i \in [l_i, u_i) \mid \varphi_1 \wedge \varphi_2$, which constrains each clock variable $c_i \in \mathcal{C} \cup \mathcal{U}_{\mathcal{I}_C}$ to lie within a semi-closed interval $[l_i, u_i)$, between l_i and u_i . l_i and u_i are constants or clock parameters ($l_i, u_i \in \mathcal{U}_{\mathcal{P}} \cup \mathcal{U}_{\mathcal{I}_P}$), denoting non-negative

real values. If τ is an idling transition ($l_s = l_t$), then $l_i = 0$, for each l_i . Otherwise, τ is a state changing transition ($l_s \neq l_t$) and $0 < l_i$, for each l_i .⁴

- cs is the clock reset specification, which is a conjunction of a clock variable reset specification and a clock parameter reset specification. The clock variable reset specification denotes the relation c_{rc} over clock variables $\mathcal{C}$ and is of the form $cvs ::= true \mid (c_i := v_i) \mid cvs_1 \wedge cvs_2$, where $c_i \in \mathcal{C}$ and $v_i \in \mathbb{R}_{\geq 0}$. The clock parameter reset specification denotes the relation c_{ru} over clock parameters $\mathcal{U}_{\mathcal{P}}$ and is of the form $cps ::= true \mid (p_i \in [l_i, u_i]) \mid cps_1 \wedge cps_2$, where $p_i \in \mathcal{U}_{\mathcal{P}}$ and $[l_i, u_i] \subseteq \mathbb{R}_{\geq 0}$. $ctBurton$ restricts the clock variable reset specification to $true$ for idling transitions and to assignments $c_i := 0$ for state changing transitions.
- $rs \in \mathbb{R}_{\geq 0}$ is a reward specification such that rate of reward r is $r = rs$ when $l_s = l_t$ and $r = rs\delta(t)$ when $l_s \neq l_t$,

where $l \neq v$ denotes $\neg(l = v)$.

τ is satisfied at t if and only if $ss(t - \epsilon) \wedge \phi(t - \epsilon) \wedge ts(t + \epsilon) \wedge cs(t + \epsilon)$. Likewise, ρ evaluates to r at t if and only if $ss(t - \epsilon) \wedge \phi(t - \epsilon) \wedge ts(t + \epsilon) \wedge cs(t + \epsilon)$. τ and ρ are in canonical form if their state condition φ_s is in canonical form. $\square$

Clock conditions are expressed over the clock of the automaton and a set of additional input clock variables. The automaton's clock is used to time the duration that the automaton resides in a location. The input clocks are used to synchronize timing of the automaton relative to other automata.

Clock conditions constrain the value of a clock c_i to a semi-closed interval $[l_i, u_i)$, as explained below. For simplicity clock conditions and resets are specified with respect to the time of transition t , since $c(t + \epsilon) = c(t)$ as $\epsilon \rightarrow 0$. For example, a transition from location $l1$ with a clock condition of $c \geq 6$ and clock variable reset $c := 0$ allows the transition to be taken when the automaton is in location $l1$ for exactly 6 time units. Upon transition the clock variable resets to 0^+ at $t = 6^+$.

To ensure that location assignments hold for a non-zero duration, all state changing transitions include positive lower bound clock conditions and the clock variables mentioned are reset to zero upon entering the source location of the relevant transitions. Lower bounds of idling transitions are set to 0, since idling must occur from when a location is entered until a state changing transition occurs. Lower bound conditions are non-strict ($\leq$), since state changing transitions occur at an instant, while upper bound conditions are strict ($<$), so that idling occurs up to the instant that a state changing transition occurs.

The modeler must ensure that the transition specifications result in a determinate automaton. In practice, descriptions of transitions often leave idling transitions implicit. For example, when not specified, the condition of a location's idling transition is constructed by conjoining the conditions of every state changing transition out from that location and by negating their conjunction.

Earlier we provided English descriptions of transitions for the Computer. We revisit them here using transition and reward specifications. We started with the state changing transition

On to Shutdown when $cmd = \text{turn-off}$ with reset $c_{cmp} = 0$,

4. We choose l_i to be positive so that two or more state changing transitions can not occur instantaneously. We choose closed intervals for state changing transitions to ensure that state changes occur at well-defined events, and open intervals for idling transitions, to ensure that transitions are determinate.

which corresponds to the quintuple

$$\langle l_{cmp}(t^-) = \text{On}, l_{cmp}(t^+) = \text{Shutdown}, cmd(t^-) = \text{turn-off}, c_{cmp}(t^+) = 0, 0 \rangle.$$

This transition goes from location On to Shutdown and includes a guard with a state condition $cmd(t^-) = \text{turn-off}$. The transition is applicable at t if $l_{cmp}(t^-) = \text{On}$ and is enabled at t if, in addition, $cmd(t^-) = \text{turn-off}$. If enabled, the effect of the transition is for the location to become Shutdown and the clock to be reset to zero at t^+ . The transition is satisfied at t if it is enabled and its target and clock reset hold at t^+ , that is, $l_{cmp}(t^+) = \text{Shutdown}$ and $c_{cmp}(t^+) = 0$. The purpose of the clock reset is to start counting the time that the computer resides in Shutdown.

This state changing transition is complemented by an idling transition for On,

Idle on On when $cmd \neq \text{turn-off}$,

which is described by the quintuple

$$\langle l_{cmp}(t^-) = \text{On}, l_{cmp}(t^+) = \text{On}, cmd(t^-) \neq \text{turn-off}, \top, 0 \rangle.$$

The fourth value of the quadruple, $\top$, indicates no reset. This idling transition is enabled when applicable and its corresponding state changing transition, above, is not enabled. Also, the idling transition continues to increase the clock without resetting it.

We use these specifications to illustrate how transitions use clock parameters to schedule the time that an automaton spends in a location. For example, the computer state changing transition,

$$\langle l_{cmp}(t^-) = \text{Shutdown}, l_{cmp}(t^+) = \text{Off}, c_{cmp}(t^-) \geq p1_{cmp}, \top, 0 \rangle,$$

contains clock condition $c_{cmp}(t^-) \geq p1_{cmp}$, which measures the time the computer is in Shutdown and exits when duration $p1_{cmp}$ is reached. A scheduling policy sets the Shutdown duration by assigning parameter $p1_{cmp}$ between resets. $p1_{cmp}$ is reset when the computer enters Shutdown using transition

$$\langle l_{cmp}(t^-) = \text{On}, l_{cmp}(t^+) = \text{Shutdown}, cmd(t^-) = \text{turn-off}, c_{cmp}(t^+) = 0 \wedge p1_{cmp}(t^-) \in [25, 35], 0 \rangle.$$

Reset $p1_{cmp}(t^-) \in [25, 35]$ tells the scheduler that it needs to select a Shutdown duration between 25 and 35. The value selected by the scheduler remains in force until the parameter's next reset.

Concurrency is central to our problem. Our extension of parameterized timed automata to concurrent automata is straightforward and is presented in Appendix B.1.

3.4 Benefits of Modeling Autonomous Transitions for Concurrent Systems

We end our parameterized timed models section by highlighting the benefits of using automata as an action model. Many planning formalisms require complete control in that each action must be initiated by an external agent. This allows the planner to assume that the world state remains constant unless the planner takes action. For many real-world processes, however, some states change ‘‘autonomously,’’ without being initiated by an external agent. To control these systems a planner must be able to reason through the effects of these autonomous transitions over time.

The PCTA modeling formalism makes it straight forward to represent these autonomous behaviors. Consider three categories of these behaviors, which we call indirect effects, automatic timed

transitions, and periodic transitions. In the *PCTA* formalism, an external agent initiates a behavior through an assignment to an external control variable. An *indirect effect* occurs when the transition of an automaton A is *enabled* by the locations of one or more automata A_i , without the need to assign an external control variable. An example is a tub overflowing when the water level reaches the top of the tub.

An *automatic timed transition* occurs when a transition is guarded only on a clock variable. An example is a timer going off. *Periodic transitions* occur when a series of automatic timed transitions start and end on the same location, thus forming a cycle. An example is a stop light that cycles between green, yellow and red.

A combination of these features can produce more complex interactions: several indirect effects can produce a chain reaction, in which one indirect transition initiates a cascade of subsequent transitions, and indirect effects with automatic timed transitions can result in periodic transitions that span several automata. Hence taken together, our modeling formalism is significantly enriched through its support for autonomous transitions.

4. Specifying Initial States

The start state s_0 of an automaton A is described by a start state specification. This specification assigns all location and clock variables of A at time e_0 and restricts the range of all clock parameters of A at time e_0 .

Definition 8. PTA Start Specification

Let s_0 be a start state for parameterized timed automaton A . A start specification for s_0 of A is a triple $\langle ss, cs_v, cs_p \rangle$:

- $ss ::= (\mathcal{L} = l_s)$, for $l_s \in R(\mathcal{L})$, is a start location specification, which assigns l_s to $\mathcal{L}$ in s_0 .
- cs_v is a clock variable reset specification. cs_v is a relation over clock variables $\mathcal{C}$ that holds in s_0 and is of the form $cvs ::= true \mid vasn_1 \dots \wedge vasn_n$, where $vasn_i ::= (c := v)$, $c \in \mathcal{C}$ and $v \in \mathbb{R}_{\geq 0}$. If $c \in \mathcal{C}$ is not assigned in cs_v , then c is assigned 0 in s_0 .
- cs_p is a clock parameter reset specification. cs_p is a relation over clock parameters $\mathcal{U}_P$ that holds in s_0 and is of the form $cps ::= true \mid pasn_1 \dots \wedge pasn_n$, where $pasn_i ::= (p \in [l, u])$, $p \in \mathcal{U}_P$ and $[l, u] \subseteq \mathbb{R}_{\geq 0}$. If $p \in \mathcal{U}_P$ is not reset in cs_p , then p is reset to range $\mathbb{R}_{\geq 0}$ in s_0 .

□

Start state specifications may be included in the definition of a *PTA* or supplied separately, when start states of an automaton A vary from one problem to the next. For a concurrent *PTA*, start state specifications may be supplied separately for each component automaton A_i . In this paper, note that we restrict clock variable resets to $c := 0$, which is the default.

The initial state in our presentation example has the computer and projectors Off and disconnected from each other. The computer automaton resets the clock variables and parameters as needed, hence no resets need to be specified for the initial state. Same for the other components. The initial state specifications are thus

$$\begin{aligned} &\langle \text{computer} = \text{off}, \top, \top \rangle, \\ &\langle \text{projector} = \text{off}, \top, \top \rangle, \\ &\langle \text{connection} = \text{disconnected}, \top, \top \rangle. \end{aligned}$$

5. Executing Parameterized Timed Automata

Next we define the execution of PCTA in terms of legal input sequences, called *traces*, and sequences of automata state transitions that result from these traces, called *runs*. Our definitions generalize standard concepts for timed automata [1] to PCTA. A PCTA trace is distinguished in that it represents the result of a hybrid activity planning and scheduling problem, where the input trace corresponds to the activity plan and the clock parameter trace corresponds to the schedule.

In the next subsection we present general definitions of trace and run for parameterized timed automata. In Subsection 5.2 we then specify traces and runs as *timed sequences* of states and transitions. In Appendices B.2 and B.3 we adapt these definitions to primitive and concurrent automata.

5.1 Traces and Runs

A timed automaton changes the state of the world through transitions that are enabled by assigning values to control variables, where control variables are comprised of input variables and clock parameters. In our formulation an activity plan corresponds to a trajectory over these control variables, called a *trace*. Given an initial state, the effect of a trace on system state is called a *run*, and corresponds to a trajectory over automaton location variables and clock variables that satisfies the transition relation of the automaton.

Traces of parameterized timed automata assign duration to actions, which is essential to combined activity planning and scheduling. To incorporate scheduling, a trace of a parameterized automaton is defined over clock parameters, as well as input variables, and the automaton run is conditioned on traces over these clock parameters. Traces and runs may be finite and their extent may be bound to an interval. If no interval is specified, the extent of the trace or run is assumed to be infinite and occurs from the start event to infinity. Below we define these concepts for a parameterized timed automaton in general and address concurrency in Appendix B.2.

When defining trace and run for a parameterized timed automaton there is a bit of a "chicken and egg problem". A run must satisfy the transition relation of the automaton and depends on a trace over the automaton's clock parameters, as well as its input variables. Conversely, these clock parameters can only change value at times when the automaton run invokes a clock parameter reset, which occurs at the time of state changing transitions. In addition, the changed values must be consistent with the range specified by its corresponding reset.

This creates a simultaneity between the clock parameter trace, the automaton run and its transition sequence. While consistency of an input variable trace is defined independent of its resulting run, consistency of the clock parameter trace is defined relative to its resulting run. Furthermore, the values given to clock parameters affect the time of subsequent transitions, hence the time of subsequent parameter value changes.

For example, consider how the computer is powered up in our projector system. Below repeats the run of the location and clock variables and the trace of the input *cm* and clock parameter *p2_{cmp}*.

$$\begin{aligned} l_{cmp} &= \langle \langle \text{Off}, 0 \rangle, \langle \text{Booting}, 5 \rangle, \langle \text{On}, 15 \rangle \rangle. \\ c_{cmp} &= \langle \langle 0, 0 \rangle, \langle 0, 5 \rangle \rangle \\ cmd &= \langle \langle \perp, 0 \rangle, \langle \text{turn-on}, 5 \rangle \rangle \\ p2_{cmp} &= \langle \langle 10, 5 \rangle \rangle. \end{aligned}$$

Location l_{cmp} starts at Off. When *cmd* is set to turn-on at 5 seconds, causing the location to transition to Booting, resetting clock variable c_{cmp} to 0 and clock parameter $p2_{cmp}$ to range [10, 20].

Once the run triggers the parameter reset, the scheduler selects 10 from the range, and the location to On at 15 seconds.

We capture this coupling in the following definition by requiring the the trace and run be consistent with the transition relation.

Definition 9. Trace

Let $\mathcal{V}$ be the state variables of A . Let $\mathcal{U} = \mathcal{U}_{\mathcal{I}} \cup \mathcal{U}_{\mathcal{P}} \cup \mathcal{U}_{\mathcal{I}_C} \cup \mathcal{U}_{\mathcal{I}_P}$ be a vector of control variables for timed automaton A , comprised of input variables $\mathcal{U}_{\mathcal{I}}$, clock parameters $\mathcal{U}_{\mathcal{P}}$, input clock variables $\mathcal{U}_{\mathcal{I}_C}$ and input clock parameters $\mathcal{U}_{\mathcal{I}_P}$. A (valid) trace of $\mathcal{U}$ over extent $(e_0, e_n) \subseteq \mathbb{R}_{\geq 0}$ is a trajectory $\mathcal{U}(t)$ over (e_0, e_j) , where $e_j \leq e_n$ is the maximum event in (e_0, e_n) , such that $\mathcal{U}$ and $\mathcal{V}$ satisfy a transition $\tau_{ij} \in \mathcal{T}(A)$ for each $t \in (e_0, e_j)$, for some trajectory $\mathcal{V}(t)$ of A . $\mathcal{U}(t)$ is complete over (e_0, e_n) if $e_j = e_n$; otherwise, $\mathcal{U}(t)$ is partial. $\square$

Intuitively, $\mathcal{U}(t)$ is a trace for a transition relation if there exists a run such that the trace and $\mathcal{U}(t)$ satisfy the relation. The trace is complete with respect to $[e_0, e_n]$ if it specifies a value for all times in the interval.

For example, above we explained how the trace and run interact with the computer's transitions to turn on the computer. This explanation informally confirms the consistency of the trace and run.

Turning to the definition of run, a run is a state trajectory that results from a trace, where the state of a timed automaton is an assignment to its location and clock variables. We make the standard observation for timed automata that a trajectory over location variables uniquely determines the trajectory over clock variables, hence we define run in terms of the location variables alone. Given an initial state and a trace for the inputs and clock parameters of a timed automaton, a run is a trajectory over automaton location variables and must satisfy the automaton's transition relation. We refer to the run of an automaton and a run of its location variables interchangeably. Similar to a trace, a run may be bounded or infinite, complete or partial.

Definition 10. Run and its Reward

Let A be a parameterized timed automaton with control variables $\mathcal{U}$ and state variables $\mathcal{V}$. Let $\mathcal{U}(t)$ be a trace of $\mathcal{U}$ over $(e_0, e_n) \subseteq \mathbb{R}_{\geq 0}$ and s_0 be the initial state of A at e_0^+ . A run of A (equivalently $\mathcal{V}$) over (e_0, e_n) , given s_0 and $\mathcal{U}(t)$, is a trajectory $\mathcal{V}(t)$ over (e_0, e_j) , where $e_j \leq e_n$ is the maximum event in (e_0, e_n) , such that a) $\mathcal{V}(e_0^+) = s_0$, b) for all $t \in (e_0, e_j)$, $\mathcal{V}(t) \in R(\mathcal{V})$ and c) $\mathcal{U}$ and $\mathcal{V}$ satisfy a transition $\tau_{ij} \in \mathcal{T}(A)$ for each $t \in (e_0, e_j)$. A run is complete if $t_k = t_n$; otherwise, it is partial.

Given a trace $\mathcal{U}$ and its run $\mathcal{V}$ of A , the reward of $\mathcal{U}$ and $\mathcal{R}$ over (e_0, e_n) is

$$\mathcal{R}(\mathcal{U}, \mathcal{V}, (e_0, e_n)) = \int_{t \in (e_0, e_n)} \mathcal{R}(\mathcal{V}(t - \epsilon), \mathcal{U}(t - \epsilon), \mathcal{L}(t + \epsilon)), \text{ as } \epsilon \rightarrow 0^+.$$

$\square$

Conditions a) and b) simply say that the trajectory starts in s_0 and draws from $\mathcal{V}$'s range. Condition c) says that the transition relation is satisfied at every point in the extent. Reward of the run is the integral of the instantaneous reward at each moment of the run.

5.2 Traces and Runs as Timed Sequences

Next we introduce a syntactic representation for trace, which we use as the planning problem output of the planning problem. We introduce similar representations for run, both in terms of state and

trajectory sequences. We return the run of the output trace as an auxiliary output. We also uses these representations to encode our planning problem as a constraint program in Section 8.

We describe traces and runs syntactically using the concept of *timed sequence*. A timed sequence represents a trajectory compactly by specifying each of its value changes and the time when each change occurs. Since time is dense ($t \in \mathbb{R}$), trajectories represent an infinite sequence of values over an interval of time. We assume, however, that the number of value changes is finite. A timed sequence is a sequence of timed values $\langle v_i, t_i \rangle$, where v_i is the target value of a change and t_i is the time when the change occurs. Upon change, v_i is assumed to remain constant from its time t_i to the time t_{i+1} of the next value. For example, the sequence $V = \langle \langle a, 1 \rangle \langle b, 7 \rangle \langle c, 20 \rangle \rangle$ over time interval $(1, 30)$ specifies a trajectory in which $v = a$ for $t \in (1, 7)$, $v = b$ for $t \in (7, 20)$, and $v = c$ for $t \in (20, 30)$. This approach is sufficient to represent trajectories over input variables or location variables.

To represent trajectories of transitions we extend the concept of timed sequence slightly. An automaton idles over *open* time intervals, delineated by state changing transitions. To represent this we define timed sequences so that a value may hold, not only over a semi-closed interval $]$, but an interval that is open $()$, semi open $[]$ or closed $]$ (including a point). We represent these cases using two *time stamps*, “ t ” and “ t^+ ”. When “ t ” is paired with a value, the value holds for an interval whose left side is closed, starting at t . Alternatively, when a value is paired with “ t^+ ”, the value holds for an interval that is open on the left side and starts immediately following t . For example, timed sequence $V = \langle \langle a, 1 \rangle \langle b, 7^+ \rangle \langle c, 30 \rangle \rangle$ over $[1, 30]$ specifies a trajectory in which $v = a$ for $t \in [1, 7]$, $v = b$ for $t \in (7, 30)$, and $v = c$ for $t = 30$.

Definition 11. Timed Sequence of a Trajectory

Let Q be a set of values. A timed sequence S of Q over interval $[t_0, t_n] \subseteq \mathbb{R}$ is a sequence of timed values $\langle q_i, t_i, r_i \rangle$, where

- $q_i \in Q$,
- $t_i \in [t_0, t_n]$, and
- $r_i \in \{\leq, <\}$.

The time stamp of timed value $\langle q_i, t_i, r_i \rangle$ is the pair $s_i = \langle t_i, r_i \rangle$. In this context we use “ t^+ ” as short for $\langle t, < \rangle$ and “ t ” as short for $\langle t, \leq \rangle$. Let $s_i = \langle t_i, r_i \rangle$ and $s_{i+1} = \langle t_{i+1}, r_{i+1} \rangle$ be time stamps. Then $I(s_i, s_{i+1})$ denotes the interval $\{t \mid t_i \leq t, \neg(t_{i+1} \leq t)\}$.

A sequence S is concise if $q_i \neq q_{i+1}$, for all successive pairs of timed values $\langle q_i, t_i, r_i \rangle$ and $\langle q_{i+1}, t_{i+1}, r_{i+1} \rangle$, where $t_i, t_{i+1} \in [t_0, t_n]$.

Let $V(t)$ be a trajectory over $t \in [t_0, t_n] \subseteq \mathbb{R}$. A timed sequence S over $[t_0, t_n]$ for trajectory $V(t)$ is a timed sequence of $R(V)$, the range of V , over $[t_0, t_n]$ that satisfies the following condition:

Consider each timed value $\langle q_i, t_i, r_i \rangle \in S$, with time stamp $s_i = \langle t_i, r_i \rangle$. If this timed value has a successor in S , then let s_{i+1} denote its time stamp, otherwise $s_{i+1} = \langle t_n, < \rangle$. Then for all $t_j \in I(s_i, s_{i+1})$, $V(t_j) = q_i$.

$V(t)$ is called the trajectory of S . $\square$

We use timed sequences to describe input and control parameter trajectories of traces and to describe location and transition trajectories of runs. Since the clock trajectory of a run can be derived from its location trajectory, we leave it implicit.

Definition 12. Timed and Trajectory Sequences of a Run

Let $\mathcal{V}(t)$ be a run of timed automaton A that is complete over $(t_0, t_n) \subseteq \mathbb{R}$ and let $\mathcal{L}(t)$ denote the location variable trajectory of $\mathcal{V}(t)$.

A timed sequence S over (t_0, t_n) for run $\mathcal{V}(t)$ is a timed sequence over (t_0, t_n) for $\mathcal{L}(t)$. $\mathcal{V}(t)$ is called the trajectory of S .

A timed trajectory sequence S' over (t_0, t_n) for run $\mathcal{V}(t)$ is a timed sequence over (t_0, t_n) for a trajectory of transitions $\mathcal{T}(A)$ that satisfies the following condition:

Consider each timed value $\langle \tau_i, t_i, r_i \rangle \in S'$ with time stamp $s_i = \langle t_i, r_i \rangle$. If this timed value has a successor in S , then let s_{i+1} denote its time stamp, else $s_{i+1} = \langle t_n, < \rangle$.

Then for all $t_j \in I(s_i, s_{i+1})$, transition τ_i is satisfied by $\mathcal{V}(t)$ at t_j .

$\mathcal{V}(t)$ is called the trajectory of timed trajectory sequence S' . $\square$

Note that a run has a unique concise timed state and transition sequence, since its automaton is determinate. However, not all timed sequences correspond to valid runs.

To describe clock variable trajectory explicitly, we introduce a slight variant of timed sequence whose timed values increase with slope one.

Definition 13. Timed Sequence of a Clock Trajectory

Let $\mathcal{C}(t)$ be the clock trajectory of a run of timed automaton A over $t \in (t_0, t_n) \subseteq \mathbb{R}$. A timed sequence S over (t_0, t_n) for $\mathcal{C}(t)$ is a timed sequence of range $R(\mathcal{C})$ over (t_0, t_n) that satisfies the following condition:

For each timed value $\langle q_i, t_i, r_i \rangle \in S$ with time stamp $s_i = \langle t_i, r_i \rangle$, if this timed value has a successor in S , let s_{i+1} denote its time stamp, else $s_{i+1} = \langle t_n, < \rangle$. Then for all $t_j \in I(s_i, s_{i+1})$, $V(t_j) = q_i + (t_j - t_i)$.

$\mathcal{C}(t)$ is called the trajectory of S . $\square$

We provided examples of timed sequences for traces and runs in Sections 3.1.3 and 3.1.4. Below is a repeat of the state sequence for the computer going from Off to On and its corresponding transition sequence.

$$\begin{aligned} l_{cmp} &= \langle \langle \text{Off}, 0 \rangle, \langle \text{Booting}, 5 \rangle, \langle \text{On}, 15 \rangle \rangle, \\ &= \langle \langle \text{Off} \rightarrow \text{Off}, 0 \rangle, \langle \text{Off} \rightarrow \text{Booting}, 5 \rangle, \langle \text{Booting} \rightarrow \text{Booting}, 5^+ \rangle, \langle \text{Booting} \rightarrow \text{On}, 15 \rangle \rangle. \end{aligned}$$

To define reward for a timed sequence description of a run, we use the concept of the run's timed trajectory sequence.

Definition 14. Reward for a Run as a Timed Trajectory Sequence

Let $\mathcal{V}(t)$ be a run of timed automaton A that is complete over $(t_0, t_n) \subseteq \mathbb{R}$ and let $S' = \{ \langle \tau_i, t_i, r_i \rangle \}$ be $\mathcal{V}(t)$'s timed trajectory sequence over (t_0, t_n) . Reward $R(\mathcal{V}(t))$ of run $\mathcal{V}(t)$ over (t_0, t_n) is

$$R(V) = \sum_{\langle \tau_i, t_i, r_i \rangle \in S'} \begin{cases} r(\rho_i) & \text{if } l_s(\tau_i) \neq l_t(\tau_i) \\ r(\rho_i)(t_{i+1} - t_i) & \text{otherwise,} \end{cases}$$

where $r(\rho_i)$ is the value of the reward associated with τ_i and t_{i+1} is t_n when i is the last element of S' . $\square$

From Section 3.3, recall that reward for an idling transition ($l_s(\tau_i) = l_t(\tau_i)$), $r(\rho_i)$ denotes reward per unit time, which is multiplied by the duration of idling, $t_{i+1} - t_i$. A state changing transition ($l_s(\tau_i) \neq l_t(\tau_i)$) is instantaneous and its reward is modeled by a unit impulse, centered at the time of transition. $r(\rho_i)$, the reward received at the time of transition, is the coefficient of the unit impulse. To compute the reward of a timed trajectory sequence, this reward is accumulated for successive pairs in this sequence. The reward for a run of a concurrent automaton is computed by summing the reward of each component automaton run.

Returning to the transition sequence for the computer,

$$l_{cmp} = \langle \langle \text{Off} \rightarrow \text{Off}, 0 \rangle, \langle \text{Off} \rightarrow \text{Booting}, 5 \rangle, \langle \text{Booting} \rightarrow \text{Booting}, 5^+ \rangle, \langle \text{Booting} \rightarrow \text{On}, 15 \rangle \rangle,$$

the reward associated with each transition is

$$l_{cmp} = \langle \langle 10W, 0 \rangle, \langle 5Wh, 5 \rangle, \langle 150W, 5^+ \rangle, \langle 0W, 15 \rangle \rangle$$

and the reward for the sequence is $10(5/3600) + 5 + 150(10/3600) + 0$.

6. Parameterized Timed Automata with Uncertain Durations

In Section 2.3 we introduced two formulations for PTA planning with uncertain duration. We also described the intuitions for two action models that underly these formulations and used our projector example to illustrate. In this section we define these action models as specializations of a more general model, called a *parameterized timed automata with uncertain duration (PTAUD)*.

Recall that in our planning formulation clock parameters are used to model activity duration. We constrain their assignments to values within intervals using clock parameter resets. To model uncertainty a PTA with uncertain duration partitions clock parameters into *controllable parameters*, which are assigned by a user or decision procedure, and *uncontrollable parameters*, which are assigned by nature. During PTA planning with uncertain durations, the values of controllable parameters are selected by the planner, one value for each clock parameter reset. When validating a candidate plan, the planner must consider that nature can choose any value allowed by these resets and must evaluate correctness relative to these possible outcomes. Possible values for uncontrolled parameters are also specified by clock resets. The difference is that values assigned by these resets are chosen by nature, not by the planner. Transition and reward specifications for PTA with uncertain durations are similar to those for PTA. They only differ for the resets of uncontrollable clock parameters, as described below.

The definition of a trace for a PTA with uncertain duration is similar to that for a deterministic PTA but is extended to account for the different outcomes of uncontrolled clock parameters. A trace of a PTA with uncertain duration, called an *outcome*, consists of a trajectory over input variables, a trajectory over controllable clock parameters and a trajectory over uncontrollable clock parameters. Similar to a PTA, a trace (outcome) for a PTA with uncertain duration must be consistent with its transition relation and initial state. The first two trajectories of an outcome are called its *controlled trace*, and are determined by the planner. The third trajectory is called its *uncontrolled trace* and is determined by nature. The same controlled trace may appear in multiple outcomes. The set of consistent outcomes of a controlled trace is called the *outcomes of the controlled trace* and represents all possible ways that nature influences the timing of the trace. Since time is real valued, this set is typically infinite.

We capture these concepts through the following definition.

Definition 15. Parameterized Timed Automaton with Uncertain Duration

A parameterized timed automaton with Uncertain Duration (PTAUD) extends a parameterized timed automaton $A = \langle \mathcal{L}, \mathcal{C}, \mathcal{U}_I, \mathcal{U}_c, \mathcal{T} \rangle$ by partitioning the set of clock parameters $\mathcal{U}_c$ into controllable and uncontrollable clock parameters, $\mathcal{U}_{cc}$ and $\mathcal{U}_{cu}$. A transition and reward specification for a PTAUD is the same as for a PTA, except for the reset specification for uncontrollable clock parameters $\mathcal{U}_{cu}$.

Given a trace T of A , the controlled trace T_C of T is the projection of T onto inputs $\mathcal{U}_I$ and controllable clock parameters $\mathcal{U}_{cc}$. Trace T is referred to as an outcome of T_C . T_C is a controlled trace of A if and only if it is a controlled trace of a feasible trace of A . A feasible outcome of controlled trace T_C consists of T_C and a trajectory over uncontrollable clock parameters $\mathcal{U}_{cu}$ whose composition is consistent with transition relation $\mathcal{T}$. The outcomes of T_C is the set of all feasible outcomes of T_C . The stochastic run of T_C , starting in s_0 at e_0 , is a set that consists of a run for each consistent outcome of T_C , starting in s_0 at e_0 . $\square$

Our two specializations of PTAUD, presented next, adopt the models of uncertain duration used by STNUs and pSTNs, respectively.

Our first model is called a PTA under uncertainty (PTAU) and is used by the PTA planning under uncertainty problem (Definition 2). A PTAU uses an interval to specify the possible values that an uncontrolled parameter can be assigned by a reset. This reset is analogous to a contingent link of an STNU, which constrains the possible values of an uncontrolled duration to an interval. Transition and reward specifications for a PTAU are identical to that of a deterministic PTA. The only difference is that the PTAU partitions clock parameters into controllable and uncontrollable subsets. When using sets to represent uncertainty during planning (e.g., robust control), reward is often taken to be the minimum reward R_{min} received for all possible outcomes of a control input. We include R_{min} in our definition.⁵

Definition 16. Parameterized Timed Automaton under Uncertainty

A parameterized timed automaton under Uncertainty (PTAU) is a parameterized timed automaton with uncertain duration $A = \langle \mathcal{L}, \mathcal{C}, \mathcal{U}_I, \mathcal{U}_c, \mathcal{T} \rangle$, with $\mathcal{U}_c$ partitioned into controllable and uncontrollable clock parameters, $\mathcal{U}_{cc}$ and $\mathcal{U}_{cu}$. A reset specification cps for an uncontrolled clock parameter $p_i \in \mathcal{U}_{cu}$ is the same as that for a controlled clock parameter, that is,

$$cps ::= true | (p_i \in [l_i, u_i]) | cps1 \wedge cps2,$$

where $[l_i, u_i] \subseteq \mathbb{R}_{\geq 0}$.

Given controllable trace T_C of A and start state s_0 at e_0 , the minimum reward of T_C is

$$R_{min}(T_C) = \min_{r \in S(T_C)} R(T_C, r),$$

where $S(T_C)$ is the set of stochastic runs of T_C starting in s_0 at e_0 . The stochastic run of T_C satisfies state plan SP over A if and only if every run of T_C 's stochastic run satisfies SP . $\square$

For a PCTA planning under uncertainty problem (Definition 2), recall that a solution, called a control plan, is a controllable trace of the problem's PTAU whose stochastic run satisfies the problem's state plan.

5. When uncertainty is specified as a deviation to a nominal control input, an alternative is to use reward of the input without deviation.

Our second action model is called a probabilistic PTA (pPTA) and is used by a chance-constrained PTA planning problem (Definition 3). A pPTA treats each uncontrolled clock parameter as a random variable, rather than a set of assignments. A reset of an uncontrolled clock parameter specifies a probability distribution over the clock parameter, which is the probability that the parameter takes on each value, when the reset is invoked.

The term *chance-constraint* is adopted from stochastic optimization. For a constrained optimization problem that includes random variables, a chance constraint $\langle C, \Delta \rangle$ specifies that the chance of violating constraint C must be at most probability Δ . intuitively, Δ is the acceptable risk of failure. For pPTA planning problem, we specify a maximum acceptable risk of violating the problem’s state plan. The planner maximizes expected utility while bounding this risk.

Definition 17. Probabilistic Parameterized Timed Automaton

A probabilistic parameterized timed automaton (pPTA) is a parameterized timed automaton under uncertainty $A = \langle \mathcal{L}, \mathcal{C}, \mathcal{U}_I, \mathcal{U}_c, \mathcal{T} \rangle$, with $\mathcal{U}_c = \langle \mathcal{U}_{cc}, \mathcal{U}_{cu} \rangle$. A reset to uncontrolled parameter $u_i \in \mathcal{U}_{cu}$ is the distribution $P(u_i | l_j, l_k)$ and specifies the probability of each reset assignment to u_i , upon transition from l_i to $l_j \in \mathcal{L}$.

Let T_O be an outcome for controllable trace T_C . Let $P(R_l) = P(u_i | l_j, l_k)$ be the distribution of the l th clock parameter reset that is invoked in T_O , and r_l be the value that the outcome assigns to u_i for this l th reset. Assuming independence of reset assignments, the probability of outcome T_O for controllable trace T_C is $P(T_O | T_C) = \prod_{r_i \in \mathbf{r}} P(r_i)$.

Given controllable trace T_C of A and starting state s_0 at e_0 , the reward $R(T_C)$ of T_C is the expected reward of the stochastic runs of T_C . The stochastic run of T_C satisfies state plan SP within risk Δ if and only if $(\sum_{T_O \in S_F} P(T_O | T_C)) \geq (1 - \Delta)$, where S_F is the set of outcomes of T_C whose run satisfies SP . $\langle SP, \Delta \rangle$ is referred to as a chance-constrained state plan. $\square$

In this definition we assume that the distributions corresponding to the resets are independent. We can easily relax this independence assumption by still treating each reset as a random variable, but then specifying a joint distribution over these variables.

7. State Plans

Our planning problem (Definition 1) specifies goals SP_{goal} using a plan-like representation, called a *state plan* [26], which we define in this section. A state plan specifies the desired state evolution of the problem’s automaton over time.⁶ The solution to our planning problem is a *control plan*, which is a trace that produces a run of the problem’s automaton. A feasible control plan is a trace whose run, starting in state s_0 , satisfies the problem’s state plan. We define control plans after state plans.

As introduced in Section 2, a state plan is comprised of *episodes* and *temporal constraints*. Each episode specifies constraints on the evolution of a subset of the automaton’s state variables over a bounded period of time. We can think of an episode as a type of activity that describes desired assignments over time.

In addition to episodes, a state plan is specified by a set of temporal constraints[13] that relate the start and end times of different episodes. These constraints leave flexibility in the time that an episode starts and ends and flexibility in the duration of the episode. A planner exploits this flexibility to find a trace that is feasible and optimal.

6. SP_{goal} may include *operating constraints* on the values of PCTA control variables.

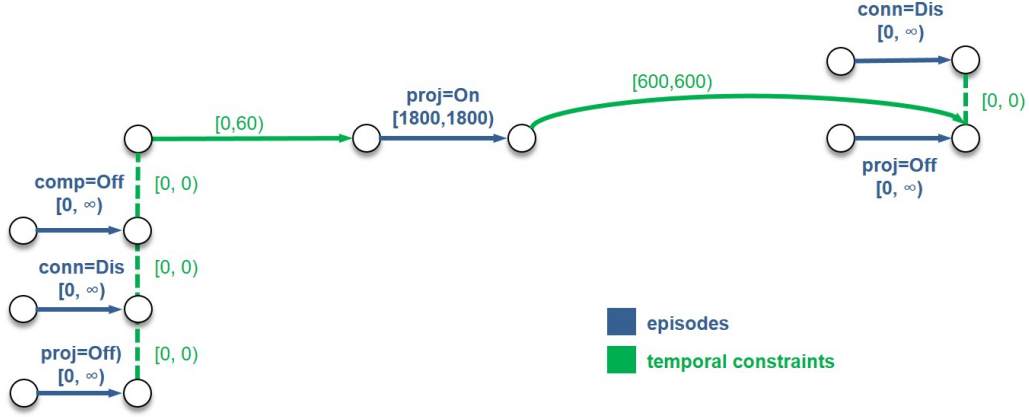


Figure 5: A state plan for a PCTA planning problem. Episodes (blue) express the desired states of the PCTA and the extent over which they need to be achieved. Temporal constraints (green) relate the start and end times of the episodes.

Figure 5 depicts a state plan for our projector example. This state plan constrains the behavior of the computer, connection and projector over time. In this state plan, blue arrows denote episodes and green arrows denote temporal constraints. The blue arrow labeled "proj=On" is an episode that specifies that the projector should be on. Each episode includes a state constraint, which constrains the values of state variables from start to end of the episode. Each episode includes an interval, which bounds the duration of the episode. This episode includes the label "[1800,1800]", which states that the projector is desired to be on for exactly 1,800 seconds, that is, 30 minutes. Temporal constraints that specify relationships between episodes are highlighted in green. For example, the green arrow labeled "[0,60]" indicates that the projector should be on at most 60 seconds after the computer and projector are off and disconnected, but not before off or disconnected.

7.1 State Plan Definition

In this section we define state plan in terms of constraints, using concepts like state and transition constraint, temporal constraint, event, ordering and schedule (see Appendix A). State plans are composed of episodes and temporal constraints, which relate distinguished events and the start and end events of different episodes. Episodes themselves are composed of state and temporal constraints. When defining behavior over time, we assume a continuous time model in which events range over the reals. The generalization of these definitions to a discrete time model is straight forward.

7.1.1 EPISODE

Episodes are the fundamental building blocks for describing state variable behavior over time, with each episode describing behavior over a bounded time interval. Each episode has a set of state variables V , whose trajectories are being constrained; start (s) and end (e) events, which delimit the interval over which the variables are constrained; a constraint sc over V , which constrains the trajectories of the variables between start and end, and, finally, a temporal constraint (tc), which constrains how long the episode occurs. tc requires, at a minimum, that $s \leq e$.

Definition 18. Episode

An episode ep is a quintuple $\langle V, s, e, tc, sc \rangle$. The set of state variables V is the scope of ep . $s, e \in \mathbb{R}$ are the start and end events of ep . Interval $[s, e]$ is the extent of ep . The extent is restricted by temporal constraint tc , called a duration constraint, where $s, e \in \text{scope}(tc)$. tc must imply that start s precedes or co-occurs with end e ($s \leq e$), and may further constrain the duration of the episode ($e - s$), also known as the temporal distance between s and e . V is restricted by a state constraint sc , for every $t \in (s, e)$. $\square$

Conceptually, an episode describes a segment of V 's state trajectory, and may be viewed as a constraint with scope V , which is stretched across time. A transition or state constraint c , which forms the episode's relation, can describe this trajectory exactly, or more abstractly. In the abstract case an episode can be thought of as a bundle of trajectories, called a *flow tube* [43, 22, 21]. ctBurton focuses on a simple case in which an episode denotes a state that is fixed over its duration. This fixed value must satisfy the episodes' state constraint.

Consider an episode that represents a segment of behavior for a parameterized timed automaton. Earlier we specified that each state of the automaton must hold for more than an instant, by imposing clock conditions that constrain each automaton location to be visited for a non-zero duration. To capture this, an episode for our automaton is lower bound by a positive duration; hence, $s < e$. Further, since all clock conditions involve non-strict inequalities, all episodes represent closed intervals of behavior.

7.1.2 STATE PLAN

We describe a set of trajectories by composing together episodes and by requiring that each state trajectory enforce every episode, according to this composition. A *state plan* composes episodes by enforcing the union of the episodes and by using temporal constraints to order and specify metric relationships between episode start and end events. These temporal constraints relate distinguished reference events, such as a deadline or plan start time, in addition to episode start and end events.

Definition 19. State Plan

A State Plan SP is a quintuple $\langle \mathcal{V}, e_0, EV, EP, TC \rangle$ where:

- $\mathcal{V} = \{v_i\}$, called the scope of SP , is a set of state variables,
- e_0 is the start event of SP , denoting a non-negative, real-valued time point.
- $EV = \{e_i\}$ is the set of events e_i of SP , denoting non-negative, real-valued time points. EV is partitioned into episode events EV_E , which are start and end events of $ep_i \in EP$, and reference events EV_R , which are not episode events.
- $EP = \{ep_i\}$ is a set of episodes of SP , where the scope of each ep_i is a subset of $\mathcal{V}$ and the start and end events of ep_i are in EV .
- $TC = \{tc_i\}$ is the set of temporal constraints tc_i of SP over events in EV . TC includes the constraint $e_0 \leq e_i$, for every event $e_i \in EV$, and the episode temporal constraints tc for every episode $ep_i \in EP$.

$\square$

The temporal constraints TC of a state plan include the duration constraints of the plan’s episodes. TC contains an additional set of constraints that relate the events across episodes. These temporal constraints order events and can impose metric constraints between events, such as “bread must be eaten within a day of baking.”

In Section 8, we first develop a PTCA “end-state” planner as a stepping stone towards a planner that handles expressive state plans. The end state planning problem, defined above, is a simplified version of the state plan planning problem that aligns closely to a classic planning problem. An end state goal specification includes a goal state that must be achieved at the end of a run; it is described by a state constraint c_g . In addition, it has a safety constraint that must be satisfied up until the goal is achieved; it is described by state constraint c_s . Finally, it includes a deadline $[l, u]$ by when this goal is achieved. This deadline specifies a lower bound as well as an upper bound on completion. An end state goal specification can be described by a state plan with two episodes in sequence and a single temporal constraint. The first episode enforces safety (c_s) and the second enforces the end goal (c_g). A temporal constraint from the start event of the first episode to the end of the second enforces the deadline.

7.2 Planner Output as a Control Plan

The primary output of a PCTA planning problem is called a *control plan*, and is a trace for the problem’s PCTA. A planner may optionally output the control plan’s run, starting in the problem’s initial state. This run is either in the form of a PCTA state trajectory or a PCTA transition trajectory. Control plans, state trajectories and transition trajectories are each returned in the form of a timed sequence, as defined in Section 5.2. A run can also be easily derived from the control plan and the problem statement. In this section we complete our problem formulation by defining solution correctness.

Informally, a control plan is a valid solution to a PCTA problem if it achieves the problem’s state plan SP_{goal} , when run on the problem’s PCTA, starting in problem initial state s_0 at time e_0 . Figure 6 depicts a control plan that achieves the state plan of our running example, along with its corresponding run. The control plan produces the effect of connecting the computer and projector, and then turning both on. To turn the projector off, the control trace breaks the connection by assigning `conn.cmd = disconnect`. The run illustrates how the PCTA location variables evolve as a result of the commands in the control plan, and shows that, when the connection with the computer is lost, the projector naturally transitions to `Waiting`, then `CoolDown` and finally to `Off`. More precisely,

Definition 20. Valid Solution to $Plan_{PCTA}$

Given a PCTA planning problem $Plan_{PCTA} = \langle A, e_0, s_0, SP_{goal} \rangle$, a control plan $SP_{control}$ of $Plan_{PCTA}$ is a trace of A . $SP_{control}$ is a valid solution to $Plan_{PCTA}$ if the run of $SP_{control}$ on A , starting in s_0 at e_0 , satisfies state plan SP_{goal} . $\square$

Finally we define what it means for a state trajectory, and hence a run, to satisfy a state plan. A state plan is a *path constraint* over state trajectories. Given a trajectory over the scope of a state plan, this trajectory *satisfies* the state plan if, first, there exists a schedule to the state plan’s events that satisfies the plan’s temporal constraints and, second, if each episode of the state plan is satisfied by the trajectory, given the episode’s assigned start and end time in the schedule. Given an episode, a trajectory over the state variable’s of the episode’s scope, and a schedule to the episode’s start and end times, the episode is satisfied by the trajectory under two conditions: first, the start and end

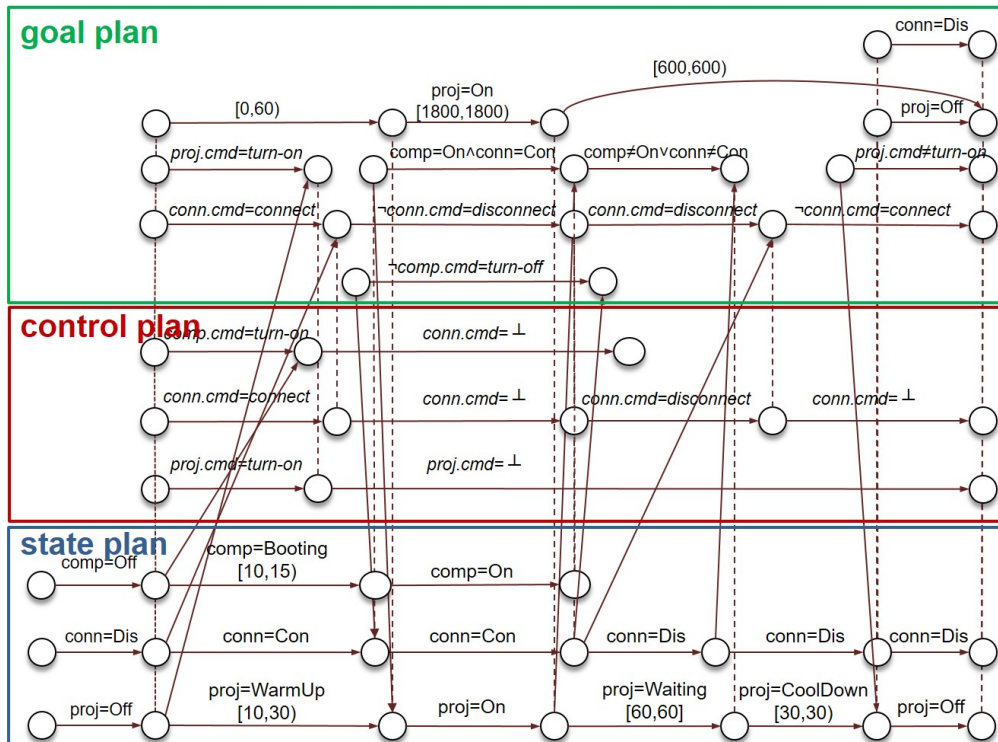


Figure 6: The solution to our PCTA planning problem, in the form of a control plan and its PCTA run. Also shown is the problem's state plan, repeated from Figure 5. **Make this figure's trace and run fully grounded. Explain connections to state plan.**

times satisfy the episode’s duration constraint and, second, the trajectory satisfies the episode’s state constraint for all times between the start and end.⁷

Definition 21. Trajectory Satisfies State Plan or Episode

Let SP be a state plan with scope $\mathcal{V}$, and $[l, u]$ be a time interval such that $e_0 \leq l$, where e_0 is the start event of SP .

Trajectory $\mathcal{V}(t)$ satisfies state plan SP over $[l, u]$ if there exists a schedule Γ to events $EV(SP)$ that satisfies temporal constraints TC of SP and $\mathcal{V}(t)$ satisfies each episode $ep_i \in EP(SP)$ over $[l, u]$, given schedule Γ .

A trajectory $\mathcal{V}(t)$ satisfies episode EP over $[l, u]$, given a schedule Γ that includes $\{s(EP), e(EP)\}$ if Γ satisfies $tc(EP)$ and for all $t_i \in [l, u] \cap (s(EP), e(EP))$, $sc[EP](\mathcal{V}(t_i))$. $\square$

In the way of summary, Figure 7 provides a factored description for our projector example, of the problem’s inputs and outputs in terms of traces and runs of the component automata – computer, projector and connection.

8. A Constraint Language for Encoding PCTA Planning Problems

In the remainder of this paper we present the ctBurton planner, which solves PCTA planning problems when time is certain (Definition 1) or uncertain (Definitions 2 and 3). ctBurton generates an optimal grounded trace and run of a PCTA that satisfies a goal state plan. ctBurton takes as input a PCTA planning problem, containing an automaton A with determinate transitions. A idles for at least a finite time in each location before making a state changing transition.⁸ In our deterministic formulation, all clock parameters are controllable, while our uncertain formulations include uncontrollable clock parameters. Key to ctBurton, in all three cases is its ability to plan with autonomous transitions.

ctBurton encodes each of the three PCTA planning problems as constraint programs, finds a valid solution to the respective constraint program, and returns the solution’s corresponding trace and run. We start with our deterministic problem, presenting encodings for both end-state and state plan PCTA planning problems (Definition 1). We show that this planner is sound, complete and optimal up to the problem’s planning horizon h (Section 9.5). We then modify this encoding to handle uncertain durations as specified Definitions 2 and 3.

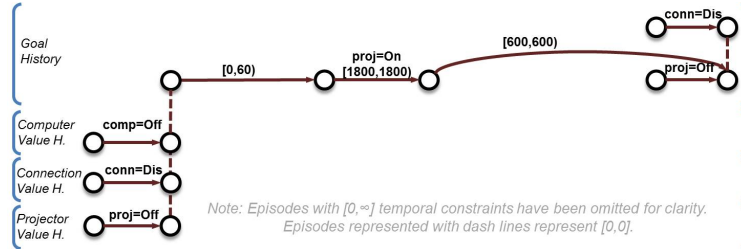
In this section we define a constraint modelling language we use to specify our encodings, and summarize the elements of suitable solvers. We then present our planner encodings in subsequent sections.

8.1 Mixed Logic Temporal and OpSat Temporal Problems

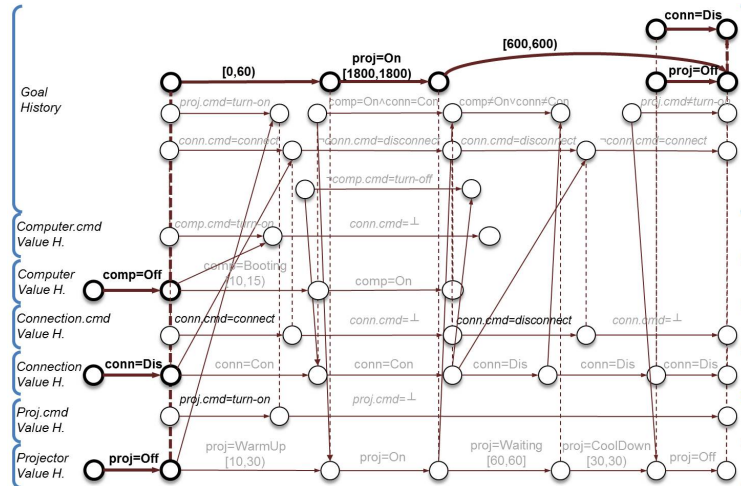
The constraint program encodings for ctBurton are in the form of mixed state logic temporal problems (MSLTPs), where constraints consist of propositional logic formula whose propositions include assignments to finite domain state variables and simple temporal constraints over real-valued events, or their uncertain variants. In this section we focus on deterministic temporal constraints (Definition 27) and incorporate uncertainty in the next section.

7. Note that an episode’s state constraint is enforced *between* start and end, but not *at* the start and end events.

8. This results from the requirement that the clock condition of a state changing transition includes a positive lower bound l_c (Definition 7).



(a) The initial state plan for the projector example, which is specified in the input problem. Events are represented as vertices. Episodes as edges. The lower and upper time bounds for episodes are labeled using interval notation. Is this a state plan or something else?



(b) The output master state plan for the projector example. The figure dark-ents the control variables that are important to plan execution, as well as the subset that is the input state plan.

Figure 7: Initial and master state plan for the projector example.

Definition 22. Mixed Logic Temporal Problem

A Mixed Logic Temporal Problem (MLTP) is denoted by a quintuple $\langle \mathcal{P}, \mathcal{V}, \mathcal{E}, \mathcal{F}, \mathcal{S} \rangle$, where $\mathcal{P}$ is a set of atomic propositions, $\mathcal{V}$ is a set of finite domain variables with corresponding domains $\mathcal{D}_{\mathcal{V}}$, $\mathcal{E}$ is a set of non-negative, real-valued events, $\mathcal{F}$ is a linear objective function that maps assignments of $\mathcal{P}$, $\mathcal{V}$ and $\mathcal{E}$ to the reals, and $\mathcal{S}$ is a set of mixed state logic temporal formula over $\mathcal{P}$, $\mathcal{V}$ and $\mathcal{E}$.

A mixed state logic temporal formula over $\mathcal{P}$, $\mathcal{V}$ and $\mathcal{E}$ is a propositional logic formula whose propositions are a) atomic propositions in $\mathcal{P}$, b) assignments of variables in $\mathcal{V}$ to elements of their domain, and c) simple temporal constraints over pairs of events in $\mathcal{E}$. $\square$

Consider how this definition applies to the computer automaton. An example of an atomic proposition is "enabled(idling)," an example of an assignment to a control variable is "cmd = turn-off", and an example of a simple temporal constraint is " $t^+(\text{shutdown}) - t^-(\text{shutdown}) \in [20, 30]$."

Simple temporal constraints use closed intervals, for example, $[20, 30]$. ctBurton also needs semi-closed intervals to encode PTA clock conditions, for example, $c_i \in [l_i, u_i)$. Within an MLTP we can express a semi-closed interval using a simple temporal constraint plus a disequality that excludes the upper bound,

$$c_i \in [l_i, u_i) \equiv c_i \in [l_i, u_i] \wedge c_i \neq u_i.$$

STN algorithms that operate directly on semi-closed intervals is an open area for future research.

Noet that we can also express difference constraints within a MLTP, where a difference constraint is an inequality constraint involving the difference between two variables,

$$e_j - e_i \geq l \equiv e_2 - e_1 \in [l, \infty].$$

A wide range of solvers exist that embed strict and non-strict inequalities within propositional logic constraints, but do not support simple temporal constraints natively. In this case we can reduce a simple temporal constraint to two difference constraints,

$$e_2 - e_1 \in [l, u] \equiv e_2 - e_1 \geq l \wedge e_2 - e_1 \leq u,$$

which these solvers support.

Finally, we can encode a strict inequality using a non-strict inequality and a disequality,

$$c_i < u_i \equiv c_i \leq u_i \wedge c_i \neq u_i.$$

A mixed logic temporal problem can be easier to solve through restrictions that exploit additional problem structure. For ctBurton we focus on a special case of a MLTP called a temporal OpSat problem (TOP), in which the objective function is over discrete propositions and finite domain variables, but not real-valued events. This is the combinatorial optimization part of the OpSat problem. Real-valued events appear only in temporal feasibility constraints, and represent the satisfiability part of the OpSat problem. As discussed in Section 8.3 an OpSat solver can achieve efficiency by employing two subsolvers, tailored to the combinatorial optimization and temporal satisfiability subproblems.

Definition 23. Temporal Optimal Satisfiability Problem (TOP)

A Temporal Optimal Satisfiability Problem is a mixed logic temporal problem $\langle \mathcal{P}, \mathcal{V}, \mathcal{E}, \mathcal{F}, \mathcal{S} \rangle$, whose linear objective function $\mathcal{F}$ is over atomic propositions $\mathcal{P}$ and finite domain variables $\mathcal{V}$, but not real-valued events $\mathcal{E}$. $\square$

Suppose we restrict a PCTA planning problem so that state changes have reward, but idling transitions have no reward. ctBurton can encode this problem as a temporal OpSat problem, since real-valued events do not appear in the objective, only discrete choices of which transitions to enable.

This restriction is both expressive and lends itself to efficient solution. Associating zero reward with idling is consistent with PDDL planning formulations, which rarely specify reward for maintaining state. In addition, a wide range of efficient algorithms exist for testing temporal satisfiability, both for deterministic and uncertain temporal problems.

There are also many real-world problems where we would like to receive reward, or pay a cost, for maintaining state. A simple example is the energy consumed by keeping a computer on. If we want reward to be a function of duration, then the reward function for our planner encoding includes real-valued events, hence the formulation is a MLTP.

The following was removed; either fold into the solver subsection or delete:

... where temporal constraints may alternatively be expressed as difference constraints. Note that many MiniZinc solvers support linear inequality constraints that involve strict as well as non-strict inequalities, making it easier to encode semi-closed intervals.

8.2 OpSat Temporal Problems with Uncertain Durations

- **Incorporate STNU and pSTN to MLTPs.**
 - **Add chance constraint.**
 - **Confirm - we do not need to separate requirement and contingent constraints since Δ is over both types of constraints.**
- **Make sure Section 8 flows.**
- **Leverage uncertainty notes.**

8.3 Solving Temporal Opsat Problems (TOP)

In this paper the bulk of our planning benchmarks specify positive reward on state changing transitions and zero reward for idling. This allows us to encode these problems as temporal optimal satisfiability (TOP) problems. We compare solution time and quality using a range of constraint solvers, and compare to metric PDDL planners that perform heuristic forward search. We also provide evaluation for some benchmarks with positive idling reward using publicly available mini-Zinc solvers. The development of efficient MLTP solvers for this general case is an area of future research, which we discuss in Section 13.

ctBurton uses the OpSat solver [41] to solve TOP encodings that involve a mix of controlled and uncontrolled duration, which are modeled using intervals or distributions. As a TOP solver, OpSat is unique in its ability to handle uncontrolled durations. OpSat solves these problems using specialized temporal subsolvers [34, 33, 15].

When all durations in the planning problem are controllable, a wide range of solvers can be employed that support the open-source MiniZinc constraint modeling language, the constraint encoding language used by ctBurton. We provide a benchmark evaluation of ctBurton on a range of these solvers, including OpSat.

To elaborate, OpSat is a hybrid solver that employs two sub-solvers, the first generates candidate assignments by using combinatorial optimization (e.g., tree search using branch and bound), while the second subsolver tests candidates by employing incremental temporal consistency. OpSat uses forward conflict-directed search to coordinate between the two subsolvers. When the tester proves that a candidate is inconsistent, it hands the candidate generator a constraint that summarizes the source of inconsistency, called a conflict. A conflict is a subset of the candidate assignment that is inconsistent. The candidate generator uses the conflict during the splitting step of search to avoid other candidates that are inconsistent for the same reason.

OpSat is organized similar to SMT solvers[3] that support optimization (called OMT solvers). The candidate generator operates on propositional clauses that are an abstraction of mixed logical temporal formula, with temporal constraints replaced with atomic propositions. The candidate truth assignment is used to identify a set of temporal constraints that need to be checked for consistency. Opsat uses a propositional state logic formulation in which propositions are assignments to finite domain variables, including Boolean variables. A candidate is an assignment to these finite domain variables plus a truth assignment to atomic propositions. The objective function is over these assignments.

The candidate generator performs search by splitting on resolutions of each conflict, with the effect of steering the next candidate away from known inconsistencies. When all conflicts are resolved, search continues by splitting on assignments to unassigned variables. Splitting is interleaved with incremental unit propagation[30, 29]. To find an optimal solution, search is ordered using either branch and bound or A*, depending on whether the best or multiple good solutions are requested.

Once the candidate generator provides the best candidate assignment to the discrete decision variables, OpSat’s child temporal sub-solver first formulates a temporal problem. This is the set of temporal constraints that correspond to atomic propositions assigned true in the candidate. The subsolver then checks consistency of the temporal subproblem (STN, STNU or pSTN) and returns a conflict when inconsistent.

Consistency is performed efficiently using graph algorithms that extract conflicts through negative loop detection and support incremental inference by exploiting similarities between successive candidates[34, 33, 15].

9. Planning with Controlled Durations

Next we specify a mixed logic temporal encoding for PCTA hybrid planning and scheduling over a finite horizon.

We present a global state encoding that is analogous to most constraint-based activity planners that employ constraint programming and most trajectory planners that employ mathematical programming. Initially, we assume that each automaton location has a single idling transition. These transitions may be autonomous, meaning that the idling time is bound and the location must be departed after that time. This assumption is made both for pedagogical simplicity and compactness of the encoding. Later we drop this assumption, by developing an extended encoding that allows for multiple idling transitions per automaton.

To start we express the space of candidate traces and runs of a PCTA using the concept of a constraint-based trellis. Next we augment this encoding for constraints representing the problem’s initial state and a linear reward for state changing and idling transitions over the plan horizon. Finally, we describe how to encode end goals, and later state plans, as accepting PTA. These are

added to the problem’s action model as components of the concurrent automaton and then encoded as mixed logic temporal constraints.

9.1 Encoding the Space of Candidate Traces and Runs

In this section we describe a constraint-based encoding for finite horizon PCTA behavior called a *constraint-based trellis*. In general, a trellis diagram describes a space of possible trajectories that a dynamical system can produce. In our case, a trellis diagram represents the planner’s candidate space of PCTA traces and runs, up to a horizon of h events. Our description applies to both primitive and concurrent automata.

In this section we use A to denote a parameterized timed automaton that follows Definition 5. To review, A is comprised of location and clock variables $\mathcal{L}$ and $\mathcal{C}$, input variables $\mathcal{U}_I$, clock parameters $\mathcal{U}_P$, input clock variables $\mathcal{U}_{I_C}$, input clock parameters $\mathcal{U}_{I_P}$, and transitions $\mathcal{T}$. If A is concurrent, each variable type is a vector, built from its component automata.

A run of A is punctuated by a series of state changing transitions, with A maintaining its location between these transitions. As discussed in Section 8, we initially constrain our input automaton to have a single idling transition per location, hence our encoding assumes that A maintains a location by enabling a single idling transition between state changing events. Later we relax this assumption by allowing an automaton to have multiple idling transitions per location and by introducing an encoding that permits two or more idling transitions to be enabled in sequence, with the result of maintaining a location for a longer period of time between state changing events. This generality, however, is at the cost of additional decision variables and stages.

We start by introducing a set of decision variables whose assignments denote candidates and then introduce constraints that limit the trellis diagram to legal traces and runs.

9.1.1 ENCODING TRAJECTORIES USING SEQUENCES OF STAGES

A trajectory of automaton A over $\mathcal{W} = \mathcal{V} \cup \mathcal{U}$ denotes both a trace of A and its corresponding run, where A is primitive or concurrent. We describe this trajectory with a time stamped sequence and bound the sequence with horizon h , which is the number of time stamped values in the sequence. The trellis describes the space of such sequences of A of length h using a sequence of decision variables $\langle e_0, \mathcal{W}_0, \dots, e_{h-1}, \mathcal{W}_{h-1}, e_h \rangle$, where $e_i \in \mathcal{E}$ denotes the i th real-valued event in the sequence and $\mathcal{W}_i$ denotes a value that holds for $\mathcal{W}$ from event e_i to event e_{i+1} .⁹ ctBurton enforces the total ordering on events with the ordering constraint

$$\forall 0 \leq i < h . e_i < e_{i+1}. \quad (1)$$

Each $\mathcal{W}_i$ is partitioned into $\langle \mathcal{L}_i, \mathcal{C}_i, \mathcal{U}_{I_i}, \mathcal{U}_{P_i}, \mathcal{U}_{I_C i}, \mathcal{U}_{I_P i} \rangle$, analogous to how $\mathcal{W}$ is partitioned into $\langle \mathcal{L}, \mathcal{C}, \mathcal{U}_I, \mathcal{U}_P, \mathcal{U}_{I_C}, \mathcal{U}_{I_P} \rangle$. We specify clock variables $\mathcal{C}_i$ slightly different from the other variables, and defer its introduction for a moment.

Intuitively, we think of the constraint-based trellis in terms of a sequence of *stages*, numbered from 0 to $h - 1$, where a new stage begins whenever a value of $\mathcal{W}$ changes from its preceding stage. The triple $\langle e_i, \mathcal{W}_i, e_{i+1} \rangle$ denotes the i th stage of the trellis, where the stage starts at event e_i , ends at event e_{i+1} , and maintains $\mathcal{W}_i$ from e_i to e_{i+1} . To be consistent with our notation for start and end events, we equivalently denote e_i by e_i^- and e_{i+1} by e_{i+1}^+ . We denote the trellis of an automaton

9. We use both e_i and $e(i)$ to denote the i th event and both X_i and $X(i)$ to denote the i th instance of variable X .

A by $Tr(A)$ and the i th stage of this trellis by $Tr(A, i)$, or equivalently $Tr_i(A)$. We sometimes reference an automaton trellis by its location variable(s), that is, $Tr(\mathcal{L}) = Tr(A)$, where $\mathcal{L}$ is the set of location variables of A . Later we generalize trellises to arbitrary state variables. Finally, when the trellis being referenced is unambiguous, we refer its i th stage simply by its index i , as a short form for $Tr(A, i)$.

Given an assignment to variables of a constraint-based trellis for automaton A , ctBurton maps this assignment to a time stamped sequence that describes a trace and run of A . Specifically, the assignments of each stage map to a pair in a time stamped sequence, where the state assignment of a stage is the first element of the pair and the value assigned to the stage's start event is the second element of the pair. A time stamped sequence is concise as long as successive pairs have distinct state assignments. Suppose that successive pairs have identical state assignments, then the sequence is not concise, since an equivalent but smaller description can be constructed by simply dropping the second of the successive pairs from the sequence.

For a primitive automaton, a stage specifies an assignment to the automaton's location variable and its input variables, as well as its clock variables and parameters. A stage ends when the location variable changes value. For a concurrent automaton a stage specifies an assignment to the variables of all component automata, including location variables, clock variables and clock parameters. A stage ends when any location variable changes value. A consequence of this encoding, for better or worse, is that it totally orders the times when the component automata change locations. In future work we will introduce a factored encoding that has the potential to be more compact, by relaxing the requirement to totally order events.

An important subtlety of our encoding is that, as discussed earlier, we consider the value of $\mathcal{W}$ to be ambiguous at the events between stages, which denote times when values change. We model this ambiguity by having successive stages $(\mathcal{W}_i, \mathcal{W}_{i+1})$ overlap at their shared event e_{i+1} , and allow $\mathcal{W}$ at e_{i+1} take on the values of either $\mathcal{W}_i$ or $\mathcal{W}_{i+1}$. The value of $\mathcal{W}_i$ is, however, uniquely defined over the open interval between the stage's start and end events. Hence, when specifying constraints that encode our trellis, we only reference values of $\mathcal{W}$ within these open intervals.

Note that clock variables $\mathcal{C}$ increase monotonically with time and have a slope of 1. The exception is when a clock is reset by a state changing transition, which occurs at an event that separates two stages. To track these resets, for each Stage i , we introduce *clock offset* $c_{ij} \in \mathcal{C}_i$, which denotes the most recent time that clock variable c_j was reset. To derive the value of the clock c_j from its offset, we note that the value of c_j at the start of Stage i is then $e_i - c_{ij}$ and the value at the end of Stage i is $e_{i+1} - c_{ij}$.

It is quite common to use a sequence of states, like our stages, within a constraint encoding of plan space. Numerous constraint-based activity planners and model-predictive controllers employ stages with an atomic or fixed time step. These are called fixed-time encodings. It is much less common for the duration of these stages to vary; these are called continuous time encodings. Planners that employ a continuous time encoding, similar to ctBurton, include constraint-based hybrid and continuous planners, like cKongming[6] and Magellan[31], and relaxed plan heuristics for heuristic forward search temporal and hybrid planners, like Crikey, Colin and Optic[9, 10, 11]. Planners based on these continuous time formulations demonstrate significant run time improvement over planners that employ discrete time encodings.

In the remainder of this section we introduce constraints on the variables of a trellis that restrict assignments to those that represent valid traces and runs of parameterized timed automaton A .

9.1.2 CONSTRAINTS THAT DEFINE INDIVIDUAL TRANSITIONS

We first introduce constraints that specify the behavior of individual enabled transitions $\tau \in \mathcal{T}$ according to Definitions 6 and 7. We use these constraints to model the transitions of primitive automata, both when constructing a trellis for a concurrent automaton and when constructing a trellis for a single primitive automaton alone.

Recall that each transition τ is a relation

$$\tau(\mathcal{V}(t - \epsilon), \mathcal{U}(t - \epsilon), \mathcal{V}(t + \epsilon), \mathcal{U}_P(t + \epsilon))$$

as $\epsilon \rightarrow O^+$. The automaton locations just before and after the transition, denoted $\mathcal{L}(t - \epsilon) \subset \mathcal{V}(t - \epsilon)$ and $\mathcal{L}(t + \epsilon) \subset \mathcal{V}(t + \epsilon)$, must satisfy the source and target locations. In addition, $\mathcal{C}(t - \epsilon) \subset \mathcal{V}(t - \epsilon)$ and $\mathcal{U}_P(t - \epsilon) \subset \mathcal{U}(t - \epsilon)$ denote assignments to clock variables and clock parameters, respectively, and $\mathcal{U}_i(t - \epsilon) \subset \mathcal{U}(t - \epsilon)$ denotes assignments to inputs, which occur just before the time of transition. Each of these variables may appear in the transition's guard condition. Finally, $\mathcal{C}(t + \epsilon) \subset \mathcal{V}(t + \epsilon)$ and $\mathcal{U}_P(t + \epsilon) \subset \mathcal{U}(t + \epsilon)$ denote resets to clock variables and parameters, and hold just after transition.

Also recall that a transition τ is specified by $\langle ss, ts, \phi, cs \rangle$, comprised of source specification ss , target specification ts , guard specification ϕ and clock reset specification cs . ϕ is divided into state condition ϕ_s and clock condition ϕ_c , while cs is divided into clock variable reset specification cvs and clock parameter reset specification cps .

By Definition 6, if τ is enabled at time t it must satisfy three conditions, which we need to encode:

1. τ 's source and target hold just before and after t ,
2. τ 's guard, whose disjuncts each contain a state and a clock condition, is satisfied just before t , and
3. τ 's reset, consisting of variable and parameter resets, holds just after t .

In other words,

$$\begin{aligned} & ss(\mathcal{L}(t - \epsilon)) \wedge ts(\mathcal{L}(t + \epsilon)) \wedge \\ & \bigvee_k \left(\varphi_{s,k}(\mathcal{L}(t - \epsilon), \mathcal{U}_I(t - \epsilon)) \wedge \varphi_{c,k}(\mathcal{C}(t - \epsilon), \mathcal{U}_{IC}(t - \epsilon), \mathcal{U}_P(t - \epsilon), \mathcal{U}_{IP}(t - \epsilon)) \right) \wedge \quad (2) \\ & cvs(\mathcal{C}(t + \epsilon)) \wedge cps(\mathcal{U}(t + \epsilon)). \end{aligned}$$

To apply a transition specification to each stage of a trellis, we rewrite these conditions in terms of the values associated with each stage, rather than the values associated with each time t . Some notation is needed to refer to these values. We write $ss(\mathcal{L}_i)$ to indicate that ss is satisfied by assignment $\mathcal{L}$ at stage i and $ts(\mathcal{L}_i)$ when this assignment satisfies ts . Likewise, we write $\varphi_{s,k}(\mathcal{L}_i, \mathcal{U}_{I_i})$ when $\varphi_{s,k}$ is satisfied at stage i by assignments $\mathcal{L}$ and $\mathcal{U}_I$. We defer our discussion of clock conditions $\varphi_{c,k}$ and resets cvs and cps for a moment.

Finally, we introduce proposition $En_o(\tau, i)$, which is true exactly when transition τ is enabled over the open interval of stage i and we introduce proposition $En_e(\tau, i)$, which is true exactly when τ is enabled at the event that ends stage i , that is, e_i^+ .

Next we use these conditions and propositions to specify constraints for when a transition is enabled throughout the open interval of a stage and when a transition is enabled at a state changing event, which separates successive stages.

Transitions Within a Stage Within a stage's open interval, we only need to encode the enablement of idling transitions, not state changing transitions; consider why. Recall that a transition is an idling transition if its source and target have the same value; otherwise, it is a state changing transition. Within the open interval of a stage i , the location variables $\mathcal{L}_i$ maintain a single value for all events t in this open interval, hence any transition enabled during this open interval must have the same source and target value, thus by definition the transition idles.

For each state changing transition τ and each stage i , where $0 \leq i < h$, we specify that it cannot be enabled throughout the stage by introducing constraint

$$\neg \text{En}_o(\tau, i). \quad (3)$$

We encode enablement of an idling transition τ throughout stage i by

$$\text{En}_o(\tau, i) \implies ss(\mathcal{L}_i) \wedge \bigvee_k (\varphi_{s,k}(\mathcal{L}_i, \mathcal{U}_{I_i}) \wedge \varphi_{co,k}(\mathcal{C}_i, \mathcal{U}_{I_{C_i}}, \mathcal{U}_{P_i}, \mathcal{U}_{I_{P_i}})). \quad (4)$$

cvs and cps have been dropped, since idling transitions do not include resets. In addition, $ts(\mathcal{L}_i)$ is dropped, since it is redundant with $ss(\mathcal{L}_i)$. We described $ss(\mathcal{L}_i)$ and each $\varphi_{s,k}(\mathcal{L}_i, \mathcal{U}_{I_i})$ above. All that remains is for us to specify clock conditions $\varphi_{c,k}$ over a stage's open interval, in terms of stage variables $\mathcal{C}_i$ and $\mathcal{U}_{P_i}$. We denote this open interval condition $\varphi_{co,k}$.

Recall from Definition 7 that a clock condition for an idling transition is specified by

$$\varphi_{c,k} ::= \bigwedge_j c_j \in [0, u_{c,j}],$$

where $c_j \in \mathcal{C}_i \cup \mathcal{U}_{I_{C_i}}$ is a single clock variable. In addition, each $u_{c,j}$ is a constant or clock parameter in $\mathcal{U}_{P_i} \cup \mathcal{U}_{I_{P_i}}$, which are non-negative and real-valued. For τ to be enabled over stage i , $\varphi_{c,k}$ must hold for every event in i 's open interval.

To show that this condition holds throughout the open interval, a key insight is that it suffices to evaluate $\varphi_{c,k}$ at i 's end event, e_i^+ . To determine the value of clock c_j at this event, recall that we introduced clock offset $c_{ij} \in \mathcal{C}_i$, which denotes the most recent time that clock c_j was reset. As a consequence, the clock at the end of i is $c_j(e_i^+) = e_i^+ - c_{ij}$. Returning to $\varphi_{c,k}$, $c_j \in [0, u_{c,j}]$ holds throughout stage i if it holds at i 's end, that is, $e_i^+ - c_{ij} \in [0, u_{c,j}]$. Pulling this together, ctBurton uses the encoding

$$\varphi_{co,k}(\mathcal{C}_i, \mathcal{U}_{I_{C_i}}, \mathcal{U}_{P_i}, \mathcal{U}_{I_{P_i}}) ::= \bigwedge_j (e_i^+ - c_{ij} \in [0, u_{c,j}]). \quad (5)$$

Replace difference with clock cj at end of stage i.

Transitions Between Stages State changing transitions, which are instantaneous, occur at the end events that separate stages. After a state changing transition, an automaton always idles before its next state changing transition, which occurs at the end of the subsequent stage. This behavior results from ctBurton's requirement that the clock condition of each state changing transition include a positive lower bound l_c (Definition 7). When concurrent automata are executed, multiple transitions occur at each stage boundary and these transitions may include idling transitions, as well as state changing transitions.

We encode the enablement of a state changing or idling transition τ at the end of stage i by

$$\begin{aligned} En_e(\tau, i) \implies & ss(\mathcal{L}_i) \wedge \bigvee_k (\varphi_{s,k}(\mathcal{L}_i, \mathcal{U}_{I_i}) \wedge \varphi_{ce,k}(\mathcal{C}_i, \mathcal{U}_{I_{C_i}}, \mathcal{U}_{P_i}, \mathcal{U}_{I_{P_i}})) \\ & \wedge ts(\mathcal{L}_{i+1}) \wedge cvs(\mathcal{C}_{i+1}) \wedge cps(\mathcal{U}_{i+1}). \end{aligned} \quad (6)$$

This encoding includes the target assignment and conditions for clock and parameter resets, which were dropped in the encoding within a stage. These three elements are listed at the end of the encoding and are evaluated on the assignment for successor stage $i + 1$.

Clock conditions $\varphi_{c,k}$ are evaluated at event e_i^+ and are denoted $\varphi_{ce,k}$. This is in contrast to our development in the preceding subsection, where clock conditions are evaluated over an interval. The value of clock c_j at e_i^+ is $e_i^+ - c_{ij}$, which is the clock's value just before the transition performs any resets. The clock condition for an idling transition is of the form $\varphi_{c,k} ::= \bigwedge_j (c_j \in [0, u_{c,j}])$, and is encoded as

$$\varphi_{ce,k}(\mathcal{C}_i, \mathcal{U}_{P_i}) ::= \bigwedge_j (e_i^+ - c_{ij} \in [0, u_{c,j}]). \quad (7)$$

Replace difference with clock cj at end of stage i.

The clock condition for a state changing transition is of the form $\varphi_{c,k} ::= \bigwedge_j (c_j \in [l_{c,j}, u_{c,j}])$ and is encoded as

$$\varphi_{ce,k}(\mathcal{C}_i, \mathcal{U}_{I_{C_i}}, \mathcal{U}_{P_i}, \mathcal{U}_{I_{P_i}}) ::= \bigwedge_j ((e_i^+ - c_{ij}) \in [l_{c,j}, u_{c,j}]). \quad (8)$$

Replace difference with clock cj at end of stage i.

Next we encode the clock variable reset, which is specified by cvs . These resets can only involve the clock variables of the automaton, not its input clock variables. Recall from Definition 7 that $cvs ::= \text{true} \mid vasn_1 \dots \wedge vasn_n$, where $vasn_i ::= (c := v)$, $c \in \mathcal{C}$ and $v \in \mathbb{R}_{\geq 0}$. ctBurton enforces that additional constraint that clocks can only be reset to 0 (hence $v = 0$). If clock variable c_j is reset at end event e_i^+ , then the clock offset $c_{i+1,j}$, for the next stage $i + 1$, takes on e_i^+ , which is the time of the reset. To encode reset cvs at the end of stage i , ctBurton introduces constraint

$$cvs(\mathcal{C}_{i+1}) ::= \forall (c_j := 0) \in cvs. En_e(\tau_k, i) \rightarrow c_{i+1,j} = e_{i+1}. \quad (9)$$

Replace clock offset with clock cj at end of stage i, and set to 0.

In the next section we discuss the case where clock c_j is not reset by a transition, and the clock offset persists into the next stage.

Finally, we encode clock parameter resets cps in an analogous manner. These resets, likewise, can only involve the clock parameters of the automaton, not its input clock parameters. Recall that cps is of the form $cps ::= \text{true} \mid pasn_1 \dots \wedge pasn_n$, where $pasn_i ::= (p_j \in [l, u])$, $p_j \in \mathcal{U}_P$ and $[l, u] \subseteq \mathbb{R}_{\geq 0}$. While a variable reset assigns 0 to a clock, a parameter reset constrains the domain of a parameter to an interval $[l, u]$. ctBurton then selects a consistent value for each reset parameter from its domain. To encode cps at the end of stage i , ctBurton introduces constraint

$$cps(\mathcal{U}_{i+1}) ::= \forall (p_j \in [l, u]) \in cps. En_e(\tau_k, i) \rightarrow p_{i+1,j} \in [l, u]. \quad (10)$$

In the next section we discuss the case where parameter p_j is not reset, and is treated similar to clocks.

9.1.3 CONSTRAINTS THAT COMPOSE TRANSITIONS

In the previous section we specified constraints that enforce the definition of a transition being enabled. In this section we include additional constraints that ensure that an assignment to variables of a trellis denotes a valid trace and its run. These constraints relate transitions that are contained within the same or successive stages. In this section we introduce three types of constraints: constraints that maintain clock offsets and clock parameters when they are not reset, constraints that ensure that runs are determinate, and constraints that ensure that a sequence of stages offers a concise description of a run.

Making Clock Variables and Parameters Persist In the last section we introduced constraints that assign new values to clock variables and parameters according to the resets of enabled state changing transitions. When a variable or parameter is not reset at the end of a stage, its value needs to be maintained in the next stage. In this section we introduce constraints that enforce this behavior.

Recall that each clock c_j of A is computed using a clock offset $c_{i,j}$, which is specified for each stage i , where $0 \leq i < h$. Suppose that a clock c_j is not reset at the end of stage i . ctBurton then carries c_j 's clock offset forward to the next stage, by enforcing constraint $c_{i+1,j} = c_{i,j}$. To determine whether clock c_j is being reset at each stage, ctBurton first constructs the set $T(c_j)$, denoting all transitions that reset c_j . Using this set, ctBurton then persists a clock offset when not reset, by introducing the following constraint for each stage i ,

$$\text{maintain}(\mathcal{C}_{i+1}) ::= \forall c_j \in \mathcal{C}. \left(\bigwedge_{\tau_k \in T(c_j)} \neg \text{En}_e(\tau_k, i) \right) \rightarrow c_{i+1,j} \equiv c_{i,j}. \quad (11)$$

Replace the constraint on clock offset an update to the clock c_j at end of stage i , as c_j for the previous stage plus the duration of the stage. In addition, durations need to be positive.

The antecedent of this implication checks whether any transition that resets c_j is enabled in stage i . If this is not the case, its consequent carries the clock offset forward to the next stage.

Values of clock parameters are maintained in a similar manner. Let $T(p_j)$ denote all transitions that reset clock parameter p_j . Using the following constraint, ctBurton maintains the assignment to p_j across successive stages where no reset occurs,

$$\text{maintain}(\mathcal{U}_{i+1}) ::= \forall p_j \in \mathcal{U}. \left(\bigwedge_{\tau_k \in T(p_j)} \neg \text{En}_E(\tau_k, i) \right) \rightarrow p_{i+1,j} \equiv p_{i,j}. \quad (12)$$

Constraints that Ensure Deterministic Execution ctBurton requires that action model A is a deterministic automaton. This means that A 's run is described by a unique state assignment at every time and A takes a unique transition at each time. When encoding a trellis for a primitive automaton A_p , ctBurton includes the constraint $\text{Determinate}(A_p, n)$ to ensure that only one transition is taken at a time. Determinate breaks into two constraints. It uses constraint Determinate_o to specify that exactly one of A_p 's transitions is enabled throughout a stage, and it uses constraint Determinate_e to specify that exactly one transition is enabled at an event between stages,

$$\begin{aligned} \text{Determinate}(A_p, n) ::= \\ \text{Determinate}_o(A_p, n) \wedge (\forall 1 \leq i < n. \text{Determinate}_o(A_p, i) \wedge \text{Determinate}_e(A_p, i)). \end{aligned} \quad (13)$$

To define Determinate_o and Determinate_e , let $S_o(A_p, i)$ and $S_e(A_p, i)$ be sets of propositions that denote the enablement of A_p 's transitions, both during and at the end of stage i , respectively.

These sets are described by

$$S_o(A_p, i) ::= \{En_o(\tau_k, i) | \tau_k \in \mathcal{T}_I(A_p)\} \quad (14)$$

and

$$S_e(A_p, i) ::= \{En_e(\tau_k, i) | \tau_k \in \mathcal{T}(A_p)\}, \quad (15)$$

where $\mathcal{T}(A_p)$ is the set of transitions of A_p and $\mathcal{T}_I(A_p)$ is the set of idling transitions of A_p .

ctBurton specifies the "Determinate" constraint for intervals and points by

$$Determinate_o(A_p, i) ::= OneOf(S_o(A_p, i)) \quad (16)$$

and

$$Determinate_e(A_p, i) ::= OneOf(S_e(A_p, i)), \quad (17)$$

where *OneOf* is a global constraint that is applied to a set of propositions P and imposes the constraint that exactly one $p \in P$ is True.¹⁰

Next consider a constraint-based trellis for a concurrent automaton A_c . A_c is guaranteed to be determinate if each of its component automata are determinate. To enforce this, ctBurton introduces the corresponding constraint,

$$Determinate(A, n) ::= \bigwedge_{A_p \in \mathcal{A}(A)} Determinate(A_p, n), \quad (18)$$

where $\mathcal{A}(A)$ denotes the component automata of A and each component automaton is a primitive automaton.

Intuitively, assuming one idling transition per location, a run of a primitive automaton is an alternating sequence of state changing and idling transitions. For the run of a concurrent automaton A , a determinate transition of A is described by a set of enabled transitions consisting of exactly one enabled transition for each component automaton. A run of A is a sequence of these sets that alternate between sets that contain at least one state changing transition and occur at an event, and sets that contain only idling transitions and occur over an open interval. More about this next.

Constraints that Ensure Concise Runs Recall that ctBurton maps a trellis assignment to a time stamped sequence by mapping each assignment of each stage to a time stamped value. This sequence is concise as long as successive values have distinct state assignments. ctBurton generates trellis assignments that produce concise sequences by constraining the state assignments of successive stages to be different. ctBurton accomplishes this by requiring that at least one state changing transition is enabled at each event between two stages.

To enforce this constraint over n stages, ctBurton first constructs the set $T_{sc}(A)$, denoting all state changing transitions of A . It then enforces a constraint *concise*(i), for every stage i with a successor,

$$\forall 0 \leq i < h - 1. \text{concise}(i), \quad (19)$$

where

$$\text{concise}(i) ::= \bigvee_{\tau_k \in T_{sc}(A)} En_E(\tau_k, i). \quad (20)$$

10. The constraint defining *Determinate_o* tightens the set of transitions to idling transition, by incorporating Constraint 3, which specifies that no stage changing transition can be enabled during a stage's open interval. Using this formulation, Constraint 3 can be dropped.

If A is a concurrent automaton, this constraint ensures that at least one component automaton changes its location between stages. It is perfectly valid, however, for all other component automata to maintain their location.

9.2 Encoding the Initial State

To encode initial state s_0 within a constraint-based trellis, recall that state assignments are maintained over the open interval of each trellis stage and change at events that delineate stages. State assignments are not unique at the event between stages, since the state is in transition at that time.

s_0 is defined to hold at e_0^+ , where the trellis begins at e_0 . All stages have a positive duration, since the clock condition of a transition always has a positive lower bound. No stage can hold for only an instant. As a consequence, to enforce the problem's initial state, we record s_0 as the assignment for the open interval of Stage 0, which is the first stage of a constraint-based trellis.

9.3 Encoding the Cost of a Run

Net we specify reward of a candidate run within the constraint-based trellis. PCTA reward was specified in Definitions 5, 6 and 7. Recall that reward is specified as an instantaneous rate ρ that is integrated over the extent of a run. For an instantaneous, state changing transition, this rate is an impulse and the reward for taking this transition is the coefficient of the impulse. The reward for idling in a location is the rate of the idling transition times the duration that the location is maintained. Along these lines, we specified reward for a run in Definition 10 and reward for a trajectory sequence of that run in Definition 14. We add a reward function to the constraint-based trellis that is similar to the equation for a run's trajectory sequence (Definition 14).

Each stage of the trellis contains an interval over which all location and control variables maintain their value, for example, by idling, followed by an instant in which one or more variables change their value, for example, through state changing transitions. The reward $R(Tr_i(A))$ of stage i of automaton A is then the reward of the idling transitions that maintain each location times the duration of the stage, plus the reward of each state changing transition, which occurs at the end of the stage. Because the run being evaluated is for a concurrent automaton, we sum the reward of transitions that are enabled for each component automaton,

$$R(Tr_i(A)) = \sum_{0 \leq i < h} \sum_{A_j \in \mathcal{A}} \left[\left(\sum_{\tau_k \in \mathcal{T}_I(A_j)} En_o(\tau_k, i) r(\rho_k) \right) (e_i^+ - e_i^-) + \sum_{\tau_k \in \mathcal{T}_C(A_j)} En_e(\tau_k, i) r(\rho_k) \right] \quad (21)$$

Recall that e_i^- and e_i^+ are the start and end times of stage i , and $e_i^+ - e_i^-$ is the stage's duration. $r(\rho_k)$ is the value of the reward associated with transition τ_k . For component automaton A_j , $\mathcal{T}(A_j)$ denotes A_j 's transitions, which are divided into idling transitions $\mathcal{T}_I(A_j)$ and state changing transitions $\mathcal{T}_C(A_j)$. Proposition $En_o(\tau, i)$ indicates that transition τ_k is enabled over the open interval of stage i and proposition $En_e(\tau, i)$ indicates that τ_k is enabled at the end of stage i . In this equation propositions $En_o(\tau, i)$ and $En_e(\tau, i)$ are automatically coerced to Booleans (1,0), where they appear within a multiplication.

To interpret Equation 24, note that for each stage i , it computes the reward contributed by each automaton A_j . This reward consists of a reward for idling the location of A_j , times the duration of stage i , plus the reward for changing the location A_j if it changes at the end of stage i . These two

rewards correspond to the two terms within the inner square bracket. Term

$$\left(\sum_{\tau_k \in \mathcal{T}_I(A_i)} En_o(\tau_k, i) r(\rho_k) \right)$$

selects the reward for the single idling transition of A_i during stage i . Likewise,

$$\left(\sum_{\tau_k \in \mathcal{T}_C(A_i)} c_p(En_e(\tau_k, i)) r(\rho_k) \right)$$

selects the reward of an enabled state changing transition of A_i if such a transition occurs.

Note that we do not consider reward for state changing transitions during a stage, since state changing transition are disallowed during stages. We don't accumulate reward of idling transitions that occur at the end event of a stage because this reward approaches zero at an instant. On the other hand, since the reward of a state changing transition is model by an impulse function, and it's value is non-zero at the stage's end event.

In Equation 24, reward is a linear function of both discrete variables ($En_o(\tau_k, i)$, $En_e(\tau_k, i)$) and real-valued variables (e_i^- , e_i^+). Including this reward function hence results in a mixed-logic temporal problem (MLTP). As discussed in Section 8.3 we reformulate a MLTP to a mixed logic linear program (MLLP), which is solved by available MILP or MiniZnc solvers.

If the reward of all idling transitions is zero, then Equation 24 simplifies to,

$$R(Tr_i(A)) = \sum_{0 < i < h} \sum_{A_j \in \mathcal{A}} \left[\sum_{\tau_k \in \mathcal{C}_I(A_i)} c_p(En_e(\tau_k, i)) r(\rho_k) \right]. \quad (22)$$

In this equation reward is over only discrete variables, while real-valued temporal events only appear in constraints. Hence the encoding reduces to a temporal optimal satisfiability problem. The OpSat solver, in particular, achieves efficiency by exploiting a decomposition between a combinatorial optimization subproblem and a temporal satisfiability subproblem (Section 8.3).

9.4 Encoding End State Goals

We complete our basic planner with an encoding of end state goals. After adding temporal uncertainty to our planner in the next section, we finish by generalizing or goal specification encoding to handle state plan planning problems. To encode goals in both cases, rather than adding constraints to the planning trellis directly, we encode the goal specification using accepting automata. Each accepting automaton is expressed as a deterministic PCTA and composed with the PCTA action model of the problem. The composed PCTA is then translated to a constraint-based trellis.

Each accepting automaton includes a location named "goal-achieved". The accepting automaton must transition to this location as soon as the goal specification is satisfied. It then must idle in this location until the plan horizon is reached. A candidate trace of a trellis is accepted as a solution if its corresponding run contains location "goal-achieved" as active in its final stage.

For an end state problem, the goal specification consists of goal constraint $gc(\mathcal{V})$ and maintenance constraint $mc(\mathcal{V})$, both of which are state constraints over state variables $\mathcal{V}$. The goal constraint must hold at some event between the start and end of the plan. The maintenance constraint must hold from $e_0 = 0$ to just before the goal constraint is satisfied, or beyond. We assume that gc does not hold in initial state e_0 . This is easy to check before setting up the planner encoding.

Accepting Automaton for End-State Problems

- Inputs: $\mathcal{V}$.
 - Internal Variables: clock variable c and parameter δ .
 - Locations: Waiting, Accepted and Failed.
 - Transitions:
 - Idle on Waiting when $mc(\mathcal{V}) \wedge \neg gc(\mathcal{V})$;
 - Waiting to Accepted when $gc(\mathcal{V})$;
 - Waiting to Failed when $\neg mc(\mathcal{V}) \wedge \neg gc(\mathcal{V})$;
 - Idle on Accepted always ($\top$), and
 - Idle on Failed always ($\top$).
 - Start: Waiting.
-
- **David, do you think we should include figures of these automata, or just leave the figures as text?**
 - **Create diagrams for end-state accepting automaton.**
 - **Put text description in an appendix.**

Figure 8: PCTA Encoding of an end goal accepting automaton, where the goal is achieved after e_0 .

Given our PCTA execution semantics, the goal constraint holds for an open interval with positive duration and the maintenance constraint holds for at least the open interval between e_0 and the event that starts the interval over which the goal constraint is first satisfied.

We encode the accepting automaton for an end state problem by the PTA shown in Figure 8. This makes it easy to incorporate into the constraint-based trellis, thus ensuring that runs satisfy the goal specification. the PTA contains locations Waiting, Accepted and Failed, where Waiting has a start transition. The automaton checks $mc(\mathcal{V})$ while in the Waiting location, and transitions to Accepted when $gc(\mathcal{V})$ is satisfied. Alternatively, it transitions to Failed if $mc(\mathcal{V})$ is violated before success.

9.5 Soundness, Completeness, Minimality and Optimality

Develop proofs of soundness, completeness and minimality

One of the key properties of this transition encoding is that it correctly generates a sequence of stages, where each stage has the maximum extent. By maximum extent we mean that the values of all location variables idle and the end event of each stage corresponds to the first change of a location variable or input variable value.

The encoding of an idling transition ensures that it is flagged as enabled for a stage only if its guard condition holds throughout the extent of the stage, to its end event.

The encoding of a state changing transition ensures that it is flagged as enabled only at the end of a stage, and only enabled if its guard is satisfied at the end event.

⟨ Explain how the encoding produces the desirable effect of successively catching the first transitions that occur next. ⟩

10. Planning with Uncertain Durations

- **Update temporal constraints.**
 - **Modify the temporal constraints of the encoding for uncontrolled durations to use contingent links or probabilistic temporal constraints.**
 - **For the probabilistic formulation, construct a chance constraint from the temporal constraints using Δ .**
- **Update reward.**
 - **For STNU, update reward to reward for mean duration. This is equivalent to expected reward assuming a uniform distribution.**
 - **For pSTN, update reward to an expectation.**
- **Update definitions.**
 - **Update All problem definitions.**
 - **Reference supporting equations when giving updated equations.**

Encoding Resets for Uncontrolled Parameters At the end of each stage, recall that a subset of the clock parameters are reset by active state changing transitions, while the values of remaining parameters are carried forward. For deterministic parameters, the clock parameter reset cps of a transition is a set of assignments $p_j \in [l, u]$, where p_j is a clock parameter and l and u are constants, denoting a positive interval. To encode cps for the end of stage i ctBurton introduces constraint

$$cps(\mathcal{U}_{i+1}) ::= \forall(p_j \in [l, u]) \in cps.En_e(\tau_k, i) \rightarrow p_{i+1,j} \in [l, u]. \quad (23)$$

During planning, for each controllable reset ctBurton assigns its parameter a consistent value from its reset interval.

Add resets of uncontrolled parameters

Expected Cost of a Run Recall that the reward for a run of automaton A is the accumulated reward $R(Tr_i(A))$ for each of its stages i . The reward of a stage is the reward of the idling transitions that maintain each location times the duration of the stage, plus the reward of each state changing transition THAT occurs at the end of the stage. For a concurrent automaton, we sum the reward of enabled transitions for each component automaton,

$$R(Tr_i(A)) = \sum_{0 \leq i < h} \sum_{A_j \in \mathcal{A}} \left[\left(\sum_{\tau_k \in \mathcal{T}_I(A_i)} En_o(\tau_k, i) r(\rho_k) \right) (e_i^+ - e_i^-) + \sum_{\tau_k \in \mathcal{T}_C(A_i)} En_e(\tau_k, i) r(\rho_k) \right] \quad (24)$$

Add expected Reward

11. State Plan Planner

Thus far our planner encoding has focused on problems whose goal specification is comprised of an end goal, maintenance constraint and deadline. In this section we return to our full problem statement, in which goals are specified as state plans (Definition 1). Recall that state plans complement metric temporal logic as a way for users to intuitively sketch desired behavior within a plan-like representation. Similar to our end state planner, we reformulate a state plan as a set of concurrent PTA accepting automata. These automata are simply added to the component automata of A and encoded within A 's constraint-based trellis, along with its other components.

We introduced a definition for state plan in Section 7. Recall that a state plan $SP = \langle \mathcal{V}, e_0, EV, EP, TC \rangle$ is comprised of a set of state variables $\mathcal{V}$, a set of events EV , which includes start event e_0 , a set of episodes EP , and a set of simple temporal constraints TC over these events. An episode includes a state constraint that holds for all states between the episode start and end, and a simple temporal constraint that bounds the duration of the episode to $[l, u]$. A run satisfies a state plan if a) there exists a schedule for EV that satisfies temporal constraints TC and b) the run satisfies the state constraint of each episode, given this schedule.

Our concurrent accepting automaton encoding is comprised of a component automaton for each episode, temporal constraint and event. These component automata are shown in Figures 9, 10 and 11. Temporal constraint and episode automata check consistency of their respective element, while event automata “glue” these elements together, by checking that the elements synchronize at their shared events. When applied to a run, each component automaton transitions to its Accepted or Failed location as soon as it reaches a time in the run where its respective element is traversed and satisfied or violated.

Temporal constraint and episode elements can take on a range of possible durations between their two events, as defined by interval $[l, u]$. To consider each duration, an element's respective automaton uses a clock parameter to make a non-deterministic choice of the element's duration, chosen from $[l, u]$. To measure this duration within a run, the automaton of each element starts a clock at the time that its first event is reached.

The component accepting automata concurrently process a run from start to end in a coordinated manner. As time t progresses along a run, the automata incrementally process their respective events, episodes, and temporal constraints along the run up to t . The automaton for a temporal constraint or episode element idles in a wait location until one of its events has been accepted by an event automaton, at which point the automaton becomes Active. The element automaton for the episode or temporal constraint then evaluates the run from this first event up to its second event. It transitions to Accepted or Failed, depending on whether the run is consistent with respect to the element's temporal constraint or episode.

In addition, an automaton for this end event concurrently waits until the time of the end event is reached. It then checks, for all elements with this event as an end event, that the event is accepted and the time assigned to this shared end event is consistent. Based on this analysis the end event automaton transitions to Accepted or Failed.

Below we describe these three types of accepting automata. Each is represented as a PCTA with deterministic transitions and a Failed location, and can easily be incorporated in a constraint-based trellis using the encoding described in Section 9.1.

In addition, we specify constraints between events and clock parameters that reflect the simple temporal network of the state plan. While this set of constraints is already implied by the accepting automaton, making these constraints explicit makes scheduling easier.

Episode Accepting Automaton Recall that a state plan episode $E = \langle V, s, e, tc, sc \rangle$ is comprised of state variables V , which are location and input variables, start and end events s and e , temporal constraint $tc : e - s \in [l, u]$, and state constraint sc over V . Given a schedule to s and e , a run satisfies episode E if the distance from s to e is in $[l, u]$ and the run satisfies $sc(V)$ for every state between s and e .

The accepting automaton for E is shown in Figure 9. It includes clock $c(E)$ and clock parameter $\delta(E)$, which we simplify to c and δ when the context is clear. The automaton begins in its Wait location, where it idles until start event s is accepted by s 's event automaton. When condition $\text{Accepting}(s)$ is true, E 's automaton transitions to the Active location, simultaneous with s 's acceptance. At this time it chooses a duration for E that is consistent with tc , by resetting clock parameter δ with range $[l, u]$. The location Active checks assignment $V(t)$ against sc , while $t \in (s, s + \delta)$. If sc is violated at any point in this time interval, the automaton transitions from the Active to the Failed location. Otherwise, the automaton idles at Active, until $t = s + \delta$. At this point the run satisfies the behavior specified by Episode E and the automaton transitions to the Accepted location. For the transition from Active to Failed, note that its guard condition is $\neg sc(V)$ rather than $\neg sc(v) \wedge c \geq \delta$. $c \geq \delta$ is not needed and dropped, since Active always exits before $\delta < c$.

If start event s is e_0 , then the automaton has a start state at Active, rather than Waiting. This start issues resets $c = 0$ and $\delta \in [l, u]$.

The episode accepting automaton also maintains the truth of a proposition Accepting , which is used by the accepting automaton for episode's end event. Accepting is assigned true just before episode's end event is reached and the episode is accepted,

$$\text{Accepting} \equiv \text{Active} \wedge sc(V) \wedge \delta \leq c.$$

Clock parameter $\delta \in [l, u]$ is the chosen duration of episode E . c is a clock that was started when s was accepted and E 's automaton became active.

Finally, the encoding introduces a constraint, between the episode's clock parameter and events, that reflects E 's simple temporal constraint,

$$e(E) - s(E) = \delta(E).$$

This constraint is instantiated on $\delta(E, i)$ for every stage in the constraint-based trellis.

Temporal Constraint Accepting Automaton Next consider $tc \in TC$, where $tc : e_2 - e_1 \in [l, u]$. tc specifies that the temporal distance from e_1 to e_2 is bounded by $[l, u]$. We assume tc is canonicalized so that $0 \leq u$ and $l \leq u$. A temporal constraint is similar to an episode; it includes two events and an interval bound on the distance between them. It differs in two respects. Most importantly, the events are unordered. In addition, it has no state condition.

To construct our accepting automaton for tc , we start with the episode accepting automaton (Figure 9) and modify it to produce the accepting automaton for tc , shown in Figure 10. In both figures we see the progression of locations Waiting, Active and Accepted, from left to right.

First we remove conditions that enforce the state constraint, by replacing sc with $\top$ in the guards of the transitions out of the Active location. The transition from Active to Failed can no longer be enabled, hence we remove the Failed location and its associated transitions.

Accepting Automaton for Episode E

- Inputs: V , event $s(E)$ accepting automaton.
 - Internal Variables: clock variable c and clock parameter δ .
 - Locations: Waiting, Active, Accepted, and Failed.
 - Transitions:
 - idle on Waiting when $\neg \text{Accepting}(s)$,
 - Waiting to Active when $\text{Accepting}(s)$; reset $c = 0$ and $\delta \in [l, u]$,
 - idle on Active when $sc(V)$ and $c < \delta$,
 - Active to Accepted when $sc(V)$ and $\delta \leq c$,
 - Active to Failed when $\neg sc(V)$,
 - idle on Accepted always ($\top$), and
 - idle on Failed always ($\top$).
 - Start:
 - When $s = e_o$,
 - * start at Active with reset $c = 0$ and $\delta \in [l, u]$;
 - Otherwise,
 - * start at Waiting.
 - Parameter Constraints:
 - $e(E) - s(E) = \delta(E, i)$ for every stage i .
-

- **Create diagrams for episode accepting automata for the typical case.**
- **Put text description in an appendix.**

Figure 9: PCTA Encoding of an episode accepting automaton for the typical case where the episode starts after e_0 .

Second, We generalize the automaton so that e_1 and e_2 can occur in any order, that is, e_1 precedes e_2 , e_1 follows e_2 , or e_1 and e_2 co-occur. These three cases correspond to the three disjuncts of the following constraint, which reframes tc :

$$(e_2 - e_1) \in (0, u] \vee (e_1 - e_2) \in (0, -l] \vee (e_1 - e_2) = 0.$$

The accepting automaton distinguishes between these three cases using propositions $\text{Accepting}(e_i)$ for the two events. $\text{Accepting}(e_i)$ is assigned true by e_i 's accepting automaton just before e_i is accepted. At time $t = e_0$, accepting automata for e_1 and e_2 start in their Waiting locations. As t progresses, just before t reaches e_1 , proposition $\text{Accepting}(e_1)$ becomes true and just before t reaches e_2 , $\text{Accepting}(e_2)$ becomes true.

tc 's automaton includes four transitions from its Waiting location, based on the truth of these propositions. First, neither e_1 nor e_2 is about to occur if $\neg\text{Accepting}(e_1) \wedge \neg\text{Accepting}(e_2)$ is true, hence tc 's automaton idles on Wait. Second, e_1 is about to occur and precede e_2 if $\text{Accepting}(e_1) \wedge \neg\text{Accepting}(e_2)$ is true, hence tc 's automaton contains a transition to Active. This transition includes a parameter reset that chooses a duration from $(0, u]$, which is the delay before e_2 occurs. The transition also issues a clock reset, in order to track delay from e_1 as t progresses. Third, e_2 is likewise about to occur and precedes e_1 if $\neg\text{Accepting}(e_1) \wedge \text{Accepting}(e_2)$ is true, hence tc 's automaton contains a transition to Active. This transition issues a parameter reset that chooses a duration in $(0, -l]$ that denotes the delay before e_1 occurs. The transition also issues a clock reset, in order to track the delay from e_2 . Finally, e_1 and e_2 are about to co-occur if $\text{Accepting}(e_1) \wedge \text{Accepting}(e_2)$ is true, hence tc 's automaton transitions directly to Accepted, since both events occur without delay.

Once tc 's automaton reaches Active, it counts off time until the selected delay is reached, similar to the episode automaton, and then transitions to Accepted, where it idles.

Next consider the placement of the start state. In the typical case, where e_1 and e_2 occur after e_0 , the start state of tc 's automaton is to Waiting. However, if e_1 and e_2 both occur at the start of a run ($e_1 = e_2 = e_0$) then the automaton's start state is Accepted. If e_1 begins at the start of the run alone ($e_1 = e_0 < e_2$), then the automaton's start state is Active and issues resets $c = 0$ and $\delta \in (0, u]$ upon start. Finally, if e_2 begins at the start of the run alone ($e_2 = e_0 < e_1$) then the automaton's start state is Active and issues resets $c = 0$ and $\delta \in (0, -l]$ upon start.

The temporal constraint accepting automaton also maintains a proposition Accepting , which is used by the accepting automaton for episode's end event. Accepting is assigned true just before the temporal constraint's end event is reached and the temporal constraint is accepted,

$$\text{Accepting} \equiv \text{Active} \wedge \delta \leq c.$$

Clock parameter $\delta \in [l, u]$ is the chosen duration between the constraint's first and second event. c is a clock that was started by tc 's automaton when it found its first Active event and became Active.

Finally, the encoding introduces a constraint between the clock parameter and events of tc ,

$$|e_2(tc) - e_1(tc)| = \delta(tc),$$

which is equivalent tc . Note that absolute value is used since δ is the distance from the event that occurs first (e_1 or e_2) to the event that occurs second (e_2 or e_1). This constraint is instantiated on $\delta(tc, i)$ for every stage i in the constraint-based trellis. While tc is also enforced by tc 's accepting automaton, alone it requires δ to be "guessed" until a value is found that respects tc . The addition of the constraint above uses constraint propagation to avoid this search process.

Accepting Automaton for Temporal Constraint TC

- Inputs: events $e_1(tc)$ and $e_2(tc)$ accepting automata.
- Internal Variables: clock variable c and clock parameter δ .
- Locations: Waiting, Active, and Accepted.
- Transitions:
 - If $l \leq 0$:
 - * Idle on Waiting when $\neg \text{Accepting}(e_1) \wedge \neg \text{Accepting}(e_2)$;
 - Waiting to Accepted when $\text{Accepting}(e_1) \wedge \text{Accepting}(e_2)$;
 - Waiting to Active when $\text{Accepting}(e_1) \wedge \neg \text{Accepting}(e_2)$; reset $c = 0$ and $\delta \in (0, u]$;
 - If $l < 0$:
 - * Waiting to Active when $\neg \text{Accepting}(e_1) \wedge \text{Accepting}(e_2)$; reset $c = 0$ and $\delta \in (0, -l]$;
 - Idle on Active when $c < p$;
 - Active to Accepted when $p \leq c$, and
 - Idle on Accepted always ($\top$).
- Start:
 - When $e_1 = e_2 = e_0$,
 - * start at Accepted;
 - When $e_1 = e_o \neq e_2$,
 - * start at Active with reset $c = 0$ and $\delta \in (0, u]$;
 - When $e_2 = e_o \neq e_1$,
 - * start at Active with reset $c = 0$ and $\delta \in (0, -l]$;
 - Otherwise,
 - * start at Waiting.
- Parameter Constraints:
 - $|e_2(tc) - e_1(tc)| = \delta(tc, i)$, for every stage i .

-
- **Create diagram for a temporal constraint accepting automaton for the common case, in which $e_0 < e_1, e_0 < e_2$.**
 - **Put text description in an appendix. Include the different start cases.**

Figure 10: PCTA Encoding of a temporal constraint accepting automaton for the common case where $e_0 < e_1, e_2$.

Event Accepting Automaton Episodes and temporal constraints of a state plan imply a simple temporal network that an event schedule must satisfy. Automata that accept temporal constraints, in the form of episode and temporal constraint automata, focus on satisfying their respective temporal constraints individually. Specifically, given an assignment to one event of a temporal constraint, its corresponding automaton chooses a value for its second event that must satisfy the constraint.

This approach ensures local consistency but does not ensure global consistency. The problem is that an event can be shared between temporal constraints. This is used, for example, to synchronize activities. For example, if two episodes share an end event, then the episodes must end together. It is not consistent, for example, for one episode to finish first, and wait for the second episode to finish, before moving forward.

To ensure global consistency, temporal constraint automata need to select durations so that their shared end events agree on their value. Synchronization is ensured for the first event of a temporal constraint that is assigned, and need not be checked. The event's value is externally assigned and adopted by each temporal constraint and episode automata that it triggers. The challenge is to ensure consistency of the assignment to a constraint's second event, which is assigned locally by the constraint's automaton. The purpose of an event automaton is to confirm that all temporal constraints that mention the automaton's event agree on the event's assignment.

The event automaton for an event $e_i \in Ev$ is shown in Figure 11. It contains the standard structure of Waiting, Accepted and Failed locations, with idling transitions on each. Locations Accepted and Failed idle always, while Waiting idles when it doesn't transition to Accepted or Failed.

To determine transitions to Accepted or Failed for event e_i , we first identify episodes that end at e_i ,

$$Ep(e_i) \equiv \{ep \in EP \mid e(ep) = e_i\},$$

and temporal constraints that contain e_i ,

$$TC(e_i) \equiv \{tc \in TC \mid e_1(tc) = e_i \vee e_2(tc) = e_i\}.$$

To accept time t as the assignment to event e_i , e_i 's automaton needs to check agreement on t among all automata of episodes and constraints that assign e_i . The automaton of every episode in $Ep(e_i)$ assigns e_i and needs to be checked. An automaton for a temporal constraint in $TC(e_i)$ assign e_i , and needs to be checked, if it is in its Active location at time t . We do not need to check an automaton of a constraint in $TC(e_i)$ that is in the Waiting location at t , since neither of its events is assigned. Instead, the automaton is waiting for a value to be agreed upon for e_i .

Next consider how an event automaton confirms agreement. The elements that mention event e_i agree on the assignment of t to e_i under two conditions. First, if proposition $\text{Accepting}(ep, t)$ is true for each episode $ep \in Ep(e_i)$. Second, proposition $\text{Accepting}(tc, t)$ is true for each constraint in $tc \in TC(e_i)$ that is active, that is, $\text{Active}(tc, t)$ holds.

When expressing this condition, it is key that we use $\text{Accepting}(tc, t)$, not $\text{Accepted}(tc, t)$. This is because $\text{Accepting}(tc, t)$ only holds at the instant t that constraint tc ends. As a consequence, using $\text{Accepting}(tc, t)$ ensures that all elements, that is episodes and active temporal constraints, end at the same time. On the other hand, $\text{Accepted}(el, t)$ holds for any time t after element el ends. Using Accepted would allow the elements to end at different times and acceptance by the event automaton would occur at the time when the last element ends.

Given this, the transition from Waiting to Accepted occurs when

$$\bigwedge_{ep \in EP(e_i)} \text{Accepting}(ep) \wedge \bigwedge_{tc \in TC(e_i)} (\text{Active}(tc) \wedge \text{Accepting}(tc) \vee \text{Waiting}(tc)).$$

The transition from Waiting to Failed occurs if at least one relevant element is Accepting and one is not Accepting,

$$\left[\bigvee_{ep \in EP(e_i)} \text{Accepting}(ep) \vee \bigvee_{tc \in TC(e_i)} \text{Active}(tc) \wedge \text{Accepting}(tc) \right] \wedge \left[\bigvee_{ep \in EP(e_i)} \neg \text{Accepting}(ep) \vee \bigvee_{tc \in TC(e_i)} \text{Active}(tc) \wedge \neg \text{Accepting}(tc) \right].$$

In addition, the Waiting location idles if no relevant element is Accepting,

$$\bigwedge_{ep \in EP(e_i)} \neg \text{Accepting}(ep) \wedge \bigwedge_{tc \in TC(e_i)} (\text{Active}(tc) \wedge \neg \text{Accepting}(tc) \vee \text{Waiting}(tc)).$$

Finally, we introduce a decision variable for each $e_i \in E$, which is used to schedule the events of the state plan. For all events e_i other than e_0 , we assume that e_i strictly follows e_0 , that is, $\forall i \neq 0. e_0 < e_i$. we used these decision variables at the end of the sections on episode and temporal constraint accepting automata, to introduce constraints that relate their respective events to their clock parameters.

12. Notes - Simple Temporal Networks Under Uncertainty

A common temporal constraint representation for modeling uncertainty is the simple temporal network under uncertainty (STNU) [STNU]. Like an STN, the values of controlled durations are set by a scheduler, while uncontrolled durations are set by nature and can take on any value within their interval. A *schedule* of an STNU is an assignment to each controlled duration and an *outcome* extends this schedule with an assignment to the uncontrolled durations, drawn from the intervals of the uncontrolled durations. A schedule is said to be consistent under *strong controllability* if every outcome of the schedule is consistent with the STNU.

13. Future Work & Conclusion

Systems that solve *combined* activity planning and scheduling problems are essential for a wide range of real world problems. To date a rich set of methods has been developed to solve activity planning problems alone and a rich set of methods have been developed to solve scheduling problems alone. While existing systems for the combined planning and scheduling problem have promising results, they tap into only a fraction of what can be achieved by combining recent advances for planning and for scheduling.

Our paper helps to bridge this gap. First we introduce an action that support rich models of state change and duration. We start with concurrent timed automata as a rich action model that supports autonomous and non-autonomous behaviors, and has a clean execution semantics in terms of traces and runs. To express planning goals, we incorporate state plans as an intuitive plan-like representation that complements existing formalisms based on metric temporal logic. To express

Accepting Automaton for Event e_i

- Inputs: accepting automata for elements
 - $Ep(e_i) \equiv \{ep \in EP | e(EP) = e_i\}$,
 - $TC(e_i) \equiv \{tc \in TC | e_1(tc) = e_i \vee e_2(tc) = e_i\}$.
 - Internal Variables:
 - $e_i = 0$ if $i = 0$;
 - $e_i \in \mathbb{R}^+$, otherwise.
 - Locations: Waiting, Accepted, and Failed;
 - Transitions:
 - Idle on Waiting when $\bigwedge_{tc \in TC(e_i)} (\text{Active}(tc) \wedge \neg \text{Accepting}(tc) \vee \text{Waiting}(tc))$;
 - Waiting to Accepted when $\bigwedge_{tc \in TC(e_i)} (\text{Active}(tc) \wedge \text{Accepting}(tc) \vee \text{Waiting}(tc))$;
 - Waiting to Failed when $(\bigvee_{tc \in TC(e_i)} \text{Active}(tc) \wedge \neg \text{Accepting}(tc)) \wedge (\bigvee_{tc \in TC(e_i)} \text{Active}(tc) \wedge \neg \text{Accepting}(tc))$;
 - Idle on Accepted always ($\top$), and
 - Idle on Failed always ($\top$).
 - Start:
 - When $e_i = e_0$,
 - * start at Accepted;
 - Otherwise,
 - * start at Waiting.
-

- **Create diagram for an event accepting automaton.**
- **Put text description in an appendix. Include start cases for normal case and e_0 .**

Figure 11: PCTA Encoding of an event accepting automaton for $e_i > e_0$.

coupled planning and scheduling problems, our key innovation is the concept of parameterized timed automata, where clock parameters mirror the behavior of clock variables. Different types of clock parameters are supported that model both controllable duration and uncontrolled duration that are set bounded or probabilistic.

To bridge the gap between methods for scheduling and for planning, we take a constraint-based approach and provide an encoding of the PCTA planning problem as a constraint optimization problem over mixed logical and temporal constraints. To facilitate comparison, we provide this encoding in the open standard constraint modeling language MiniZinc. This makes it easy for us to employ a range of publicly available constraint solvers, out of the box, to solve this mixed planning and scheduling problem. Our MiniZinc encoding also offers the promising of enabling constraint programming researchers to make advances in the performance of mixed planning / scheduling systems without becoming planning experts.

Finally, this paper demonstrates how we can leverage a range of advanced schedulers for dealing with controlled and uncontrolled duration, simply by plugging them in as subsolvers within the hybrid solver OpSat.

We end this paper by highlighting two opportunities for future work that leverage additional advances in scheduling – methods for solving problems with preference over duration and problems involving dynamic scheduling.

13.1 Solving Problems with Rate of Reward

Reward is often a function of how long activities can take. We modeled this by expressing reward as a rate per unit time, and by encoding the planning problem as a mixed logic temporal problem. When this encoding has only controllable durations, a straight forward approach to solving the problem is to reformulate the problem as a mixed logic linear program (MLLP) [23], by mapping each simple temporal constraint to a pair of inequality constraints. The MLLP can then be solved by using a propositional logic to integer linear program reduction, to reformulate the MLLP to a mixed integer linear program (MILP). A rich set of state of the art MILP methods are available, such as Cplex.

More efficient, alternative approaches is a significant area of open research. Alternatively, a mixed logic linear program can be solved directly using a hybrid decision algorithm, similar to SMT or OpSat, which coordinates a set of subsolvers that solve MILP, propositional satisfiability and finite domain constraint satisfaction problems [4]. There does not yet exist a mature set of hybrid solvers for MLLP.

Alternatively, a mixed logic temporal problem could be solved directly using a hybrid solver, along the lines of OpSat. Decision procedures currently exist for solving temporal problems involving a preference function over duration, both for problems with controlled duration and uncontrolled duration [25, 35, 16]. To employ these temporal solvers effectively within a hybrid solver, these sub-solvers should support incremental update and conflict extraction, similar to [34]. For temporal problems with preference, this is an area of open research.

13.2 Online PCTA Planning and Scheduling

A wide range of scheduling algorithms have been developed that are robust to uncertainty but reduce conservatism by scheduling on the fly. The PCTA formalism lends itself naturally to this dynamic scheduling approach. When a run produces a transition that resets a clock parameter, dynamic

scheduling selects the value of this clock parameter during execution rather than planning. A clock parameter value could be chosen during execution at the time that the run results in a transition that resets the parameter.

Alternatively, an online scheduler might incrementally tighten a parameter value range during execution, rather than select a value right away. This tightening is valid as long as it ensures that each evaluation of a clock condition is unambiguously true or false. This approach allows the scheduler to defer its final scheduling decisions to the last minute. For example, some dynamic schedulers, such as the foot placement executive [22], waits to decide on the duration of an automaton location until after that location is entered. It instead assigns an end time of the location at the moment that the location end is reached. Returning to the computer example, this might be represented by the parameter traces

$$p1_{cmp} = \langle \langle \text{not-yet}, 20 \rangle, \langle 20, 25 \rangle, \langle 35, \text{not-yet} \rangle, \langle 35, 110 \rangle \rangle \text{ and}$$

$$p2_{cmp} = \langle \langle \text{not-yet}, 5 \rangle, \langle 10, 15 \rangle, \langle \text{not-yet}, 50 \rangle, \langle 20, 70 \rangle \rangle.$$

The value "not-yet", from 20 to 25, indicates that a specific value for $p1_{cmp}$ has not been selected from 20 to 25. Instead it is restricted to not include a value that would result in an exit transition during that interval.

Appendix A. State and Temporal Constraints

In this appendix we review the state and temporal constraint satisfaction problems that are leveraged as sub-problems in this paper. A state constraint relates state variable values at each. A temporal constraint relates distinguished events of the state variable trajectories, for example, the start and end points of episodes.

Constraints We start with basic constraints. A constraint network is defined in terms of a set of variables (scope) and a relation over these variables.

Definition 24. Constraint Network

A constraint network (CN) $N = \langle V, C \rangle$ consists of

- a set of variables V . Each $v_i \in V$ has an associated domain $D(v_i)$, or range $R(v_i)$ when v_i is a state variable, and
- a set of constraints C . Each constraint $c = \langle SP, RL \rangle$, for $c \in C$, consists of
 - scope $Sp \subseteq V$, and
 - relation Rl , which is a set of assignments to Sp .

□

Definition 25 (Constraint Satisfaction). A feasible assignment of CN $N = \langle V, C \rangle$ is an assignment to variables V that satisfies each $c \in C$. A feasible assignment of N is called a solution to a constraint satisfaction problem with network N . If a feasible assignment exists then N is said to be satisfiable or consistent. □

A constraint network N is equivalent to the constraint $\langle V, \bowtie_i c_i \rangle$. N is satisfiable if $\Downarrow_{\{\}} \bowtie_i c_i$ is True. $\bowtie$ denotes the standard join operator for composing two constraints, while $\Downarrow_V$ denotes the standard operator for projecting a constraint on to a subset of its scope Sp .

Constraints over State Variables ctBurton constrains the trajectories of state variables using two types of constraints, *transition* and *state constraints*. A *transition constraint* relates the values of state variables between successive time steps, for example, times $t - \epsilon$ and $t + \epsilon$, for step size 2ϵ . A *state constraint* relates the values of state variables at a single point in time t . Since ctBurton uses a continuous model of time, we take the limit as $\epsilon \rightarrow 0^+$.

Definition 26. State and Transition Constraints

A state constraint $SC = \langle Sp, Rl \rangle$ is a constraint whose scope Sp is a set of state variables and relation relates values of these variables at the same point in time, that is, $Rl(v_1(t) \dots v_n(t))$.

A transition constraint is a constraint $TC = \langle Sp, Rl \rangle$. Its scope Sp is a set of state variables. Its relation is over state variables, evaluated for successive pairs of time points. Within a continuous model of time, the relation is of the form $Rl(v_1(t^-) \dots v_n(t^-), v_1(t^+) \dots v_n(t^+))$, where $t^- = t - \epsilon$ and $t^+ = t + \epsilon$, as $\epsilon \rightarrow 0^+$. The set of t^+ variables of transition TC is the set of state variables $v_i \in Sp$ that appear as $v_i(t^+)$ in relation Rl . The set of t^- variables of transition TC is the set of state variables $v_i \in Sp$ that appear as $v_i(t^-)$ in relation Rl . $\square$

Temporal Constraints for Certain Events Our review of temporal constraints is taken from [15]. Temporal constraints are used to restrict the times when events occur, where an event is a point in time. We start by considering deterministic events, whose times are under our control. In this paper we use events to mark the times that behavior changes, such as the change in value of a control variable or an automaton location, and to denote significant reference events, such as the start of a task or a holiday. Since we use a continuous model of time in this paper, these events are real-valued. Temporal constraints express relationships between these real-valued events. A simple, but expressive form of temporal constraint is a *simple temporal constraint*, which bounds the temporal distance between two events to be within a closed interval[14].

Definition 27. Temporal and Simple Temporal Network

A simple temporal network (STN) $N = \langle E, C \rangle$ is a constraint network consisting of

- a scope, which is comprised of real-valued events E , and
- a set of constraints, comprised of simple temporal constraints (STCs) C . Each STC $c = \langle X, Y, [l, u] \rangle$, for $c \in C$, is on the duration between pairs of events X and Y , and is of the form $Y - X \in [l, u]$, where $X, Y \in E$ and $l, u \in \mathbb{R}$. $Y - X$ is called a duration or temporal distance.

$\square$

A simple temporal constraint may be equivalently expressed by the two constraints $l \leq Y - X \leq u$, denoting lower bound l and upper bound u on $Y - X$. To execute a plan, its events are scheduled, while satisfying the plan's temporal constraints. For a plan to be consistent its events need to be schedulable.

Definition 28 (Schedule). A schedule of STN $N = \langle E, C \rangle$ is an assignment to events E that satisfies constraints C . A schedule of N is called a solution to a simple temporal problem with network N . If a schedule exists, then the STN is said to be consistent. $\square$

Temporal Constraints with Uncertain Events Next we incorporate uncertainty into simple temporal constraints. In a deterministic STN, given above, simple temporal constraints are used to represent both temporal requirements and the duration of actions. An implicit assumption is that the duration of actions, as well as requirements, are fully controllable; we may choose any value for an action duration that lies within its bound.

This model is insufficient for real world scenarios in which actions have uncontrollable duration; the values of these durations are chosen by nature. A temporal network should model these uncontrolled durations and the scheduler for this network should choose values for controllable durations while measuring success relative to the uncontrolled durations. Uncertain durations are commonly modeled using intervals or distributions. In the next two subsections we briefly summarize these two formalisms.

Simple Temporal Networks with Uncertainty A simple temporal network with uncertainty models each uncontrolled duration by an interval, representing the possible values that nature might choose for that duration [36] (Definition 29). An STNU differs from a STN in that events are divided into controlled and uncontrolled events. Likewise, simple temporal constraints are divided into requirement and contingent constraints. Both constraints bound the difference between events to an interval. However, while requirement constraints allow their difference to be chosen by a scheduler, contingent constraints represent the uncertain action durations that are chosen by nature.

Definition 29 (Simple Temporal Network with Uncertainty). *A simple temporal network with uncertainty (STNU) is defined as $N_u = \langle E_c, E_u, C_r, C_c \rangle$, where*

- E_c is a set of real valued controllable events that the agent can assign;
- E_u is a set of real-valued uncontrollable events assigned by nature;
- C_r is a set of requirement constraints, $Y - X \in [l, u]$, for events $X, Y \in E_c \cup E_u$ and $l, u \in \mathbb{R}$, and
- C_c is a set of contingent constraints, $Y - X \in [l, u]$ where $X \in E_c \cup E_u$, $Y \in E_u$, and $l, u \in \mathbb{R}^+$.

Y is called the terminating event. Every $Y \in E_u$ is a terminating event of exactly one $c \in C_c$.

A duration $Y - X$ is controllable if $X, Y \in E_c$ and uncontrollable, otherwise. $\square$

For an STNU, there is a 1-1 mapping between uncontrolled events E_u and contingent constraints C_c . A contingent constraint must terminate on an uncontrollable event. Conversely, every uncontrollable event has exactly one contingent constraint that terminates on it. The reason for this is that the duration associated with every contingent constraint corresponds to an independent stochastic process, hence it does not make sense for two contingent constraints to determine the value of the same event.

To characterize the behavior of an STNU for different values of uncontrolled events, we introduce the concept of outcome (Definition 30).

Definition 30 (STNU Outcome and Scheduling Policy). *An outcome ω of STNU N_u is a set of assignments to the durations of its contingent constraints C_c . $\omega_{X,Y}$ denotes the assignment $\omega(Y - X)$ to duration $Y - X$. A scheduling policy $\pi : \mathbb{R}^{|C_c|} \rightarrow \mathbb{R}^{|E_c|}$ for N_u assigns controllable events E_c based on outcomes ω for its contingent constraints C_c . A policy denotes a fixed schedule if $\pi(\omega) = \pi(\omega')$ for all outcomes ω and ω' . $\square$*

The concept of correct fixed schedule with respect to possible outcomes is captured by strong controllability (Definition 31). Intuitively, an STNU is strongly controllable if there exists a schedule such that, for any outcome of its contingent constraints, its set of requirement constraints is satisfied. There is thus at least one schedule that always “works”.

Definition 31 (STNU Strong Controllability). *Consider STNU $N_u = \langle E_c, E_u, C_r, C_c \rangle$. Let Ω be the set of outcomes for contingent constraints C_c . We say that N_u is strongly controllable if there exists a fixed schedule π such that for any $\omega \in \Omega$, all constraints in C_r are satisfied. $\square$*

Probabilistic Simple Temporal Networks with Risk Bounds A probabilistic Simple Temporal Network (pSTN) associates a probability distribution with each contingent duration, rather than an interval. A pSTN associates a probability with every outcome, where an STNU simply classifies each outcome as possible or not. In addition, for many real-world processes, risk of failure cannot be completely removed. To address this, many pSTN formulations augment the requirement constraints with an upper bound on the acceptable risk of violating these requirements (called a *chance constraint*). We start by defining probabilistic STN (Definition 32).

Definition 32 (Probabilistic Simple Temporal Network). *$\langle N_p = E_c, E_u, C_r, C_p \rangle$ defines a probabilistic simple temporal network (pSTN), with*

- E_c, E_u, C_c as in Definition 29, and
- C_p , a set of probabilistic contingent constraints.

For each $c_{XY} \in C_p$ from X to Y , there is a known probability distribution d such that $Y - X \sim d$. $\square$

A pSTN offers a well-defined joint distribution over outcomes. Using this joint distribution, we define probability of success in terms of the cumulative probability of a schedule’s outcomes satisfying the requirement constraints, where the schedule is generated by some policy.

Definition 33 (pSTN policy risk). *Consider pSTN $N_p = \langle E_c, E_u, C_r, C_p \rangle$. For policy π , let $\Omega_\pi \subseteq \Omega$ be the subset of the sample space in which some requirement constraint is violated, that is, for all $\omega \in \Omega_\pi$, there exists some constraint $c \in C_r$, where $c : Y - X \in [l, u]$, such that $Y(\omega) - X(\omega) \in [l, u]$, then $\omega \in \Omega_\pi$. The risk of policy π with respect to the set of requirement constraints C_r is, therefore, $\rho(\pi) = P(\Omega_\pi)$. $\square$*

Using this definition of policy risk, we define chance-constrained pSTN satisfaction as the problem of finding a policy whose risk is below an acceptable bound.

Definition 34 (Chance-Constrained Probabilistic Simple Temporal Problem cc-PSTP). *Given:*

- $N_p = \langle E_c, E_u, C_r, C_p \rangle$, a pSTN; and
- $\Delta \in [0, 1]$, an upper bound on the risk of failure.

Find:

- π , a scheduling policy for E_c based on outcomes of contingent constraint durations.

Subject to:

- $\rho(\pi) \leq \Delta$, the probability of policy risk being bounded by Δ .

N_p is said to be controllable with probability of failure at most Δ if and only if such a policy π exists. $\square$

While this problem definition supports both offline and online scheduling, this paper focuses only on offline scheduling of controllable events. Offline schedulability of a cc-pstn has a strong parallel to STNU strong controllability.

Finally, an optimal cc-pSTP adds an objective function to the problem formulation, which is calculated from the timing of the controllable events.

Definition 35 (Optimal cc-pSTP). *Given:*

- $N_p = E_c, E_u, C_r, C_p$, a pSTN; and
- $\Delta \in [0, 1]$, an upper bound on the risk of failure.

Find:

- π , a scheduling policy for E_c based on outcomes of contingent constraint durations.

Minimizing:

- $E[c(\pi)]$, the expected cost given the scheduling policy.

Subject to:

- $\rho(\pi) \leq \Delta$, the probability of policy risk being bounded by Δ .

N_p is said to be controllable with probability of failure at most Δ if and only if such a policy π exists. $\square$

Appendix B. Modelling Concurrency

In this appendix we extend the formalism of parameterized timed automata (PTA) to concurrent systems. While these definitions are essential, the extension of PTA to concurrent automata is straight forward.

B.1 Concurrent Automata

Timed automata are used to model systems at multiple levels of abstraction. To start, we model individual processes by introducing *primitive parameterized automata*, which have a single location and clock variable.

Definition 36. Primitive Parameterized Timed Automaton

A primitive parameterized timed automaton A is a parameterized timed automaton with a single location and single clock variable, that is, $\mathcal{L} = \{l\}$ and $\mathcal{C} = \{c\}$. $\square$

The state of a primitive automaton is an assignment to its location variable and its clock variable. In the projector example (Figure 1), the computer, connection and projector are modeled as primitive automata. The primitive automaton for projector has locations Off, WarmUp, On, Waiting,

Confirm, and CoolDown. We provided examples of transitions and transition specifications in Section 3.1. Transition guards for primitive automata are conditioned on input variables (denoted cmd), clock variables (denoted c) and clock parameters (denoted p), and the location variables, clock variables and clock parameters of other automata. As an example of the use of location variables within guards, in Figure 1 the projector transitions from On to Waiting and Waiting to Cooldown are conditioned on the location of the Computer not being On.

To model system behavior we compose multiple parameterized timed automata into a *concurrent parameterized timed automaton* (PCTA). A PCTA models a system using a state that is specified by assignments to location and clock variables, and through external inputs, including assignments of commands to *exogenous control variables* of the top-level system.

The behavior of a PCTA A results from interactions between its component automata. These interactions result from guarded transitions that change the location of one component automaton A_i through conditions on the location variables, clock variables and clock parameters of other component automata, as well as the clock variables and clock parameters of A_i and the inputs to A . The computer-projector system of Figure 1 is modeled as a PCTA with three component automata, each of which is a primitive automaton - computer, connection and projector. The PCTA has a single input, cmd .

Note that a component automaton of a PCTA may be a primitive automaton or a PCTA. If a PCTA is used to model an overall system, we refer to it as a *top-level automaton*.

Definition 37. Parameterized Concurrent Timed Automaton

A Parameterized Concurrent Timed Automaton $A = \langle \mathcal{U}_I, \mathcal{A} \rangle$ is a timed automaton whose behavior is specified by a finite set of concurrently operating component automata $\mathcal{A}$, which interact through input variables $\mathcal{U}$ and shared location variables $\mathcal{L}$, clock variables $\mathcal{C}$ and clock parameters $\mathcal{U}_P$.

- $\mathcal{U}_I$ is the finite set of (exogenous) input variables of A .
- $\mathcal{A} = \{A_i\}$ is the finite set of component automata, where each A_i is a timed automaton. Component variables $L_i, C_i, U_i, U_{I_i}, U_{I_{C_i}}, U_{P_i}, U_{I_{P_i}}, T_i$ and R_i are short for $\mathcal{L}(A_i), \mathcal{C}(A_i), \mathcal{U}(A_i), \mathcal{U}_I(A_i), \mathcal{U}_{I_C}(A_i), \mathcal{U}_P(A_i), \mathcal{U}_{I_P}(A_i), \mathcal{T}(A_i)$ and $\mathcal{R}(A_i)$, respectively. The input variables U_{I_i} of each A_i is a subset of $\mathcal{U}_I \cup \mathcal{L} - L_i$. That is, the input variables of A_i consists of A 's (exogenous) input variables and the location variables of other component automata. Likewise, the input clock variables $U_{I_{C_i}}$ of each A_i is a subset of $\mathcal{C} - C_i$, that is, the input clock variables of A_i consists of clock variables of other component automata. Finally, the input clock parameters $U_{I_{P_i}}$ of each A_i is a subset of $\mathcal{U}_P - U_{P_i}$, that is, the input clock parameters of A_i consists of clock parameters of other component automata.

The derived attributes of A are:

- $\mathcal{L} = \bigcup_i L_i$ is the finite set of location variables of A .
- $\mathcal{C} = \bigcup_i C_i$ is the finite set of non-negative, real-valued clock variables of A .
- $\mathcal{U}_P = \bigcup_i U_{P_i}$ is the finite set of real-valued clock parameters of A .
- Transition relation $\mathcal{T} = \{\tau_i\}$ of A is the composition $\prod_i T_i$ of the transition relations of each A_i .

- Each transition $\tau \in \mathcal{T}$ of A has an associated set of component transitions $\{\tau_i\}$, one for each A_i , where the composition of $\{\tau_i\}$ is satisfiable. $\tau = \langle l_s, l_t, g = g_s \wedge g_c, c_r \rangle$ is defined from $\{\tau_i\}$ as $l_s = \bigcup_i l_s(\tau_i)$, $l_t = \bigcup_i l_t(\tau_i)$, $g_s = \bigwedge_i g_s(\tau_i)$, $g_c = \bigwedge_i g_c(\tau_i)$, and $c_r = \boxtimes_i c_r(\tau_i)$. Transition specification $\langle ss, \varphi, ts, cs \rangle$ for τ is defined by conjoining elements of the specifications for τ_i , that is, $ss = \bigwedge_i ss(\tau_i)$, $tt = \bigwedge_i tt(\tau_i)$, $\varphi = \bigwedge_i \varphi(\tau_i)$, and $cs = \bigwedge_i cs(\tau_i)$.
- Each reward $\rho \in \mathcal{R}$ of A is the composition of respective rewards $\{\rho_i\}$ for components A_i . Component rewards are additive, that is, $\rho = \sum_i \rho_i$.

If A is not a component of some PCTA, then A is referred to as a top-level automaton. $\square$

We often explicitly refer to $\mathcal{U}_I$ as *exogenous* input variables, to avoid confusion with component input variables U_{I_i} for component A_i . The current state of PCTA A assigns each of its location and clock variables. It is simply the union of the current states of all its component automata A_i .

A 's transition relation $\mathcal{T}$ is the join of the transition relations of A 's component automata. ct-Burton requires each component automaton A_i of PCTA A to be deterministic, with the result that A is deterministic. Each transition τ in $\mathcal{T}$ is a *consistent composition* of component transitions, one component transition from each component automaton. A composition may be inconsistent, for example, due to inconsistent assignments to sources, targets or resets, or because of a joint guard that is unsatisfiable. τ is applicable, enabled and satisfied at time t exactly when its component transitions are all applicable, enabled and satisfied at t , respectively.

Transition τ 's source, target and clock variable reset assignments are the unions of the respective assignments of τ 's component transitions. Likewise, τ 's clock parameter reset is the join of the respective resets of τ 's component transitions. In addition, τ 's state and clock conditions are the conjunction of the respective conditions of τ 's component transitions.

When A is a top-level automaton, its inputs are the system control variables μ , whose assignments may include commands. The inputs to A 's component automata are a mix of these control variables and the location variables, clock variables and clock parameters of the other component automata.

B.2 Concurrent Traces and Runs

We introduced clock parameters in Section 3.1 so that an activity planner had the flexibility to schedule the duration of actions, allowing it to satisfy coordination constraints on timing. We then introduced concurrent automata in Appendix B.1, to model and plan for the coordination of simultaneous actions by multiple agents, devices and physical processes.

A concurrent automaton, with its simultaneously executing component automata, offers a factored description of a parameterized timed automaton. In this section we provide factored descriptions of traces, runs, state sequences and trajectory sequences for these concurrent automata. These concepts are analogous to those presented in Section 5.1.

A trajectory is an assignment to a vector of variables V . A *concurrent trajectory* splits this trajectory into a set of trajectories by partitioning V , where each partition is comprised of one or more variables, and by specifying a trajectory for each partition.

A concurrent trace instantiates this concept for the control variables of a timed automaton. This includes input variables of the TCA, input variables of each component automata, and clock param-

eters of the component automata. The concurrent trace simply partitions a trace according to the variables of each component automaton.

Definition 38. Concurrent Trace

Let $\{V_i\}$ be a partition of variables V . A concurrent trajectory of $\{V_i\}$ over $[t_0, t_n] \subseteq \mathbb{R}$ is a set of trajectories $V(t) = \{V_i(t)\}$ over $[t_0, t_n]$, called component trajectories, one for each V_i . Let $\{U_i\}$ be a partition of control variables $\mathcal{U}$ of timed automaton A . A concurrent trace of $\{U_i\}$ over $[t_0, t_n] \subseteq \mathbb{R}$ is a concurrent trajectory $\mathcal{U}(t) = \{U_i(t)\}$ of $\mathcal{U}$, such that $\mathcal{U}(t)$ forms a trace of A . $\{U_i(t)\}$ is complete if each component trace $U_i(t)$ is complete; otherwise, $\{U_i(t)\}$ is partial. $\square$

We likewise describe a PCTA run as a concurrent set of runs, one for each component automaton.

Definition 39. Concurrent Run

Let $A = \langle \mathcal{U}, \{A_i\} \rangle$ be a parameterized timed concurrent automaton with control variables $\mathcal{U}$, state variables $\mathcal{V}$, and component automata A_i , where U_i and V_i are the control and state variables of A_i . Let $\mathcal{U}(t)$ be a trace of $\mathcal{U}$ over $[t_0, t_n] \subseteq \mathbb{R}$ and $s_0 = \bigcup s_{0_i}$ be an initial state of A at t_0 . A run of A over $[t_0, t_n]$, for s_0 and trace $\mathcal{U}(t)$, is a trajectory $\mathcal{V}(t)$ over $[t_0, t_n]$ such that for each component automaton A_i , $V_i(t)$ is a valid run of A_i over $[t_0, t_n]$, for s_{0_i} and trace $U_i(t)$. The set of component runs $\{V_i(t)\}$ is called a concurrent run of A over $[t_0, t_n]$. $\mathcal{V}(t)$ is complete if the trace $\mathcal{U}(t)$ and every component run $V_i(t)$ is complete; otherwise, $\mathcal{V}(t)$ is partial. $\square$

B.3 Describing Concurrent Traces and Runs

To describe concurrent trajectories, traces and runs, we generalize timed sequence to *concurrent timed sequence* in a straight forward manner.

Definition 40. Concurrent Timed Sequence

A concurrent timed sequence $\{T_i\}$ of sets $\{Q_i\}$ over $[t_0, t_n] \subseteq \mathbb{R}$ is a set of timed sequences T_i over $[t_0, t_n]$. $\{T_i\}$ is concise if and only if every T_i is concise.

Let $\{V_i(t)\}$ be a set of concurrent trajectories over $t \in [t_0, t_n] \subseteq \mathbb{R}$. A concurrent timed state sequence of $\{V_i(t)\}$ over $[t_0, t_n]$, is a set $S = \{S_i\}$, where S_i is a timed state sequence of $V_i(t)$ over $[t_0, t_n]$. $\{V_i(t)\}$ is called the concurrent trajectory of S . $\square$

We use concurrent timed state and transition sequences to describe PCTA runs, which describe state changes or transitions of component automata. As with PTA, runs are specified over location variables, while clock trajectories are implied by these location trajectories.

Definition 41. Concurrent Timed Sequence of an Automaton

Let A be a concurrent timed automaton with component automata $\{A_i\}$ and associated state and location variables, $\{V_i\}$ and $\{L_i\}$. Let $\{V_i(t)\}$ be a concurrent run of A that is complete over $[t_0, t_n] \subseteq \mathbb{R}$.

A concurrent timed state sequence of $\{V_i(t)\}$ over $[t_0, t_n]$ is a set $\{S_i\}$, where S_i is a timed state sequence of $L_i(t)$ over $[t_0, t_n]$, and $\{V_i(t)\}$ is the concurrent trajectory of $\{S_i\}$.

A concurrent timed transition sequence of $\{V_i(t)\}$ over $[t_0, t_n]$ is a set $\{S'_i\}$, where S'_i is a timed transition sequence of $V_i(t)$ over $[t_0, t_n]$ and $\{V_i(t)\}$ is the concurrent trajectory of $\{S'_i\}$. $\square$

Appendix C. Factored Problem Encoding Based on Concise Histories

Defer to another paper.

- Think through histories for clock offsets and clock parameters.

C.1 The Impact of Using Global State Sequences

We call our encoding of the last section a *totally ordered trellis* as it imposes a total ordering on state changing events. Intuitively, each stage ends as soon as any location variable changes value, hence a sequence of stages specifies the order in which any two value changes occur. An effect of this encoding is that the value of many location variable from one stage to the next will not have changed, on average, and are carried forward to subsequent stages. The stage's end event is thus superfluous to these unchanged variables. Values that are duplicated across stages are propagated through transition as inputs to additional component automata, where additional unchanged values are duplicated. Each duplicated value corresponds with a variable that was unnecessarily duplicated.

In the worst case, imposing this total ordering can result in an exponential number of orderings that have no impact on automaton behavior, hence an exponential number of candidate plans that are not meaningfully different. To ground this argument, suppose our concurrent automaton has n primitive automata and we want to generate a run that tracks each automata across m value changes. These value changes are not synchronized and may occur at different times. As a result, our trellis encoding for this run would need $n \times m$ stages. If our encoding of each component automaton employed l decision variables per stage, our trellis encoding would use $l \times n^2 \times m$ variables. ctBurton's space of candidate plans is exponential in the number of variables, hence the influence of duplicated variables is significant.

While a representation involving a total ordering may be efficient for concurrent systems in which all components are tightly coupled, many systems may only involve weak couplings, and a more compact encoding is desired. As an extreme example, suppose that the runs of the component automata in our example are independent of each other. In this case it is sufficient to construct a separate trellis for each component automaton. This produces n independent trellises, each having l variables per stage and m stages. The total encoding size is $l \times n \times m$, which is a factor of n smaller than the example above. This reduces the exponent in the candidate space size by a factor of n .

C.2 Factored Encoding

In this section we introduce a more compact encoding for planning concurrent activities, which we call a *partially ordered trellis*. Our first step is to construct a separate trellis for each component automaton, called a *factor*, similar to our example. In this formulation no two stages from different component automata share a common event. Consequently, the i th stage of one component has a different event from the i th stage of all other automata and can be assigned a different duration from the other parallel stages. ctBurton chooses the end time of each component stage to correspond to a value change in the component's location variable, resulting in a succinct description of the run of that component.

Next, to represent weak couplings without imposing a total ordering, we note that an automaton is only influenced directly by its inputs, which may be exogenous inputs or location variables of other component automata. Hence, our second step is to augment the set of component trellises with

a set of coupling constraints between the location variable of each component and the component's inputs.

The source of inspiration for our factored encoding is the concept of *concise histories* [42]. We review this concept in the next section, and then present our encoding in the subsequent section.

C.2.1 CONCISE VALUE AND JUSTIFICATION HISTORIES

Discrete event simulators typically represent a simulation result as a state sequence, which imposes a total ordering on events, similar to a candidate generated from a totally ordered trellis. A *History-based simulator* [42] represents the behavior of each state variable individually using a *value history* and describes how each value history is influenced by the histories of other variables using *justification histories*. Justification histories have the added benefit that they can be used to generate explanations for simulation results, for example, within *self-explanatory simulators* [17].

Following [42], a *value history* for variable x_i is composed of *value episodes*, each consisting of an assignment $x_i = v_{ij}$ that holds over a time interval (e^-, e^+) , called an *extent*. e^- and e^+ are real-valued and denote *start* and *end* events of the extent. Successive episodes in a history meet, that is, when an episode ends, its successor begins. A history that eliminates superfluous events is called a *concise history*. A concise value history is composed of *maximal value episodes*, where a value episode with assignment $x_i = v_{ij}$ and extent (e^-, e^+) is *maximal* if there is no proper superset of extent (e^-, e^+) in which $x_i = v_{ij}$ holds.

Justification histories represent the relevant couplings between value histories. Suppose variable x_i is determined by assignments to a set of input variables I_i . A justification history for x_i represents this coupling throughout the value history and is comprised of a sequence of *justification episodes* that meet. Each justification episode represents a particular influence over a bounded interval of time and is similar to a value episode. A justification episode is comprised of an assignment to both x_i and inputs I_i such that the assignment to I_i implies the assignment to x_i , and each assignment holds throughout the extent of the justification episode. A justification episode is maximal if its extent is the largest extent over which the assignment to I_i implies the assignment to x_i and a justification history is *concise* if it is composed of maximal justification episodes.

It is important to note that a maximal justification episode can be equivalently represented by a set of maximal value episodes, one for x_i and each I_i . The extent of the justification episode is the intersection of the extents of the value episodes of I_i . This intersection must be non-empty. In addition, the justification episode's extent must be a subset of the extent of x_i 's value episode. This alternative representation of a justification episode makes explicit the coupling between a value episode and its corresponding input value episodes. It will play an important role in our next section, when we augment our trellis encoding with coupling constraints.

Finally, note that concise histories offer a parsimonious representation that only introduces events that denote changes in variable values (i.e., the value episodes) and changes in the way that a set of values influence another variable (i.e., the justification episodes). In addition, events in this representation are only partially ordered. Specifically, the set of events associated with each history is totally ordered and justification episodes relate the events of relevant episodes across histories, but outside of this, events across histories are not ordered.

Provide examples throughout the description above

C.2.2 A PARTIALLY ORDERED TRELLIS BASED ON HISTORIES

in this and the next section we introduce a factored, partially ordered trellis that mimics the structure of value and justification histories. By the end of the next section we produce an encoding comprised of a trellis for each state variable whose assignments denote concise value histories, and a trellis for each component automaton whose assignments denote concise justification histories. This formulation has the added benefit that location variables may have multiple idling transitions and a value may be sustained by a sequence of idling transitions.

In this section we present a more compact encoding that merges a value history with its justification history, but under our assumption that each automaton location has a single idling transition, which can sustain the location.

To start we note that a trellis for a single automaton represents a concise value history for its location variable *plus* an associated justification history, over its input variables. Each stage of this trellis denotes a maximal value episode *plus* its maximal justification episode. To elaborate, recall that a stage has an extent that is defined by its start and end events. The stage includes an assignment to its location variable(s) and a set of input variables. This assignment satisfies the state condition of an enabled transition. The stage also includes assignments to clock offsets and parameters, which satisfy the clock condition of the transition guard. We focus on the state condition and input assignment, since the state condition represents the transition’s coupling to other state variables.

Taken together, a stage’s location variable assignment and extent constitute a maximal value episode. The episode is maximal since the location variable assignment is different in the preceding and succeeding stages. On the other hand, the stages extent and assignments to the location variable and input variables constitute a maximal justification episode. It is a justification episode because the input assignment determines the location assignment, through the state condition of an idling transition. It is maximal because at least one of the inputs changes value at the start of the extent. Likewise, one input changes value at the end of the extent.

Consider why we associate a stage with both a value episode *and* a justification episode. We have assumed that a single idling transition maintains a location from when it is entered until when it is departed. A consequence of this assumption is that each maximal value episode has a single justification episode that holds throughout the value episode’s extent. We relax this assumption in our next section.

Next we consider how to express coupling constraints between each component trellis and its input histories. Our encoding relies on having a concise value history for each input. For inputs that are location variables, their component trellis describes a concise value history. For inputs $u \in \mathcal{U}_{\mathcal{I}}$ that are exogenous variables, we introduce a new type of trellis, called a *variable trellis*, which describes a concise value history for those variable.

A variable trellis consists of a single input variable as a state variable, it does not include any other variables, such as location variables, clock offsets or clock parameters. The assignment to this variable must be different for successive stages, to ensure that each stage denotes a maximal value episode. A variable trellis for variable v is the sequence $\langle e_1, v_1, \dots, e_h, v_h, e_{h+1} \rangle$, where $\langle e_i, v_i \rangle$ is the i th stage, e_i is the i th real-valued event and v_i is the assignment to v from event e_i to event e_{i+1} . Successive values of v_i are distinct and events monotonically increase in value, that is,

$$\forall 0 < i < h . u_i \neq u_{i+1} \wedge e_i < e_{i+1}. \quad (25)$$

Given a trellis for each component automaton and each exogenous input, we add a set of coupling constraints between each component trellis and the trellises of its inputs. At the end of the previous section (Section C.2.1), recall that we described a justification episode in terms of a set of input value episodes and the intersection of their extents.

A justification episode is defined by a set of value episodes, one for each of its inputs. Both the justification episode and the input value episodes have corresponding stages, hence forward we treat them as synonymous. The input assignments of the justification episode and the input value episodes must agree on their respective variables. In addition, each input value episode, hence its stage, must hold throughout the extent of the justification episode. Third, the justification history starts at the beginning of each input value history, hence the input value episodes of the first justification episode are the first episode in their respective histories. Finally, given a justification episode J and its successor J' , each input value episode of J or its successor appears as an input value episode of J' .¹¹

Specifically, for each stage i of a component trellis, ctBurton introduces a set of variables $\mathcal{S}_i$ called *input stages*, such that each component input variable $u_{cj} \in \mathcal{U}_c$ has a corresponding input stage $s_{ij} \in \mathcal{S}_i$ for stage i . s_{ij} is assigned an index that points to a stage of the trellis $Tr(u_{cj})$ of u_{cj} . In addition, ctBurton introduces four types of constraints, relating stage i to its input stages. First, ctBurton constrains the assignment to each input $u_{cj} \in \mathcal{U}_c$ of stage i to be the same as the assignment to u_{cj} for its input stage s_{ij} ,

$$\forall u_{cj} \in \mathcal{U}_c. u_{cj}(i) = u_{cj}(s_{ij}). \quad (26)$$

Second, ctBurton constrains the extent of stage i to be a subset of the extent of each of its input stages s_{ij} ,

$$\forall s_{ij} \in \mathcal{S}_i. (e_i^-, e_i^+) \subseteq (e^-(s_{ij}), e^+(s_{ij})). \quad (27)$$

Third, when stage i is the first stage of its trellis ($i = 1$), ctBurton constrains each input stage s_{ij} of i to be the first stage of u_{cj} 's trellis ($Tr(u_{cj})$),

$$\forall s_{1j} \in \mathcal{S}_1. s_{1j} = Tr(u_{cj}, 1). \quad (28)$$

Finally, when stage i has a successor, that is, it is not the last stage h , ctBurton constrains each input stage $s_{i+1,j}$ of i 's successor ($i + 1$) to be a) the input stage $s_{i,j}$ of i or b) the successor of input stage $s_{i,j}$,

$$\forall 1 \leq i < h. s_{i+1,j} = s_{i,j} \vee s_{i+1,j} = Tr(u_{cj})(s_{i,j} + 1). \quad (29)$$

C.3 Planning with Sequences of Idling Transitions

Thus far we've assumed that a location is maintained by a single idling transition. We can imagine several real-world problems in which a value is maintained through a sequence of different influences. Consider a common sense example. A 150 lb shelf needs to be held in place, while it is glued to the wall. The shelf takes 20 minutes for the glue to set. A worker can hold the shelf up for at

11. To ensure that a stage denotes a valid, maximal justification episode, ctBurton must ensure that the extents of the input value episodes have a non-empty intersection and the extent of the justification episode equals this intersection. This first condition is implied by the constraint that the extent of the stage contains the extent of each input stage. The second condition is implied by the constraint that the stage ends with a change in location value, hence it corresponds to a maximal value episode.

most 15 minutes. To hold the shelf in place, worker A holds the shelf for 10-15 minutes, and then worker B holds the shelf for the remaining time.

If we model this example with a concurrent timed automaton, the shelf and two workers are each modeled as component automata. the shelf position is a location in its automaton, and it has two idling transitions. The first, is guarded by worker A having strength (a location in its automaton) and the second is guarded by worker B having strength. The run that describes our scenario sustains the shelf location through the two idling transitions in sequence.

Idling transitions with limited hold time are common. There are two reasons why idling transitions often can not be sustained indefinitely. First, an idling transition may have a clock condition that bounds duration. For example, when running water into a bath tub, there is a time when it over flows. Second, an assignment in an idling transition's state condition may be sustainable for only a limited time. We saw this in the example of the worker holding the shelf.

To model this behavior, in our problem input we allow an automaton to have multiple idling transitions per location, as long as the automaton remains determinate. This relaxes our single restriction made earlier, that an automaton has a single idling transition. Then to be complete, we allow our problem output to generate a trace whose run is produced in part by multiple idling transitions in sequence.

Next, we augment both our total order and partial order trellis encodings to allow for multiple idling transitions between state changes.

C.3.1 A TOTAL ORDER TRELLIS WITH MULTIPLE IDLING TRANSITIONS

To handle a sequence of idling transitions within our total order trellis, we modify the constraint on a stage end event so that a stage ends when any new transition is enabled, not just a state changing transition. We present the rationale for this change following by a modified constraint that captures this concept.

Important properties of a total order trellis include, first, values and enabled transitions are maintained throughout each stage, second, at least one value changes between stages, and, finally, for this value to change, at least one state changing transition occurs between stages.

To allow a value to be maintained by a sequence of idling transitions, we preserve the property that enabled transitions are maintained across a stage, but we drop the constraint that a value must change between stages, hence the requirement that a state changing transition occurs.

It is important that we preserve the concept that stages are delineated by events where behavior changes; otherwise, our candidate trace and run can arbitrarily introduce a potentially infinite number of events that are merely syntactic, and do not impact the candidate's behavior. The candidate space then explodes.

Behavior is influenced at times when transitions become enabled or disabled, irrespective of whether it produces a value change. The property of our previous encoding, that values change at each event, is simply a consequence of this insight, plus the constraint that only one idling transition is permitted per location. In our new encoding we enforce the constraint:

One transition is enabled at the end event of each stage.

This could be implemented by comparing the enablement of every transition between during and at the end of each stage. However, we know that state changing transitions must be disabled during a stage. Instead, we require that a state changing transition occurs at a stage's end event, as in our

previous encoding, *or* an idling transition is newly enabled at this end event. We capture this by replacing Constraint 20 of Section 9.1.3 with

$$\text{concise}(i) ::= \bigvee_{\tau_k \in T_{sc}(A)} \text{En}_E(\tau_k, i) \vee \bigvee_{\tau_k \in T_{id}(A)} \neg \text{En}_O(\tau_k, i) \wedge \text{En}_E(\tau_k, i). \quad (30)$$

where $T_{sc}(A)$ is the set of state changing transitions of A and $T_{id}(A)$ is the set of idling transitions of A .

The impact of this new encoding is that stages are punctuated by changes in the transitions that are enabled and values may be sustained across stages through a sequence of distinct idling transitions.

C.3.2 A PARTIAL ORDER TRELLIS WITH MULTIPLE IDLING TRANSITIONS

To handle a sequence of idling transitions within our partial order trellis, we modify our encoding so that every variable has a variable trellis, encoding a concise value history, and every location variable has a component trellis, encoding a concise justification history. coupling constraints are then introduced between trellises encoding value and justification histories, through the addition of constraints between each location variable trellis and its corresponding component trellis. We first motivate this approach, and then present our newly added coupling constraints.

To introduce sequences of idling transitions into our candidates, we start by implementing each component trellis using our modified, total order encoding. The net effect of this change is that a stage in the component trellis still represents a maximal justification episode, but no longer a maximal value episode, since the successor stage may have the same value for its location variable as the preceding stage.

This change is key, since our coupling constraints between a component trellis and its inputs relied on the fact that the trellis of each input was an explicit representation of a concise value history for that input.

To use these coupling constraints from before, in our modified encoding, we need a mechanism that generates this explicit representation. To accomplish this, we introduce a separate variable trellis for each location variable, which represents a concise value history. We then use this variable trellis, instead of its corresponding component trellis, for each input location variable that appears in our coupling constraints.

We then introduce a new set of coupling constraints between each location variable trellis and its component trellis, whose purpose is to extract a concise value history from the component trellis.

The result of this modification is that each component trellis represents a concise justification history, but no longer a concise value history, and each variable trellis represents a concise value history, as described in the introduction to this section.

The coupling constraints between a variable trellis and its component trellis associates each component trellis stage, denoting a justification episode, with a variable trellis stage, denoting a value episode, such that the justification episode supports the value episode. A justification episode supports a value episode if they agree on the assignment to their shared variable and the extent of justification episode is a subset of the extent of the value episode.

For each stage i of a component trellis ctBurton establishes this correspondence as follows. First ctBurton adds to stage i a variable $s_i^{\mathcal{L}}$, called a *location stage*, which is assigned an index that points to a stage in the variable trellis of $\mathcal{L}$ that stage i supports. In addition, ctBurton introduces four

constraints, relating stage i to its location stage. First, ctBurton constrains the assignment to $\mathcal{L}$ in stage i to be the same as the assignment to $\mathcal{L}$ in its location stage $s_i^{\mathcal{L}}$,

$$\mathcal{L}(i) = \mathcal{L}(s_i^{\mathcal{L}}). \quad (31)$$

Second, ctBurton constrains the extent of stage i to be a subset of the extent of its location stage $s_i^{\mathcal{L}}$,

$$(e_i^-, e_i^+) \subseteq (e^-(s_i^{\mathcal{L}}), e^+(s_i^{\mathcal{L}})). \quad (32)$$

Third, when stage i is the first stage of its trellis ($i = 1$), ctBurton constrains its location stage $s_i^{\mathcal{L}}$ to be the first stage of $\mathcal{L}$'s trellis ($Tr(\mathcal{L})$),

$$s_1^{\mathcal{L}} = Tr(\mathcal{L}, 1). \quad (33)$$

Finally, when stage i has a successor, that is, it is not the last stage h , ctBurton constrains the location stage $s_{i+1}^{\mathcal{L}}$ of i 's successor ($i + 1$) to be a) the location stage $s_i^{\mathcal{L}}$ of i or b) the successor of location stage $s_i^{\mathcal{L}}$,¹²

$$\forall 1 \leq i < h . s_{i+1}^{\mathcal{L}} = s_i^{\mathcal{L}} \vee s_{i+1}^{\mathcal{L}} = Tr(\mathcal{L})(s_i^{\mathcal{L}} + 1). \quad (34)$$

C.4 Appendix: Histories

Need for a paper on the factored encoding.

Define value and justification histories succinctly.

C.5 Generating Explanations and Proofs of Correctness

Defer to another paper.

As discussed, a stage represents a justification episode for the state condition of an idling transition. To construct a complete proof for a set of value histories, this description would need to be augmented with justification episodes for the clock condition of each idling transition, and analogous justification episodes for the state changing transitions.

Appendix D. A Future Hybrid MLTP Solver

Defer to another paper.

- Coordinate hybrid solver similar to DLP-BB[Li-DLP-BB].
- Cangen's clauses include atomic propositions, variable assignments and difference constraints.
- Child solver solves STPPS incrementally and with conflict extraction.
 - An STPP is expressed as a difference LP.
 - the difference LP solver performs a specialized version of simplex on a distance graph.
 - think through incremental.
 - think through extraction of suboptimality and infeasibility conflicts.

12. An optimization is to select the first disjunct if the stages agree on value and select the second disjunct if they don't agree.

References

- [1] Rajeev Alur and David L Dill. “A theory of timed automata”. In: *Theoretical computer science* 126.2 (1994), pp. 183–235.
- [2] C. Bäckström and B. Nebel. “Complexity results for SAS+ planning”. In: *Computational Intelligence* 11.4 (1995), pp. 625–655.
- [3] Andrea Bauer et al. “The Autonomous City Explorer: Towards Natural Human-Robot Interaction in Urban Environments”. In: *International Journal of Social Robotics* 1.2 (Apr. 2009), pp. 127–140.
- [4] Alberto Bemporad and Nicolo Giorgetti. “Logic-Based Solution Methods for Optimal Control of Hybrid Systems”. In: *Automatic Control, IEEE Transactions on* 51 (July 2006), pp. 963–976. DOI: 10.1109/TAC.2006.876949.
- [5] J Benton, Amanda Jane Coles, and Andrew Coles. “Temporal Planning with Preferences and Time-Dependent Continuous Costs.” In: *ICAPS*. Vol. 77. 2012, p. 78.
- [6] Jingkai Chen, Brian C. Williams, and Chuchu Fan. “Optimal Mixed Discrete-Continuous Planning for Linear Hybrid Systems”. In: *The 24th ACM International Conference on Hybrid Systems: Computation and Control (HSCC 2021)*. 2021.
- [7] Steve Chien et al. “ASPEN—automated planning and scheduling for space mission operations”. In: *Space Ops*. 2000, pp. 1–10.
- [8] Seung Hwa Chung and Brian C Williams. “A decomposed symbolic approach to reactive planning”. PhD thesis. Massachusetts Institute of Technology, Department of Aeronautics and Astronautics, 2003.
- [9] A. Coles et al. “Managing concurrency in temporal planning using planner-scheduler interaction”. In: *Artificial Intelligence* 173.1 (2009), pp. 1–44.
- [10] AJ Coles et al. “Temporal planning in domains with linear processes”. In: *Proceedings of the International Joint Conference on Artificial Intelligence (IJCAI)*. 2009.
- [11] Amanda Jane Coles et al. “Forward-Chaining Partial-Order Planning.” In: *ICAPS*. 2010, pp. 42–49.
- [12] William Cushing, Subbarao Kambhampati, Daniel S Weld, et al. “When is temporal planning really temporal?” In: *Proceedings of the 20th international joint conference on Artificial intelligence*. Morgan Kaufmann Publishers Inc. 2007, pp. 1852–1859.
- [13] R. Dechter. *Constraint processing*. Morgan Kaufmann, 2003.
- [14] R. Dechter, I. Meiri, and J. Pearl. “Temporal constraint networks”. In: *Artificial intelligence* 49.1-3 (1991), pp. 61–95.
- [15] Cheng Fang, Andrew J. Wang, and Brian C. Williams. “Chance-constrained Static Schedules for Temporally Probabilistic Plans”. In: *Journal of Artificial Intelligence Research* 75 (2022), pp. 1323–1372.
- [16] Cheng Fang, Peng Yu, and Brian C. Williams. “Chance-Constrained Probabilistic Simple Temporal Problems”. In: *Proceedings of the Twenty-Eighth AAAI Conference on Artificial Intelligence*. Quebec City, Canada, Aug. 2014.

- [17] Kenneth D. Forbus and Brian Falkenhainer. “Self-Explanatory Simulations: An Integration of Qualitative and Quantitative Knowledge”. In: *AAAI Conference on Artificial Intelligence*. 1990.
- [18] Jeremy Frank and Ari Jónsson. “Constraint-based attribute and interval planning”. In: *Constraints* 8.4 (2003), pp. 339–364.
- [19] W. Hamscher, L. Console, and J. De Kleer. *Readings in Model-based Diagnosis*. Morgan Kaufmann Publishers, 1992. ISBN: 9781558602496. URL: <https://books.google.com/books?id=eHpRAAAAMAAJ>.
- [20] M. Helmert. “The fast downward planning system”. In: *Journal of Artificial Intelligence Research* 26.1 (2006), pp. 191–246.
- [21] A.G. Hofmann. “Robust execution of bipedal walking tasks from biomechanical principles”. PhD thesis. Massachusetts Institute of Technology, 2005.
- [22] A.G. Hofmann and B.C. Williams. “Exploiting spatial and temporal flexibility for plan execution of hybrid, under-actuated systems”. In: *Proc. AAAI*. 2006.
- [23] John N. Hooker and Maria Auxilio Osorio Lama. “Mixed Logical-linear Programming”. In: *Discret. Appl. Math.* 96-97 (1999), pp. 395–442. DOI: 10.1016/S0166-218X(99)00100-6. URL: [https://doi.org/10.1016/S0166-218X\(99\)00100-6](https://doi.org/10.1016/S0166-218X(99)00100-6).
- [24] Michel Donald Ingham. “Timed model-based programming: Executable specifications for robust mission-critical sequences”. PhD thesis. Massachusetts Institute of Technology, 2003.
- [25] Lina Khatib et al. “Temporal Constraint Reasoning With Preferences”. In: *International Joint Conference on Artificial Intelligence*. 2001. URL: <https://api.semanticscholar.org/CorpusID:4525492>.
- [26] T. Léauté and B.C. Williams. “Coordinating agile systems through the model-based execution of temporal plans”. In: *Proceedings of the National Conference on Artificial Intelligence*. Vol. 20. 1. Menlo Park, CA; Cambridge, MA; London; AAAI Press; MIT Press; 1999. 2005, p. 114.
- [27] Hui Li. “Kongming: A Generative Planner for Hybrid Systems With Temporally Extended Goals”. PhD thesis. Massachusetts Institute of Technology, 2010.
- [28] Nicola Muscettola, Paul Morris, and Ioannis Tsamardinos. “Reformulating temporal plans for efficient execution”. In: *In Principles of Knowledge Representation and Reasoning*. Citeseer. 1998.
- [29] P. Pandurang Nayak and Brian C. Williams. “Fast Context Switching in Real-Time Propositional Reasoning”. In: *Proceedings of the National Conference on Artificial Intelligence*. 1997, pp. 50–56.
- [30] Robert Ragno. “Solving Optimal Satisfiability Problems Through Clause-Directed A*”. MA thesis. Massachusetts Institute of Technology, May 2002.
- [31] Marlyse Reeves. “Magellan: A Robust Executive Enabling Long Horizon Multi-Agent Campaigns”. MA thesis. Massachusetts Institute of Technology, 2020.
- [32] Gabriele Röger, Patrick Eyerich, and Robert Mattmüller. *TFD: A numeric temporal extension to Fast Downward. IPC 2008 short papers*. 2008.

- [33] J. Shah et al. “A fast incremental algorithm for maintaining dispatchability of partially controllable plans”. In: *Proceedings of the 17th International Conference on Automated Planning and Scheduling (ICAPS-07)*. 2007, pp. 296–303.
- [34] I. Shu, R. Effinger, and B. Williams. “Enabling fast flexible planning through incremental temporal reasoning with conflict extraction”. In: *Proce. ICAPS-05, Monterey* (2005).
- [35] Kristen Venable and Neil Yorke-smith. “Simple Temporal Problems with Preferences and Uncertainty”. In: (June 2003).
- [36] T. Vidal and H. Fargier. “Handling contingency in temporal constraint networks: from consistency to control-abilities”. In: 11.1 (1999), pp. 23–45.
- [37] Andrew J. Wang and Brian C. Williams. “Chance-Constrained Scheduling via Conflict-Directed Risk Allocation”. In: *Proceedings of the Twenty-Ninth AAAI Conference on Artificial Intelligence*. Jan. 2015.
- [38] David Wang and Brian C. Williams. “tBurton: A Divide and Conquer Temporal Planner”. In: *Proceedings of the Twenty-Ninth Conference on Artificial Intelligence*. Austin, Texas, Jan. 2015.
- [39] B.C. Williams and P. Nayak. “A reactive planner for a model-based executive”. In: *International Joint Conference on Artificial Intelligence*. Vol. 15. LAWRENCE ERLBAUM ASSOCIATES LTD. 1997, pp. 1178–1185.
- [40] B.C. Williams and P.P. Nayak. “A model-based approach to reactive self-configuring systems”. In: *Proceedings of the National Conference on Artificial Intelligence*. 1996, pp. 971–978.
- [41] B.C. Williams and R.J. Ragno. “Conflict-directed A* and its role in model-based embedded systems”. In: *Journal of Discrete Applied Mathematics*. Citeseer. 2003.
- [42] Brian C. Williams. “Doing Time: Putting Qualitative Reasoning on Firmer Ground”. In: *Proceedings of the National Conference on Artificial Intelligence*. Philadelphia, PA, 1986, pp. 105–113.
- [43] Feng Zhao. “Automatic analysis and synthesis of controllers for dynamical systems based on phase-space knowledge”. PhD thesis. Massachusetts Institute of Technology, 1992.